MAPX

COMPLETE DEVELOPMENT ROADMAP

Native Android - Kotlin - Jetpack Compose - MapLibre Native - offline-first geographic briefing and presentation system

Roadmap version	1.0
Date prepared	4 November 2026
Authority	Master Project Specification v1.0 > UI/UX Design Guide v1.0 (incl. Layout Blueprint, BA-01..BA-17)
Built for	Windows 11 Home PC (i7-9700*, 16 GB, RTX 2070 8 GB, ~50 GB free, slow internet); Lenovo Tab P12 (primary), Samsung Galaxy A16; free-tier and local AI only
Companion file	MapX_Progress_Tracker.md (same phase/stage/step IDs)
Status	PLANNING DOCUMENT. Nothing described here has been built or tested.

* CPU identified from a screenshot; verified in step 01.1.1.

Contents

How to use this roadmap 5

Overview diagram . . . 5
Phase summary . . . 5
Architecture and layout at a glance . . . 7
AI workflow loop . . . 7
Disk budget . . . 7
Key decisions made in this roadmap . . . 8

PHASE 01 — ENVIRONMENT PREPARATION 9

STAGE 01.1 — Verify Existing Development Environment . . . 9
STAGE 01.2 — Close Gaps and Optimize Downloads . . . 21
STAGE 01.3 — Establish the Free-Tier AI Workflow . . . 25

PHASE 02 — PROJECT FOUNDATION 30

STAGE 02.1 — Repository and Engineering Documents . . . 30
STAGE 02.2 — Create the New Android Project . . . 36
STAGE 02.3 — Skeleton, Localization and Theme . . . 43

PHASE 03 — TECHNICAL RISK SPIKES 48

STAGE 03.1 — Map Engine Feasibility . . . 48
STAGE 03.2 — Networking and Display Feasibility . . . 52
STAGE 03.3 — Spike Decision Gate . . . 56

PHASE 04 — CORE DOMAIN LOGIC 58

STAGE 04.1 — Geography and Coordinates . . . 58
STAGE 04.2 — Model, Commands and Serialization . . . 63

PHASE 05 — DATA LAYER AND PROJECT PERSISTENCE 72

STAGE 05.1 — Storage Foundation . . . 72
STAGE 05.2 — Project Repository . . . 76

PHASE 06 — AUTHENTICATION AND SECURITY 82

STAGE 06.1 — Accounts and Credentials . . . 82
STAGE 06.2 — Login UI . . . 85

PHASE 07 — LAYOUT SYSTEM AND APPLICATION SHELL 91

STAGE 07.1 — Layout Engine . . . 91
STAGE 07.2 — Shell Screens and Navigation . . . 97

PHASE 08 — MAP ENGINE AND BUNDLED BASEMAP 105

STAGE 08.1 — MapEngine Abstraction . . . 105
STAGE 08.2 — Bundled Offline Basemap . . . 109

PHASE 09 — MAP SOURCES AND IMPORT PIPELINE 113

STAGE 09.1 — Import Foundation . . . 113
STAGE 09.2 — Tile Sources . . . 116
STAGE 09.3 — Image Sources and Maps Panel . . . 120

PHASE 10 — EDITOR CORE: OBJECTS, DRAWING, LAYERS 125

STAGE 10.1 — Rendering and Interaction Foundation . . . 125
STAGE 10.2 — Drawing Tools . . . 130
STAGE 10.3 — Properties, Layers and Undo . . . 136

PHASE 11 — MEDIA, SYMBOLS AND ANIMATION 142

STAGE 11.1 — Images . . . 142
STAGE 11.2 — Animation . . . 145
STAGE 11.3 — Symbols . . . 147

PHASE 12 — IMPORT, EXPORT AND PORTABILITY 151

STAGE 12.1 — GIS Import . . . 151
STAGE 12.2 — Exchange Export . . . 156
STAGE 12.3 — MapX Packages . . . 158

PHASE 13 — GEOREFERENCING 166

STAGE 13.1 — Georeferencing . . . 166

PHASE 14 — SCENES, PRESENTER MODE AND BROADCAST GRAPHICS 171

STAGE 14.1 — Scenes . . . 171
STAGE 14.2 — Presenter Mode and Graphics . . . 173

PHASE 15 — EXTERNAL DISPLAY 180

STAGE 15.1 — External Display . . . 180

PHASE 16 — PHONE REMOTE 185

STAGE 16.1 — Protocol and Host . . . 185
STAGE 16.2 — Phone Side . . . 188

PHASE 17 — GPS LOCATION 193

STAGE 17.1 — Location . . . 193

PHASE 18 — IMAGE AND PDF EXPORT 197

STAGE 18.1 — Image and PDF Export . . . 197

PHASE 19 — LOCALIZATION, ACCESSIBILITY AND POLISH 200

STAGE 19.1 — Localization and Accessibility . . . 200

PHASE 20 — P2 ADVANCED FEATURES 205

STAGE 20.1 — Advanced Formats . . . 205
STAGE 20.2 — Other P2 Features . . . 208

PHASE 21 — HARDENING, PERFORMANCE AND ACCEPTANCE 212

STAGE 21.1 — Performance and Stability . . . 212
STAGE 21.2 — Security and Offline Audit . . . 214
STAGE 21.3 — Full Acceptance . . . 217

PHASE 22 — RELEASE AND HANDOFF 220

STAGE 22.1 — Release . . . 220

Appendix A - Provenance and verified research (4 Nov 2026) 223

Appendix B - Standing Orders (paste at the start of every AI session) 224

Appendix C - Free-tier AI tool matrix and fallback ladder 225

Appendix D - Engineering document lifecycle 226

Appendix E - Dependency register 227

Appendix F - Format Support Matrix evidence plan 228

Appendix G - Requirements traceability 229

Appendix H - Official links 230

Appendix I - Assumptions and unresolved limitations 231

Assumptions . . . 231

Limitations . . . 231

Appendix J - Command cheat sheet 232

Appendix K - Quality audit (master prompt section 17) 233

How to use this roadmap

This document turns the two project documents into a complete, ordered build plan. Work strictly in step order. Every step has the same 17 parts so nothing is left to guess. You are a non-programmer: AI tools write the code, you run commands, verify results and tick the tracker only when the acceptance criteria are met.

- PHASE > STAGE > STEP identifiers are permanent. The tracker uses exactly the same IDs.
- Planning a task never means it is done. All 184 steps start unticked.
- Evidence vocabulary used everywhere: Implemented, Built, Automated-test verified, Device verified, Not tested, Deferred.
- Hardware features (external display, GPS, phone remote) are never marked tested without a real run; simulated tests are labelled Simulated only.
- Provenance tags: [SPEC] from your documents; [VERIFIED date] checked in official pages on 4 Nov 2026; [REC] recommendation; [ASSUMED] unconfirmed assumption.

Overview diagram



Dark = main build line. Blue = phases that carry the highest technical risk (spikes, external display, phone remote). Order follows dependencies, not the specification's feature or

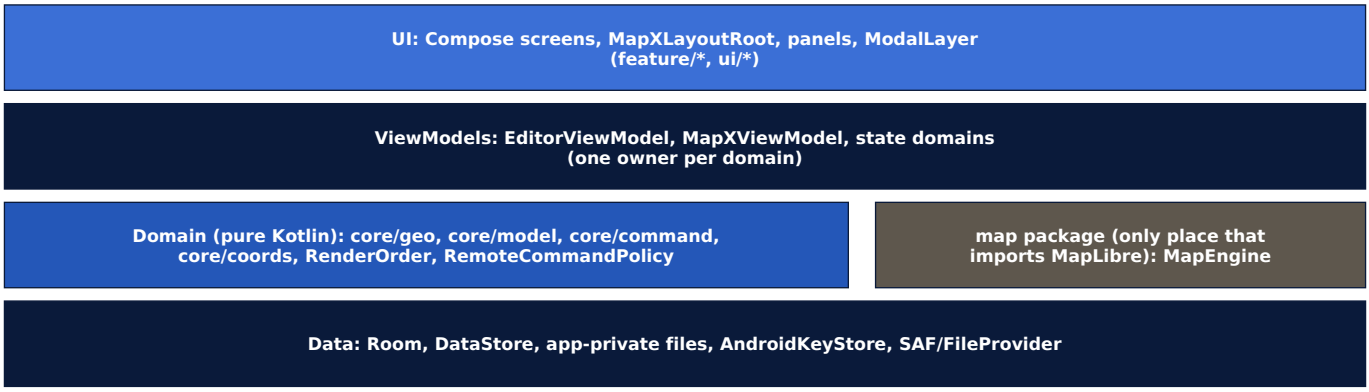
Phase summary

Phase	Title	Stages	Steps	Effort*	Gate
01	Environment Preparation	3	17	S	Gate: the environment report is complete, one physical device is visible in adb, and total planned downloads are known.
02	Project Foundation	3	15	M	Gate: a clean debug build installs and starts on both devices; the six documents exist; architecture tests run.
03	Technical Risk Spikes	3	8	M	Gate: every spike has a recorded result and decision (spike code is discarded; only decisions and tested snippets are kept).
04	Core Domain Logic	2	12	L	Gate: all JVM tests in Spec 16.2 for this layer pass; architecture tests pass.
05	Data Layer And Project Persistence	2	9	L	Gate: Room tests and persistence tests pass; kill-process restore proven later in 10.3.4.
06	Authentication And Security	2	8	L	Gate: AC-SEC-1 passes on devices except items needing later features.
07	Layout System And Application Shell	2	12	L	Gate: regionsNeverOverlap passes in four modes and two languages; stress sizes checked on devices.
08	Map Engine And Bundled Basemap	2	7	L	Gate: a new project shows the offline world map in airplane mode on both devices.
09	Map Sources And Import Pipeline	3	11	XL	Gate: every format in the matrix is classified Supported/Partial/Deferred with a real sample.
10	Editor Core: Objects, Drawing, Layers	3	16	XL	Gate: AC-GEO-1, AC-DRW-1, AC-LYR-1 pass on devices.
11	Media, Symbols And Animation	3	8	L	Gate: AC-IMG-1 passes; symbols searchable in both languages.
12	Import, Export And Portability	3	14	XL	Gate: AC-EXP-1..5 pass; two-device transfer verified.
13	Georeferencing	1	4	M	Gate: AC-SRC-2 passes.

Phase	Title	Stages	Steps	Effort*	Gate
14	Scenes, Presenter Mode And Broadcast Graphics	2	8	XL	Gate: AC-PRS-1 passes including byte-identical document test.
15	External Display	1	4	M	Gate: AC-DSP-1 with simulated or real evidence labelled honestly.
16	Phone Remote	2	7	L	Gate: AC-REM-1 passes between Lenovo and Samsung.
17	Gps Location	1	3	S	Gate: AC-GPS-1 verified on the GNSS device.
18	Image And Pdf Export	1	2	M	Gate: AC-EXP-4 passes.
19	Localization, Accessibility And Polish	1	4	M	Gate: AC-I18N-1 and accessibility checks pass.
20	P2 Advanced Features	2	6	L	Gate: each P2 item either verified or explicitly marked Not built/Deferred.
21	Hardening, Performance And Acceptance	3	7	L	Gate: all AC-* criteria have evidence or an honest Not tested label.
22	Release And Handoff	1	2	S	Gate: signed APK installed; final report written with honest verdict.

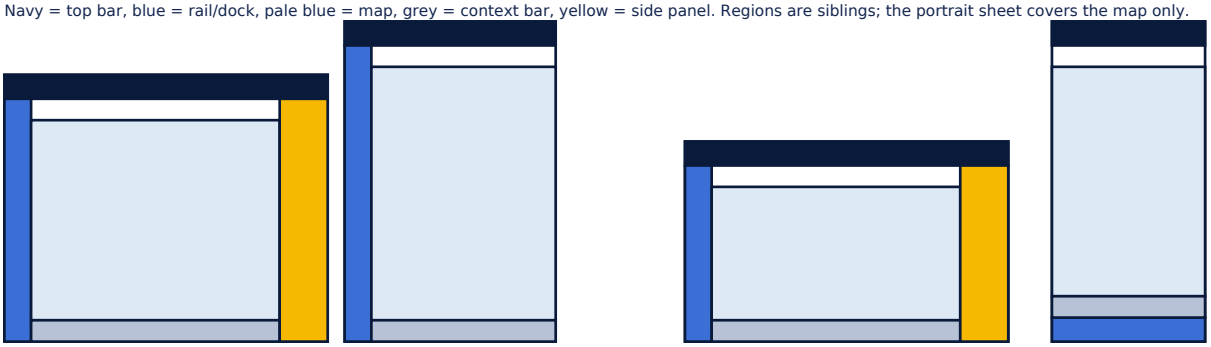
* Relative effort (S small, M medium, L large, XL very large); a planning estimate, not a time promise.

Architecture and layout at a glance

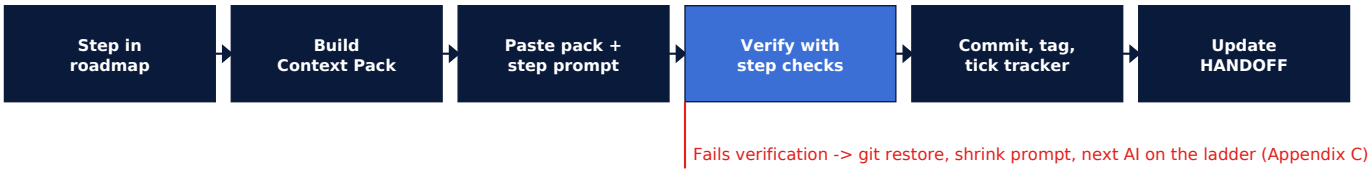


Rule: UI -> ViewModel -> Domain -> Data. Domain imports no Android UI and no MapLibre; an architecture test enforces it (step 02.3.2).

Navy = top bar, blue = rail/dock, pale blue = map, grey = context bar, yellow = side panel. Regions are siblings; the portrait sheet covers the map only.



AI workflow loop



Documents, not chat history, carry the project memory between AI platforms.

Disk budget



Planning estimate [REC, not measured] of about 12 GB of new disk use out of ~50 GB free; no emulator image. Replace with your measured values in step 01.1.10.

Key decisions made in this roadmap

- Order is dependency-driven: environment, foundation, risk spikes, pure domain, persistence, security, layout shell, map, sources, editor, media, import/export, georeferencing, presentation, display, remote, GPS, export images, polish, P2, hardening, release.
- Risk spikes run in Phase 03, before the product depends on MapLibre/MBTiles/PMTiles, Arabic map labels, TLS WebSocket pairing, external display or GNSS.
- Defaults from the specification: DN-1 several accounts; DN-2 no app-level document encryption in the first release; DN-3 Java-WebSocket with TLS-socket fallback; DN-4 partial GeoTIFF; DN-5 Transfer Bundle above ~2 GB; DN-6 documented remote controls as session overrides; DN-7 IMAGE_LOCAL approved; DN-8 duplicate storage accepted; DN-9 verified by adb; DN-10 nearby transfer P2.
- Alert red #E3201B is authoritative (Guide register C-02); text on red uses white and is checked in the contrast test.
- Emulator images are skipped; physical devices are the test targets. Only one tool per task area is installed.

PHASE 01 — ENVIRONMENT PREPARATION

Verify what is already installed on the Windows 11 PC, install only what is genuinely missing, prepare the free AI workflow, and record everything in an environment report. Nothing is reinstalled unless a check fails. Flutter stays installed but is never used for MapX.

Estimated relative effort: S. Gate: the environment report is complete, one physical device is visible in adb, and total planned downloads are known.

STAGE 01.1 — Verify Existing Development Environment

Read-only checks. No installs happen in this stage.

STEP 01.1.1 — Verify Windows Configuration

Provenance: [REC]

1. Step ID / 2. Title

01.1.1 — Verify Windows Configuration (type: environment check)

3. Objective

Confirm the PC meets the needs of Android Studio, Gradle and a local 7B model, and note the real free disk space.

4. Why it is necessary

The roadmap is sized for ~50 GB free and 16 GB RAM. Every later size and memory decision depends on the true numbers.

5. Prerequisites

None (first step) Phase Gate: the environment report is complete, one physical device is visible in adb, and total planned downloads are known.

6. Required software

Windows PowerShell (built in).

7. Exact instructions

1. Open PowerShell (normal user, not Administrator): press Win, type PowerShell, Enter.
2. Run the command block below and copy the whole output.
3. Check three things: Windows 11 64-bit build 22631; at least 16 GB RAM; free space on C: (target: 35 GB or more free).
4. Choose the project drive: use C: unless it has less than 25 GB free and another drive has more. Write the choice down.
5. Confirm the project path will be short and have no spaces: C:\dev\MapX (assumption). Long paths cause Gradle failures on Windows.

8. Terminal commands

Where to run: PowerShell, any folder.

```
# What: reads system facts only. Success: prints the values listed in the instructions.
Get-ComputerInfo | Select-Object OsName, OsVersion, OsBuildNumber, OsArchitecture, CsTotalPhysicalMemory
Get-CimInstance Win32_Processor | Select-Object Name, NumberOfCores, NumberOfLogicalProcessors
Get-CimInstance Win32_VideoController | Select-Object Name, AdapterRAM, DriverVersion
Get-PSDrive -PSProvider FileSystem | Select-Object Name, @{n='FreeGB';e={[math]::Round($_.Free/1GB,1)}},
  @{n='UsedGB';e={[math]::Round($_.Used/1GB,1)}}
(Get-ItemProperty 'HKLM:\SYSTEM\CurrentControlSet\Control\FileSystem').LongPathsEnabled
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).

Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 01.1.1 - Verify Windows Configuration
TASK: Interpret the outputs of my Windows system check and tell me whether this PC can run Android Studio,
  Gradle and a 7B local model together.
CONTEXT TO ATTACH: Paste the full PowerShell output. Mention: i7-9700 (unverified), 16 GB RAM, RTX 2070 8 GB,
  ~50 GB free, slow limited internet.
FILES TO CREATE OR MODIFY: Only the files needed for this step; list them before writing code.
RULES: Do not suggest upgrades or reinstalls. Do not assume values that are not in the pasted output. Native
  Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room, DataStore, kotlinx.serialization,
  Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a dependency that is not already in the
  version catalog; if one is needed, stop and say so. Store geographic data as WGS84 lat/lon, never screen
  pixels. Every edit is a command. No hard-coded user-visible strings (use string resources, English and
  Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or update unit tests. Do not
  weaken or delete existing tests. Never claim something works unless it was built and tested.
OUTPUT: A table: item, my value, required value, pass/fail; then a list of any risks (low disk, low RAM, long
  paths disabled) with the exact fix.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-01.1.1. If the result fails verification: git restore . (or git reset --hard before-01.1.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Windows 11, build 22631, 64-bit, 16 GB RAM, free space known, CPU model confirmed or corrected.

12. Verification procedure

Your pasted output shows OsBuildNumber 22631 and OsArchitecture 64-bit; the AI table has no unexplained 'fail'.

13. Acceptance criteria

- [ ] Real free GB on every drive is recorded in the environment report.
- [ ] CPU model confirmed (the i7-9700 is only 'identified from a screenshot').
- [ ] Project drive and path chosen.
- [ ] LongPathsEnabled value recorded (1 is preferred; fixing it is optional and covered in 01.2.1).

14. Common errors

- 'Get-ComputerInfo is slow' -> wait up to a minute; it is normal.
- Script execution blocked -> these commands are not scripts; make sure you did not paste a .ps1 file path.
- Free space under 25 GB -> do 01.2.2 early and skip every optional download.

15. Troubleshooting

1) Re-run the command exactly as printed. 2) Compare your output with the 'Expected result'. 3) Paste command and output to the AI with 'Interpret this; do not suggest reinstalling anything unless it is proven missing.' 4) Record UNVERIFIED if still unclear.

16. Dependencies

Depends on step none.

17. Estimated download / storage

No download.

STEP 01.1.2 — Verify Java JDK

Provenance: [REC]

1. Step ID / 2. Title

01.1.2 — Verify Java JDK (type: environment check)

3. Objective

Find out which Java you have, and which Java Gradle will really use.

4. Why it is necessary

Android Studio ships its own JetBrains Runtime (JBR) and builds should use it. A stray old system JDK can break command-line builds if JAVA_HOME points to it.

5. Prerequisites

Step 01.1.1 done and verified (tracker ticked). Phase Gate: the environment report is complete, one physical device is visible in adb, and total planned downloads are known.

6. Required software

Windows PowerShell, adb (Platform-Tools), a text editor; an AI tool from the AI field for interpretation.

7. Exact instructions

1. Run the commands below. Note the version printed by java -version, the JAVA_HOME value, and whether the Android Studio JBR folder exists.
2. Rule: for MapX, builds run with the JDK that Android Studio supplies (Gradle JDK = 'jbr'). A separate system JDK is allowed but not required.
3. If JAVA_HOME is set to something other than the JBR, note it. Do not change it yet; 01.2.1 handles the fix only if a command-line build fails.
4. Do not download a JDK now. Only if neither a system JDK nor the JBR is found do you proceed to 01.2.1.

8. Terminal commands

Where to run: PowerShell, any folder. If Android Studio was installed per-user, the JBR may be under %LOCALAPPDATA%\Programs\Android Studio\jbr instead.

```
# What: shows Java versions and JAVA_HOME. Success: a version line (17 or newer) or 'not recognized' (acceptable
  if JBR exists).
java -version
echo $env:JAVA_HOME
Test-Path 'C:\Program Files\Android\Android Studio\jbr\bin\java.exe'
& 'C:\Program Files\Android\Android Studio\jbr\bin\java.exe' -version
```

9. Official download links

- Android Studio and its bundled JDK: https://developer.android.com/studio
- Only if a JDK is truly missing: https://adoptium.net/

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 01.1.2 - Verify Java JDK
TASK: Explain these Java outputs in simple language: which Java will Android Studio's Gradle use, and is any
  action needed?
CONTEXT TO ATTACH: Paste the four outputs. State that MapX is a native Kotlin Android project built with Android
  Studio.
FILES TO CREATE OR MODIFY: Only the files needed for this step; list them before writing code.
RULES: Do not recommend installing a JDK if the Android Studio JBR exists. Native Android only: Kotlin, Jetpack
  Compose, Material 3, MapLibre Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels,
  manual DI via AppContainer. Do not add a dependency that is not already in the version catalog; if one is
  needed, stop and say so. Store geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command.
  No hard-coded user-visible strings (use string resources, English and Arabic). Touch targets at least 48 dp.
  Use start/end, never left/right. Write or update unit tests. Do not weaken or delete existing tests. Never
  claim something works unless it was built and tested.
OUTPUT: Plain-language verdict: 'No action needed' or exactly one corrective action.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-01.1.2. If the result fails verification: git restore . (or git reset --hard before-01.1.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

You know the JBR path and version, and whether JAVA_HOME conflicts.

12. Verification procedure

The JBR java.exe prints a version (17 or newer is expected for current AGP; confirm in 02.1.5).

13. Acceptance criteria

[ ] JBR path recorded.
[ ] JAVA_HOME recorded (blank is fine).
[ ] No JDK download started unless the JBR is missing.

14. Common errors

- 'java is not recognized' -> normal when only the JBR exists.
- Test-Path returns False -> Android Studio is installed in another folder; find it with 01.1.3.

15. Troubleshooting

1) Re-run the command exactly as printed. 2) Compare your output with the 'Expected result'. 3) Paste command and output to the AI with 'Interpret this; do not suggest reinstalling anything unless it is proven missing.' 4) Record UNVERIFIED if still unclear.

16. Dependencies

Depends on step 01.1.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 01.1.3 — Verify Android Studio

Provenance: [VERIFIED 4 Nov 2026: release page structure] [REC]

1. Step ID / 2. Title

01.1.3 — Verify Android Studio (type: environment check)

3. Objective

Confirm Android Studio opens, note its exact version, and confirm it is a stable channel build.

4. Why it is necessary

MapX needs a current stable Studio to create a project with the latest Android Gradle Plugin (AGP), Compose and API 36/37 support. Old Studio versions cannot open new AGP versions.

5. Prerequisites

Step 01.1.2 done and verified (tracker ticked). Phase Gate: the environment report is complete, one physical device is visible in adb, and total planned downloads are known.

6. Required software

Android Studio.

7. Exact instructions

1. Open Android Studio. Choose Help > About and copy the version text (name, build number, runtime).
2. Open Help > Check for Updates. Note whether an update exists. Do NOT update yet if the download is over 1 GB and your connection is slow: finish 01.1 first, then decide in 01.2.1.
3. Open the official release page and compare your version with the current stable channel (links below). Research on 4 Nov 2026 found the stable channel moving through new releases quickly (a stable build named 'Rabbit 1' with AGP

9.4.0 was listed on the preview page); recheck the page yourself because it changes.

4. Open File > Settings > Build, Execution, Deployment > Gradle and note 'Gradle JDK'. Set it to the bundled JBR if it is not.
5. Open File > Settings > Appearance & Behavior > System Settings > Android SDK and note the SDK location (step 01.1.4 uses it).

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
# No command required beyond those in the instructions.
```

9. Official download links

- Download and release channels: https://developer.android.com/studio
- Release notes and AGP compatibility table: https://developer.android.com/studio/releases

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 01.1.3 - Verify Android Studio
TASK: Compare my Android Studio version with the current stable release and tell me if I must update before
  creating a new Compose project. I will paste Help > About and the release page headline.
CONTEXT TO ATTACH: Paste Help > About text and the 'current versions' table from the release page.
FILES TO CREATE OR MODIFY: Only the files needed for this step; list them before writing code.
RULES: Do not recommend preview/canary builds. Do not recommend updating if my version can already create a
  project with a current stable AGP. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native,
  Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Verdict: 'Update required' or 'Current enough', with one reason.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-01.1.3. If the result fails verification: git restore . (or git reset --hard before-01.1.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Studio version and build are recorded; Gradle JDK is the JBR; SDK path is known.

12. Verification procedure

Help > About shows a name and build number; Settings > Gradle shows the JBR.

13. Acceptance criteria

[ ] Version string saved in the environment report.
[ ] Stable (not canary) channel confirmed.
[ ] Update decision made and recorded (update now / update after Phase 01 / not needed).

14. Common errors

- Studio is very old and cannot create the Compose template -> mark 'Update required' for 01.2.1.
- Studio updates fail on slow internet -> download the installer once from the official page, resume if interrupted, and keep the installer (about 1 GB class) to avoid repeating.

15. Troubleshooting

1) Re-run the command exactly as printed. 2) Compare your output with the 'Expected result'. 3) Paste command and output to the AI with 'Interpret this; do not suggest reinstalling anything unless it is proven missing.' 4) Record UNVERIFIED if still unclear.

16. Dependencies

Depends on step 01.1.2; leads to the next step in the phase order.

17. Estimated download / storage

Update, if needed: roughly 1 GB class installer (check the page; do not assume).

STEP 01.1.4 — Verify Android SDK

Provenance: [REC]

1. Step ID / 2. Title

01.1.4 — Verify Android SDK (type: environment check)

3. Objective

List the installed SDK packages and decide which ones MapX really needs and which can be removed to save space.

4. Why it is necessary

A default SDK install can hold many platforms, images and tools you never use. MapX needs a platform for compileSdk, matching build-tools and platform-tools; it needs no emulator image and no NDK (the spec forbids project-owned native code).

5. Prerequisites

Step 01.1.3 done and verified (tracker ticked). Phase Gate: the environment report is complete, one physical device is visible in adb, and total planned downloads are known.

6. Required software

Android Studio SDK Manager, sdkmanager.

7. Exact instructions

1. In Android Studio open Tools > SDK Manager. Tab 'SDK Platforms': note which API levels are ticked (installed). Tab 'SDK Tools': note installed tools.
2. Run the command block below to list installed packages and the folder size.
3. Needed for MapX: one platform at the compileSdk level (decided in 02.1.5; API 36 is the expected stable choice; API 37 is optional later), Build-Tools of the matching generation, Platform-Tools (adb), Android SDK Command-line Tools (latest).
4. Not needed by default: Android Emulator, system images, NDK, CMake. Note them as 'removable candidates'; do not delete anything yet.
5. Record the SDK folder size; it tells you how much space removal could free.

8. Terminal commands

Where to run: PowerShell. If your SDK is elsewhere, use the path shown in Android Studio's SDK Manager.

```
# What: lists installed SDK packages and total SDK size. Success: a package list and a size in GB.
$sdk = "$env:LOCALAPPDATA\Android\Sdk"
Test-Path $sdk
Get-ChildItem $sdk -Directory | ForEach-Object { '{0,-22} {1,8:N2} GB' -f $_.Name, ((Get-ChildItem $_.FullName
  -Recurse -Force -ErrorAction SilentlyContinue | Measure-Object Length -Sum).Sum/1GB) }
& "$sdk\cmdline-tools\latest\bin\sdkmanager.bat" --list_installed
```

9. Official download links

- sdkmanager reference: https://developer.android.com/tools/sdkmanager
- Platform-Tools (adb): https://developer.android.com/tools/releases/platform-tools

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 01.1.4 - Verify Android SDK
TASK: From this SDK package list and folder-size table, tell me which packages MapX needs and which are safe to
  remove to save disk space. MapX: native Kotlin Compose app, minSdk 28, no NDK code, physical devices only.
CONTEXT TO ATTACH: Paste the folder sizes and the sdkmanager --list_installed output.
FILES TO CREATE OR MODIFY: Only the files needed for this step; list them before writing code.
RULES: Never advise deleting platform-tools, build-tools or the platform I will compile against. Offline
  emulator images and NDK are removable candidates. Native Android only: Kotlin, Jetpack Compose, Material 3,
  MapLibre Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via
  AppContainer. Do not add a dependency that is not already in the version catalog; if one is needed, stop and
  say so. Store geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded
  user-visible strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use start/end,
  never left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim something
  works unless it was built and tested.
OUTPUT: Two lists: KEEP and REMOVE-CANDIDATES, each item with size and reason.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-01.1.4. If the result fails verification: git restore . (or git reset --hard before-01.1.4), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Installed packages known; removable candidates and potential savings in GB recorded.

12. Verification procedure

sdkmanager prints a list without errors; the folder sizes add up to the SDK total.

13. Acceptance criteria

- [ ] cmdline-tools present or flagged missing.
- [ ] platform-tools present or flagged missing.
- [ ] Total SDK size recorded.
- [ ] Emulator/system image presence recorded.

14. Common errors

- sdkmanager.bat not found -> command-line tools not installed; flag for 01.2.1 (small download).

- 'JAVA_HOME is not set' printed by sdkmanager -> set it for this session: $env:JAVA_HOME='C:\Program Files\Android\Android Studio\jbr'.

15. Troubleshooting

1) Re-run the command exactly as printed. 2) Compare your output with the 'Expected result'. 3) Paste command and output to the AI with 'Interpret this; do not suggest reinstalling anything unless it is proven missing.' 4) Record UNVERIFIED if still unclear.

16. Dependencies

Depends on step 01.1.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 01.1.5 — Verify Git

Provenance: [REC]

1. Step ID / 2. Title

01.1.5 — Verify Git (type: environment check)

3. Objective

Confirm Git is installed and configured with your name and e-mail.

4. Why it is necessary

Git is the safety net for AI-generated code: every step is tagged so a bad AI answer can be undone in seconds.

5. Prerequisites

Step 01.1.4 done and verified (tracker ticked). Phase Gate: the environment report is complete, one physical device is visible in adb, and total planned downloads are known.

6. Required software

Windows PowerShell, adb (Platform-Tools), a text editor; an AI tool from the AI field for interpretation.

7. Exact instructions

1. Run the commands below.
2. If user.name and user.email are empty, set them (commands in the block, commented).
3. Confirm autocrlf behaviour is Windows-friendly (true is acceptable).
4. Do not create a GitHub account or remote: not required. A local repo plus a copy on a second drive or USB is the backup plan (02.1.1).

8. Terminal commands

Where to run: PowerShell, any folder.

```
# What: shows Git version and identity. Success: a version line and your name/e-mail.
git --version
git config --global user.name
git config --global user.email
git config --global core.autocrlf
# If name/e-mail are empty, run (replace with your details):
# git config --global user.name "Your Name"
# git config --global user.email "you@example.com"
# git config --global core.autocrlf true
```

9. Official download links

- Only if Git is missing: https://git-scm.com/download/win

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 01.1.5 - Verify Git
TASK: Check this Git configuration output and tell me if anything is missing for a local-only repository on
  Windows.
CONTEXT TO ATTACH: Paste the outputs.
FILES TO CREATE OR MODIFY: Only the files needed for this step; list them before writing code.
RULES: Do not suggest GitHub or any cloud service. Native Android only: Kotlin, Jetpack Compose, Material 3,
  MapLibre Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via
  AppContainer. Do not add a dependency that is not already in the version catalog; if one is needed, stop and
  say so. Store geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded
  user-visible strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use start/end,
  never left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim something
  works unless it was built and tested.
OUTPUT: Pass/fail list.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-01.1.5. If the result fails verification: git restore . (or git reset --hard before-01.1.5), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the

backup.ps1 copy only if the repository itself is damaged.

11. Expected result
Git version printed; identity configured.

12. Verification procedure
git --version prints a version; git config user.name prints a name.

13. Acceptance criteria
[ ] Git version recorded.
[ ] Identity set.
[ ] No remote repository created.

14. Common errors
- 'git is not recognized' -> install from the official link (about 60 MB class) with default options.
- Line-ending warnings later -> harmless; add .gitattributes in 02.1.1.

15. Troubleshooting
1) Re-run the command exactly as printed. 2) Compare your output with the 'Expected result'. 3) Paste command and output to the AI with 'Interpret this; do not suggest reinstalling anything unless it is proven missing.' 4) Record UNVERIFIED if still unclear.

16. Dependencies
Depends on step 01.1.4; leads to the next step in the phase order.

17. Estimated download / storage
Only if missing: tens of MB.

STEP 01.1.6 — Verify Node.js and Python (Optional Tools)

Provenance: [REC]

1. Step ID / 2. Title
01.1.6 — Verify Node.js and Python (Optional Tools) (type: environment check)

3. Objective
Check Node.js and Python so they can be used for data-preparation scripts; do not fix them unless a later step needs them.

4. Why it is necessary
Node (mapshaper) helps prepare the bundled Natural Earth basemap in 08.2.1 and Python runs small helper scripts. Python is installed but 'not verified'; the Windows Store alias often masks a missing install.

5. Prerequisites
Step 01.1.5 done and verified (tracker ticked). Phase Gate: the environment report is complete, one physical device is visible in adb, and total planned downloads are known.

6. Required software
Windows PowerShell, adb (Platform-Tools), a text editor; an AI tool from the AI field for interpretation.

7. Exact instructions
1. Run the commands below.
2. For Python, try py --version first (the Windows launcher); if it prints a version, use 'py' in this roadmap. If 'python' opens the Microsoft Store, ignore the alias.
3. Node: note node and npm versions.
4. Decision rule: if both work, do nothing. If Python does not work, it is optional: the Context Pack generator (01.3.3) has a PowerShell version too.

8. Terminal commands
Where to run: PowerShell.

```
# What: version checks. Success: versions printed (a failure is acceptable and just recorded).
node --version
npm --version
py --version
python --version
py -c "import sys; print(sys.executable)"
```

9. Official download links
- Node.js: https://nodejs.org/
- Python for Windows: https://www.python.org/downloads/windows/

10. Complete AI implementation prompt
Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 01.1.6 - Verify Node.js and Python (Optional Tools)
TASK: Interpret these Node and Python version outputs and say which scripts in a Windows project I can run with
  them.
CONTEXT TO ATTACH: Paste outputs.
FILES TO CREATE OR MODIFY: Only the files needed for this step; list them before writing code.
RULES: Do not recommend installing anything unless a command failed AND I confirm I need that tool. Native
  Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room, DataStore, kotlinx.serialization,
  Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a dependency that is not already in the
  version catalog; if one is needed, stop and say so. Store geographic data as WGS84 lat/lon, never screen
  pixels. Every edit is a command. No hard-coded user-visible strings (use string resources, English and
  Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or update unit tests. Do not
  weaken or delete existing tests. Never claim something works unless it was built and tested.
OUTPUT: Short verdict per tool.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-01.1.6. If the result fails verification: git restore . (or git reset --hard before-01.1.6), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Node/npm and Python status recorded.

12. Verification procedure

Version lines or documented absence.

13. Acceptance criteria

[ ] Node version recorded.
[ ] Python launcher status recorded.
[ ] No installs performed.

14. Common errors

- 'python' opens Microsoft Store -> use 'py' or install Python later (about 30 MB class installer) only when 01.3.3 or 08.2.1 needs it.

15. Troubleshooting

1) Re-run the command exactly as printed. 2) Compare your output with the 'Expected result'. 3) Paste command and output to the AI with 'Interpret this; do not suggest reinstalling anything unless it is proven missing.' 4) Record UNVERIFIED if still unclear.

16. Dependencies

Depends on step 01.1.5; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 01.1.7 — Verify Flutter (Installed, Not Used)

Provenance: [SPEC D-15]

1. Step ID / 2. Title

01.1.7 — Verify Flutter (Installed, Not Used) (type: environment check)

3. Objective

Confirm Flutter is installed only to know its size and ensure it does not interfere with MapX; MapX never uses Flutter.

4. Why it is necessary

The owner's decision D-15 dropped Flutter. Flutter can still share the Android SDK, so the SDK path must stay consistent, and its folder consumes disk space.

5. Prerequisites

Step 01.1.6 done and verified (tracker ticked). Phase Gate: the environment report is complete, one physical device is visible in adb, and total planned downloads are known.

6. Required software

Windows PowerShell, adb (Platform-Tools), a text editor; an AI tool from the AI field for interpretation.

7. Exact instructions

1. Run flutter --version only (do not run flutter doctor --android-licenses unless needed).
2. Check the Flutter folder size. If disk space is tight, the Flutter folder is the largest safe deletion candidate; record this but do not delete without your decision.
3. Make sure no Flutter command is ever run inside C:\dev\MapX.
4. Confirm Flutter's Android SDK path (flutter config) matches the Android Studio SDK path, so only one SDK exists.

8. Terminal commands

Where to run: PowerShell, any folder.

```
# What: checks Flutter version and its configured SDK. Success: version printed.
flutter --version
flutter config --list
```

9. Official download links

- Reference only: https://docs.flutter.dev/

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 01.1.7 - Verify Flutter (Installed, Not Used)
TASK: Given this Flutter version and config output, confirm Flutter shares the same Android SDK path as Android
  Studio and tell me if it can interfere with a native Kotlin project.
CONTEXT TO ATTACH: Paste outputs and the SDK path from 01.1.4.
FILES TO CREATE OR MODIFY: Only the files needed for this step; list them before writing code.
RULES: Do not suggest using Flutter for MapX. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre
  Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do
  not add a dependency that is not already in the version catalog; if one is needed, stop and say so. Store
  geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible
  strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never
  left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim something works
  unless it was built and tested.
OUTPUT: Verdict and any SDK-path mismatch.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-01.1.7. If the result fails verification: git restore . (or git reset --hard before-01.1.7), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Flutter noted as unused; SDK path consistent.

12. Verification procedure

flutter --version prints a version; android-sdk path equals Studio's.

13. Acceptance criteria

- [ ] Flutter marked 'Installed, not used' in the tracker.
- [ ] Single Android SDK path confirmed.

14. Common errors

- Two different SDK folders exist -> delete none yet; point Flutter at the Studio SDK with: flutter config --android-sdk "<path>".

15. Troubleshooting

1) Re-run the command exactly as printed. 2) Compare your output with the 'Expected result'. 3) Paste command and output to the AI with 'Interpret this; do not suggest reinstalling anything unless it is proven missing.' 4) Record UNVERIFIED if still unclear.

16. Dependencies

Depends on step 01.1.6; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 01.1.8 — Verify LM Studio and the Local Qwen Model

Provenance: [REC]

1. Step ID / 2. Title

01.1.8 — Verify LM Studio and the Local Qwen Model (type: environment check)

3. Objective

Confirm LM Studio runs Qwen2.5-Coder-7B-Instruct and measure how much context it can handle on your RTX 2070 (8 GB).

4. Why it is necessary

Local Qwen is your unlimited fallback when free cloud tiers run out. Its usable context size decides how big each prompt may be.

5. Prerequisites

Step 01.1.7 done and verified (tracker ticked). Phase Gate: the environment report is complete, one physical device is visible in adb, and total planned downloads are known.

6. Required software

LM Studio, Qwen2.5-Coder-7B-Instruct.

7. Exact instructions

1. Open LM Studio and load Qwen2.5-Coder-7B-Instruct. Note the quantization (for example Q4_K_M) and the file size.
2. In the model settings set 'Context Length' to 8192 first. If VRAM allows, try 16384. Watch GPU memory in Task Manager (Performance > GPU): keep dedicated GPU memory under about 7 GB.

3. Set GPU offload to maximum layers that fit. Test by pasting a 100-line Kotlin function and asking it to explain it.
4. Record: tokens per second, the context length that works without errors, and the model path. Rule for this roadmap: prompts for Qwen stay under about 6,000 tokens (roughly 4,500 words), because input plus answer must fit the context.
5. Close LM Studio when running long Gradle builds if RAM or GPU memory is short.

8. Terminal commands

Where to run: LM Studio (graphical).

```
# No command required beyond those in the instructions.
```

9. Official download links

- LM Studio: https://lmstudio.ai/

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 01.1.8 - Verify LM Studio and the Local Qwen Model
TASK: I run a 7B model with N tokens of context on an 8 GB GPU. Suggest the settings that keep generation stable
  for Kotlin code tasks.
CONTEXT TO ATTACH: State GPU, quantization, context length, tokens/s.
FILES TO CREATE OR MODIFY: Only the files needed for this step; list them before writing code.
RULES: Do not recommend larger models or paid hosting. Native Android only: Kotlin, Jetpack Compose, Material 3,
  MapLibre Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via
  AppContainer. Do not add a dependency that is not already in the version catalog; if one is needed, stop and
  say so. Store geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded
  user-visible strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use start/end,
  never left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim something
  works unless it was built and tested.
OUTPUT: Recommended context length and GPU offload settings.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-01.1.8. If the result fails verification: git restore . (or git reset --hard before-01.1.8), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Qwen loaded and answering; working context length recorded.

12. Verification procedure

A pasted 100-line Kotlin function is explained correctly.

13. Acceptance criteria

- [ ] Working context length recorded.
- [ ] Tokens per second recorded.
- [ ] Local-server option noted for 01.3.2.

14. Common errors

- Out-of-memory when loading -> lower context length or use a smaller quantization.
- Very slow answers -> GPU offload is not enabled; check the GPU setting.

15. Troubleshooting

1) Re-run the command exactly as printed. 2) Compare your output with the 'Expected result'. 3) Paste command and output to the AI with 'Interpret this; do not suggest reinstalling anything unless it is proven missing.' 4) Record UNVERIFIED if still unclear.

16. Dependencies

Depends on step 01.1.7; leads to the next step in the phase order.

17. Estimated download / storage

No new download (model already present).

STEP 01.1.9 — Verify Physical Android Devices over ADB

Provenance: [SPEC DN-9] [VERIFIED research on listings, to be replaced by your adb evidence]

1. Step ID / 2. Title

01.1.9 — Verify Physical Android Devices over ADB (type: device test)

3. Objective

Make the Lenovo Tab P12 and the Samsung Galaxy A16 visible to adb with USB debugging, and record each device's facts (Android version, page size, GNSS, display).

4. Why it is necessary

All testing happens on physical devices, so adb must work before any code is written. Device facts also resolve open decisions DN-9 (GNSS) and the 16 KB page-size risk.

5. Prerequisites

Step 01.1.8 done and verified (tracker ticked). Phase Gate: the environment report is complete, one physical device is visible in adb, and total planned downloads are known.

6. Required software

adb (Android SDK Platform-Tools).

7. Exact instructions

1. On each device: Settings > About tablet/phone > tap 'Build number' seven times; then Settings > System > Developer options > turn on USB debugging.
2. Connect one device by USB cable (a data cable, not charge-only). Accept the 'Allow USB debugging' prompt on the device and tick 'Always allow'.
3. Run the commands below once per device (connect one at a time to avoid confusion) and save the output.
4. Check: Android version (the spec reference is Android 15 for the Lenovo), PAGE_SIZE (4096 or 16384), and whether the feature list contains android.hardware.location.gps.
5. Research note (4 Nov 2026): retail listings for the Lenovo Tab P12 (Wi-Fi only TB370FU, USB-C 2.0) disagree on GPS; one specification database lists 'Positioning: No' while several retailers list GPS/GLONASS, and no DisplayPort output was found. Therefore trust only your adb result, not listings.
6. Developer options: also enable 'Stay awake' while charging for long test runs.

8. Terminal commands

Where to run: PowerShell. Connect one device at a time.

```
# What: lists devices and reads facts. Success: device shows 'device' (not 'unauthorized').
$adb = "$env:LOCALAPPDATA\Android\Sdk\platform-tools\adb.exe"
& $adb devices -l
& $adb shell getprop ro.product.model
& $adb shell getprop ro.build.version.release
& $adb shell getprop ro.build.version.sdk
& $adb shell getconf PAGE_SIZE
& $adb shell pm list features | Select-String -Pattern "location|gps|camera|wifi|display"
& $adb shell wm size
& $adb shell wm density
& $adb shell dumpsys display | Select-String -Pattern "mDisplayId|Display Id|DisplayDeviceInfo" | Select-Object
  -First 8
```

9. Official download links

- adb reference: https://developer.android.com/tools/adb

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 01.1.9 - Verify Physical Android Devices over ADB
TASK: Interpret these adb outputs for my device: tell me Android version, whether the device has GPS hardware,
  its memory page size, and its screen size class in dp.
CONTEXT TO ATTACH: Paste outputs for each device separately.
FILES TO CREATE OR MODIFY: Only the files needed for this step; list them before writing code.
RULES: Do not guess: if the feature list does not show gps, say 'no GNSS hardware reported'. Native Android
  only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room, DataStore, kotlinx.serialization,
  Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a dependency that is not already in the
  version catalog; if one is needed, stop and say so. Store geographic data as WGS84 lat/lon, never screen
  pixels. Every edit is a command. No hard-coded user-visible strings (use string resources, English and
  Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or update unit tests. Do not
  weaken or delete existing tests. Never claim something works unless it was built and tested.
OUTPUT: A small table per device: model, Android, API, page size, GNSS yes/no, smallest width dp.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-01.1.9. If the result fails verification: git restore . (or git reset --hard before-01.1.9), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Both devices authorized in adb; facts recorded; DN-9 answered (or the Samsung chosen as the GNSS test device).

12. Verification procedure

adb devices -l lists the device as 'device'; getprop returns the model names.

13. Acceptance criteria

- [ ] Lenovo facts recorded.
- [ ] Samsung facts recorded.
- [ ] DN-9 outcome recorded in MAPX_DECISIONS (done in 02.1.4).
- [ ] Page size of each device recorded.

14. Common errors

- 'unauthorized' -> unplug, revoke USB debugging authorizations in Developer options, reconnect and accept the prompt.
- Device not listed -> use another cable/port; install the Lenovo or Samsung USB driver only if Windows shows an unknown device.
- adb not found -> platform-tools missing: 01.2.1.

15. Troubleshooting

1) Run: adb devices -l and confirm 'device'. 2) Uninstall and reinstall: adb uninstall app.mapx then installDebug. 3) Capture the crash: adb logcat -d | Select-String 'FATAL|AndroidRuntime' and give the lines to the AI. 4) Retest on the other device to separate device-specific from general faults.

16. Dependencies

Depends on step 01.1.8; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 01.1.10 — Record Storage Budget and Environment Report

Provenance: [REC]

1. Step ID / 2. Title

01.1.10 — Record Storage Budget and Environment Report (type: document)

3. Objective

Assemble all findings into one ENVIRONMENT_REPORT.md and a storage budget with approximate download sizes.

4. Why it is necessary

This report is the baseline for the tracker and is pasted into every AI platform so nobody guesses your environment.

5. Prerequisites

Step 01.1.9 done and verified (tracker ticked). Phase Gate: the environment report is complete, one physical device is visible in adb, and total planned downloads are known.

6. Required software

Any text editor (VS Code).

7. Exact instructions

1. Create C:\dev\MapX_docs\ENVIRONMENT_REPORT.md (it moves into the repo in 02.1.1).
2. Fill each heading with your real outputs: Windows, CPU/GPU/RAM, drives, Java/JBR, Android Studio, SDK, Git, Node/Python, Flutter, LM Studio, devices.
3. Add the storage budget table: current free GB; planned installs; estimated Gradle cache growth. Planning estimates [REC, not measured]: Android Studio install about 1-2 GB, each SDK platform about 0.1-0.3 GB, Gradle cache about 3-8 GB after MapX dependencies, project build output about 0.5-2 GB, each emulator image 1.5 GB or more (skipped). Goal: stay below about 12 GB of new use.
4. Mark every unverified fact 'UNVERIFIED' (for example: CPU identified from screenshot).
5. Ask an AI to review the finished report for gaps using the prompt below.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
# No command required. Commit when finished:
git add -A
git commit -m "docs: step done"
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 01.1.10 - Record Storage Budget and Environment Report
TASK: Review my environment report for MapX Android development. List missing facts, contradictions with the
  project requirements (native Kotlin+Compose, minSdk 28, physical-device testing, slow internet, 50 GB disk),
  and the five biggest risks.
CONTEXT TO ATTACH: Paste ENVIRONMENT_REPORT.md.
FILES TO CREATE OR MODIFY: Only the files needed for this step; list them before writing code.
RULES: Do not invent facts. Do not propose paid tools. Native Android only: Kotlin, Jetpack Compose, Material 3,
  MapLibre Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via
  AppContainer. Do not add a dependency that is not already in the version catalog; if one is needed, stop and
  say so. Store geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded
  user-visible strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use start/end,
  never left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim something
  works unless it was built and tested.
OUTPUT: Numbered gap list, then risk list.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-01.1.10. If the result fails verification: git restore . (or git reset --hard before-01.1.10), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

A complete ENVIRONMENT_REPORT.md exists.

12. Verification procedure

Every heading has a value or the word UNVERIFIED.

13. Acceptance criteria

- [ ] Report saved.
- [ ] Storage budget has a target ceiling.
- [ ] Tracker 'Development environment' section can be filled from it.

14. Common errors

- Missing outputs -> rerun the relevant step in 01.1.

15. Troubleshooting

1) Compare the document with the listed sources. 2) Ask the AI to list contradictions between the document and the specification. 3) Fix and commit.

16. Dependencies

Depends on step 01.1.9; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STAGE 01.2 — Close Gaps and Optimize Downloads

Only act on failures found in 01.1. Each step says 'skip if not needed'.

STEP 01.2.1 — Install or Update Only What Is Missing

Provenance: [REC]

1. Step ID / 2. Title

01.2.1 — Install or Update Only What Is Missing (type: environment check)

3. Objective

Fix each gap found in Stage 01.1 with the smallest possible download.

4. Why it is necessary

Your internet is slow and limited; every unnecessary gigabyte costs time and money.

5. Prerequisites

Step 01.1.10 done and verified (tracker ticked). Phase Gate: the environment report is complete, one physical device is visible in adb, and total planned downloads are known.

6. Required software

Windows PowerShell, adb (Platform-Tools), a text editor; an AI tool from the AI field for interpretation.

7. Exact instructions

1. Make a list of gaps from the report. Typical candidates: (a) Android Studio update; (b) cmdline-tools or platform-tools; (c) SDK Platform API 36 and matching Build-Tools; (d) Git; (e) Windows long paths.
2. Rule: download each installer once and keep it. Use SDK Manager checkboxes, not full re-installs.
3. Install the SDK pieces from Android Studio (SDK Manager > tick: your compileSdk Platform, Build-Tools, Platform-Tools, Command-line Tools (latest)). Untick: Emulator, system images, NDK, CMake.
4. Optional free-space actions if needed: uninstall unused older SDK Platforms and system images in SDK Manager (shows size beside each).
5. Windows long paths (optional, needs Administrator PowerShell): the command below; restart afterwards.
6. Re-run the relevant 01.1 check to prove the gap is closed.

8. Terminal commands

Where to run: Android Studio SDK Manager; PowerShell (Administrator) only for the long-path command.

```
# OPTIONAL, Administrator PowerShell. What: enables Windows long paths. Success: returns 1 after.
New-ItemProperty -Path 'HKLM:\SYSTEM\CurrentControlSet\Control\FileSystem' -Name 'LongPathsEnabled' -Value 1
  -PropertyType DWORD -Force
```

9. Official download links

- Android Studio: https://developer.android.com/studio
- Git for Windows: https://git-scm.com/download/win
- Platform-Tools: https://developer.android.com/tools/releases/platform-tools

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 01.2.1 - Install or Update Only What Is Missing
TASK: From my gap list, give the minimum installation steps and tell me the approximate download size of each so
  I can decide what to skip.
CONTEXT TO ATTACH: Paste the gap list from the environment report.
FILES TO CREATE OR MODIFY: Only the files needed for this step; list them before writing code.
RULES: Only items in my gap list. Flag anything optional. Native Android only: Kotlin, Jetpack Compose, Material
  3, MapLibre Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via
  AppContainer. Do not add a dependency that is not already in the version catalog; if one is needed, stop and
  say so. Store geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded
  user-visible strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use start/end,
  never left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim something
  works unless it was built and tested.
OUTPUT: Ordered list: action, where to click, approximate size, 'skip if' condition.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-01.2.1. If the result fails verification: git restore . (or git reset --hard before-01.2.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

All blocking gaps closed; optional items consciously skipped.

12. Verification procedure

Re-run of the matching 01.1 command shows success.

13. Acceptance criteria

- [ ] No tool was reinstalled that already worked.
- [ ] Downloaded installers are kept for reuse.
- [ ] ENVIRONMENT_REPORT.md updated with new versions.

14. Common errors

- Download stalls -> pause and resume in SDK Manager; do not cancel and restart.
- 'Licenses not accepted' -> in SDK Manager click the prompt, or run sdkmanager --licenses.

15. Troubleshooting

1) Re-run the command exactly as printed. 2) Compare your output with the 'Expected result'. 3) Paste command and output to the AI with 'Interpret this; do not suggest reinstalling anything unless it is proven missing.' 4) Record UNVERIFIED if still unclear.

16. Dependencies

Depends on step 01.1.10; leads to the next step in the phase order.

17. Estimated download / storage

Variable; typically 0.1-0.5 GB for SDK items; Android Studio update about 1 GB class if chosen.

STEP 01.2.2 — Configure Gradle for Low Bandwidth and Offline Reuse

Provenance: [REC]

1. Step ID / 2. Title

01.2.2 — Configure Gradle for Low Bandwidth and Offline Reuse (type: environment check)

3. Objective

Set Gradle options that reduce downloads and memory use, and learn how to build offline once dependencies are cached.

4. Why it is necessary

Gradle downloads many libraries once, then reuses the cache. Correct settings avoid repeat downloads and leave RAM for LM Studio.

5. Prerequisites

Step 01.2.1 done and verified (tracker ticked). Phase Gate: the environment report is complete, one physical device is visible in adb, and total planned downloads are known.

6. Required software

Windows PowerShell, adb (Platform-Tools), a text editor; an AI tool from the AI field for interpretation.

7. Exact instructions

1. Understand the two folders: the project (small) and the Gradle user home (large cache, default C:\Users\<you>\.gradle). Do not delete the cache casually.
2. These settings go into the project's gradle.properties in 02.2.2: org.gradle.jvmargs=-Xmx3g (leave memory for other tools), org.gradle.caching=true, org.gradle.parallel=true, kotlin.daemon.jvm.options=-Xmx2g. [REC]
3. Offline mode: after one successful sync, Android Studio's Gradle settings have an 'Offline work' checkbox; the command line uses --offline.
4. If C: is short on space and another drive has room, set the user environment variable GRADLE_USER_HOME to the other drive before the first sync (command below; sign out and in). Skip this if C: has 25 GB or more free.
5. Write the offline rule in the report: 'Never delete .gradle or run clean-cache commands unless disk is critically low.'

8. Terminal commands

Where to run: PowerShell; Android Studio Settings > Build, Execution, Deployment > Gradle.

```
# OPTIONAL, only if relocating the Gradle cache. What: sets a per-user variable. Success: prints the path after
  restart.
[Environment]::SetEnvironmentVariable('GRADLE_USER_HOME','D:\gradle-home','User')
# After first successful sync, command-line offline build (run in the project folder):
.\gradlew.bat assembleDebug --offline
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 01.2.2 - Configure Gradle for Low Bandwidth and Offline Reuse
TASK: Explain in simple words what gradle.properties options reduce RAM use and repeated downloads for a 16 GB
  Windows PC that also runs a local LLM.
CONTEXT TO ATTACH: State RAM, disk, slow internet.
FILES TO CREATE OR MODIFY: Only the files needed for this step; list them before writing code.
RULES: No experimental flags. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: A gradle.properties snippet with a comment on each line.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-01.2.2. If the result fails verification: git restore . (or git reset --hard before-01.2.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Chosen Gradle options recorded; cache location decided.

12. Verification procedure

After 02.2.4, a second build with --offline succeeds.

13. Acceptance criteria

- [ ] Cache location decided.
- [ ] Gradle options recorded for 02.2.2.

14. Common errors

- 'No cached version available for offline mode' -> run one online sync first, then retry.

15. Troubleshooting

1) Re-run the command exactly as printed. 2) Compare your output with the 'Expected result'. 3) Paste command and output to the AI with 'Interpret this; do not suggest reinstalling anything unless it is proven missing.' 4) Record UNVERIFIED if still unclear.

16. Dependencies

Depends on step 01.2.1; leads to the next step in the phase order.

17. Estimated download / storage

Cache grows with use; see storage budget.

STEP 01.2.3 — Decide on Emulator Use (Optional, Default: Skip)

Provenance: [REC]

1. Step ID / 2. Title

01.2.3 — Decide on Emulator Use (Optional, Default: Skip) (type: decision)

3. Objective

Record the decision to use physical devices only, and define the exact conditions under which one emulator image may be installed.

4. Why it is necessary

An emulator image costs 1.5 GB or more and is not needed for most tests; the spec's Section 17.2 item 9 notes an emulator can partly cover external display and a second device.

5. Prerequisites

Step 01.2.2 done and verified (tracker ticked). Phase Gate: the environment report is complete, one physical device is visible in adb, and total planned downloads are known.

6. Required software

A web browser for official pages, VS Code, Git, one AI tool from the AI field.

7. Exact instructions

1. Default decision: no emulator. Use the Lenovo Tab P12 and Samsung Galaxy A16.
2. Allowed exception 1: stress sizes you do not own (for example 1920x1200 or 915x412 dp) can be approximated on the tablet with 'adb shell wm size' and 'wm density' overrides (reset afterwards with 'wm size reset'). Try this first; it costs nothing.
3. Allowed exception 2: a second display can be simulated on a real device with Developer options > 'Simulate secondary displays'. This tests the Presentation code path but is NOT hardware evidence.
4. Only if both exceptions are insufficient: install ONE x86_64 Google-free (AOSP) system image of the newest stable API after checking hardware virtualization support (Task Manager > Performance > CPU > Virtualization: Enabled). Record its size before downloading.
5. Write the decision in MAPX_DECISIONS.md later (02.1.4) as D-R01.

8. Terminal commands

Where to run: PowerShell with a device connected.

```
# What: temporary screen-size override for layout stress tests. Success: the device re-lays-out. Always reset.
$adb = "$env:LOCALAPPDATA\Android\Sdk\platform-tools\adb.exe"
& $adb shell wm size 1200x1920     # example portrait tablet stress size (pixels, not dp)
& $adb shell wm density            # note current density
& $adb shell wm size reset
& $adb shell wm density reset
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).

Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 01.2.3 - Decide on Emulator Use (Optional, Default: Skip)
TASK: List the three cheapest ways to test different screen sizes and a second display on a real Android tablet
  without installing an emulator image.
CONTEXT TO ATTACH: Mention Lenovo Tab P12, adb wm size, Developer option 'Simulate secondary displays'.
FILES TO CREATE OR MODIFY: Only the files needed for this step; list them before writing code.
RULES: Do not recommend downloading images. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre
  Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do
  not add a dependency that is not already in the version catalog; if one is needed, stop and say so. Store
  geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible
  strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never
  left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim something works
  unless it was built and tested.
OUTPUT: Numbered list with exact settings paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-01.2.3. If the result fails verification: git restore . (or git reset --hard before-01.2.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Emulator decision recorded as 'not installed'.

12. Verification procedure

No emulator image present in SDK Manager.

13. Acceptance criteria

[ ] Decision recorded.
[ ] Exceptions documented.

14. Common errors

- wm size leaves the device in a strange state -> run wm size reset and wm density reset, then reboot.

15. Troubleshooting

1) Reopen the official page and copy the exact text or version. 2) If two sources disagree, prefer the specification, then the official page, and record the conflict in MAPX_DECISIONS.md.

16. Dependencies

Depends on step 01.2.2; leads to the next step in the phase order.

17. Estimated download / storage

No download by default.

STAGE 01.3 — Establish the Free-Tier AI Workflow

A repeatable way to use several free AI tools without losing project context.

STEP 01.3.1 — Build the AI Tool Matrix and Fallback Ladder

Provenance: [REC] [ASSUMED: 'Cloud Free' means Claude]

1. Step ID / 2. Title

01.3.1 — Build the AI Tool Matrix and Fallback Ladder (type: document)

3. Objective

Decide which AI tool handles which kind of task and what to do when a free limit is reached.

4. Why it is necessary

Free tiers change without notice and have message, token and rate limits. A written ladder prevents stalls.

5. Prerequisites

Step 01.2.3 done and verified (tracker ticked). Phase Gate: the environment report is complete, one physical device is visible in adb, and total planned downloads are known.

6. Required software

VS Code or any text editor, Git, one AI tool from the AI field.

7. Exact instructions

1. Create AI_WORKFLOW.md in the docs folder. Copy the tool matrix and fallback ladder from Appendix C of this roadmap.
2. Open each platform once and record, in your own words, the current free limit message you see (do not trust any number in this document: limits change).
3. Rule 1: the project documents (not chat history) are the memory. Every new chat starts with the Context Pack (01.3.3).
4. Rule 2: one task per prompt, one answer verified before the next prompt.
5. Rule 3: when a cloud AI hits its limit, copy the same prompt to the next tool on the ladder; the final fallback is local Qwen with the prompt split into one file per request.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
# No command required. Commit when finished:
git add -A
git commit -m "docs: step done"
```

9. Official download links

- Google AI Studio: https://aistudio.google.com/
- ChatGPT: https://chatgpt.com/
- Gemini: https://gemini.google.com/
- Claude: https://claude.ai/

10. Complete AI implementation prompt

Which AI: Primary: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 01.3.1 - Build the AI Tool Matrix and Fallback Ladder
TASK: Review my AI tool ladder for a non-programmer building an Android app using only free tiers. Find
  weaknesses and propose improvements that keep every essential step possible with the local Qwen model alone.
CONTEXT TO ATTACH: Paste AI_WORKFLOW.md.
FILES TO CREATE OR MODIFY: Only the files needed for this step; list them before writing code.
RULES: No paid options. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Weakness list with fixes.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-01.3.1. If the result fails verification: git restore . (or git reset --hard before-01.3.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

AI_WORKFLOW.md exists.

12. Verification procedure

It lists a fallback for each tool.

13. Acceptance criteria

[ ] Ladder written.

[ ] No step depends on a paid service.

14. Common errors

- A platform changes its limits -> update the 'observed limits' line only; the ladder stays.

15. Troubleshooting

1) Compare the document with the listed sources. 2) Ask the AI to list contradictions between the document and the specification. 3) Fix and commit.

16. Dependencies

Depends on step 01.2.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 01.3.2 — Configure LM Studio as a Local Coding Assistant

Provenance: [REC]

1. Step ID / 2. Title

01.3.2 — Configure LM Studio as a Local Coding Assistant (type: environment check)

3. Objective

Enable LM Studio's local server and set a coding system prompt for Qwen.

4. Why it is necessary

A saved system prompt makes Qwen follow MapX rules without retyping them.

5. Prerequisites

Step 01.3.1 done and verified (tracker ticked). Phase Gate: the environment report is complete, one physical device is visible in adb, and total planned downloads are known.

6. Required software

Windows PowerShell, adb (Platform-Tools), a text editor; an AI tool from the AI field for interpretation.

7. Exact instructions

1. In LM Studio create a preset named 'MapX Coder' with the System Prompt from Appendix B.
2. Settings: temperature 0.2, context 8192 (or the value from 01.1.8), max tokens 2048.
3. Optional: start the local server (Developer tab) so AntiGravity or VS Code extensions can use it. Only listen on localhost.
4. Test with: 'Write a Kotlin data class GeoPoint(lat, lon) that validates ranges and a JUnit test.' Compare to Spec 7.3 (lat in [-90,90], lon in [-180,180]).

8. Terminal commands

Where to run: LM Studio.

```
# No command required beyond those in the instructions.
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 01.3.2 - Configure LM Studio as a Local Coding Assistant
TASK: Write a system prompt for a local 7B Kotlin coding model that enforces: Kotlin only, Jetpack Compose, no
  new dependencies, return complete files with paths, include JUnit tests.
CONTEXT TO ATTACH: MapX stack from Spec 7.1.
FILES TO CREATE OR MODIFY: Only the files needed for this step; list them before writing code.
RULES: Max 200 words. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: One system prompt.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-01.3.2. If the result fails verification: git restore . (or git reset --hard before-01.3.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Preset saved; test prompt produces a validating GeoPoint.

12. Verification procedure

The generated class rejects lat 91.

13. Acceptance criteria

[ ] Preset saved.
[ ] Local server (if used) bound to localhost only.

14. Common errors

- Qwen ignores the system prompt -> repeat the rules at the top of the user prompt.

15. Troubleshooting

1) Re-run the command exactly as printed. 2) Compare your output with the 'Expected result'. 3) Paste command and output to the AI with 'Interpret this; do not suggest reinstalling anything unless it is proven missing.' 4) Record UNVERIFIED if still unclear.

16. Dependencies

Depends on step 01.3.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 01.3.3 — Create the Context Pack Generator

Provenance: [REC]

1. Step ID / 2. Title

01.3.3 — Create the Context Pack Generator (type: AI-assisted coding)

3. Objective

Create a script that builds one MAPX_CONTEXT_PACK.md (specification digest, current status, handoff, and the files relevant to the current step) to paste into any AI.

4. Why it is necessary

This is the mechanism that moves project knowledge between AI platforms without relying on conversation memory.

5. Prerequisites

Step 01.3.2 done and verified (tracker ticked). Phase Gate: the environment report is complete, one physical device is visible in adb, and total planned downloads are known.

6. Required software

PowerShell, VS Code.

7. Exact instructions

1. Create C:\dev\MapX_docs\tools\make-context.ps1 (PowerShell, no Python required) with the AI prompt below.
2. The script concatenates: STANDING ORDERS (Appendix B), MAPX_HANDOFF.md, MAPX_STATUS.md, a chosen list of spec sections, and any source files named on the command line, then reports the approximate token count (characters divided by 4).
3. It warns if the pack exceeds 5,000 tokens for Qwen or 60,000 for AI Studio (limits are guidance; confirm yours).
4. Run it with sample arguments and open the output.

8. Terminal commands

Where to run: PowerShell in C:\dev\MapX_docs (later moved into the repo as tools\).

```
# What: builds a pasteable context file. Success: prints the output path and an estimated token count.
.\tools\make-context.ps1 -Spec "5.1,5.5" -Files "app\src\main\java\app\mapx\core\model\ProjectModels.kt"
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 01.3.3 - Create the Context Pack Generator
TASK: Write a Windows PowerShell 5.1-compatible script make-context.ps1 that builds MAPX_CONTEXT_PACK.md.
CONTEXT TO ATTACH: Inputs: docs/MAPX_HANDOFF.md, docs/MAPX_STATUS.md, docs/STANDING_ORDERS.md,
  docs/MapX_Master_Project_Specification.md (extract sections by heading number such as '## 5.1').
FILES TO CREATE OR MODIFY: tools/make-context.ps1
RULES: No external modules. UTF-8 without BOM. Print estimated tokens = chars/4. Fail with a clear message if a
  file is missing. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room, DataStore,
  kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a dependency that
  is not already in the version catalog; if one is needed, stop and say so. Store geographic data as WGS84
  lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use string
  resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or update
  unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: The complete script and three example invocations.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-01.3.3. If the result fails verification: git restore . (or git reset --hard before-01.3.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

A context pack file is created and is readable.

12. Verification procedure

Open the file; the sections appear in order; the token estimate prints.

13. Acceptance criteria

- [ ] Spec sections can be extracted by number.
- [ ] Missing file produces a clear error.
- [ ] Output is plain Markdown.

14. Common errors

- Execution policy blocks the script -> run: powershell -ExecutionPolicy Bypass -File .\tools\make-context.ps1 ...
- Spec sections not found -> headings must be copied exactly (for example '### 5.1 Projects and saving').

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 01.3.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 01.3.4 — Create the Standing Orders and Handoff Routine

Provenance: [SPEC 18.1]

1. Step ID / 2. Title

01.3.4 — Create the Standing Orders and Handoff Routine (type: document)

3. Objective

Create STANDING_ORDERS.md (rules every AI must follow) and define the 5-minute end-of-session routine that updates MAPX_HANDOFF.md and the tracker.

4. Why it is necessary

Spec 18.1 rules must reach every AI session; a consistent end-of-session routine keeps documents truthful.

5. Prerequisites

Step 01.3.3 done and verified (tracker ticked). Phase Gate: the environment report is complete, one physical device is visible in adb, and total planned downloads are known.

6. Required software

VS Code or any text editor, Git, one AI tool from the AI field.

7. Exact instructions

1. Copy Appendix B into docs\STANDING_ORDERS.md.
2. End-of-session routine (write it in AI_WORKFLOW.md): (1) run the build and tests; (2) commit with a message that names the step ID; (3) tick the step in MapX_Progress_Tracker.md only if its acceptance criteria are met; (4) update MAPX_HANDOFF.md 'what is built / known failures / next action'; (5) tag: git tag done-<stepID>.
3. Rule: never tick a step because code was generated; tick it only after the verification procedure in that step passed.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
# No command required. Commit when finished:
git add -A
git commit -m "docs: step done"
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).

Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 01.3.4 - Create the Standing Orders and Handoff Routine
TASK: Turn my end-of-session checklist into a one-page routine that a non-programmer can follow, with exact git
  commands.
CONTEXT TO ATTACH: Paste the five checklist lines.
FILES TO CREATE OR MODIFY: Only the files needed for this step; list them before writing code.
RULES: Windows PowerShell only. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Numbered routine with commands.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-01.3.4. If the result fails verification: git restore . (or git reset --hard before-01.3.4), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

STANDING_ORDERS.md and the routine exist.

12. Verification procedure

Files present.

13. Acceptance criteria

[ ] Orders include Spec 18.1 rules 1-15 in short form.

14. Common errors

- AI invents facts -> require it to quote the specification section it relied on.

15. Troubleshooting

1) Compare the document with the listed sources. 2) Ask the AI to list contradictions between the document and the specification. 3) Fix and commit.

16. Dependencies

Depends on step 01.3.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

PHASE 02 — PROJECT FOUNDATION

Create a brand-new native Android project and the six engineering documents required by Spec 18.2. No old MapX code is reused. The Version Gate pins current dependency versions from official sources instead of copying anything from old documents.

Estimated relative effort: M. Gate: a clean debug build installs and starts on both devices; the six documents exist; architecture tests run.

STAGE 02.1 — Repository and Engineering Documents

STEP 02.1.1 — Create the Repository, .gitignore and Backup Routine

Provenance: [REC]

1. Step ID / 2. Title

02.1.1 — Create the Repository, .gitignore and Backup Routine (type: environment check)

3. Objective

Create C:\dev\MapX as an empty Git repository with a safe .gitignore and a second-copy backup routine.

4. Why it is necessary

A clean start and a reliable backup are the recovery plan for every AI mistake and for a disk failure.

5. Prerequisites

Step 01.3.4 done and verified (tracker ticked). Phase Gate: a clean debug build installs and starts on both devices; the six documents exist; architecture tests run.

6. Required software

Windows PowerShell, adb (Platform-Tools), a text editor; an AI tool from the AI field for interpretation.

7. Exact instructions

1. Create the folder and initialise Git (commands below).
2. Create .gitattributes and .gitignore (content in the AI prompt output; typical entries: build/, .gradle/, local.properties, *.jks, *.keystore, .idea/ except shared run configs, captures/).
3. Never commit signing keys or local.properties.
4. Create the backup script backup.ps1: copies the repo (excluding build and .gradle) to a second location (another drive or USB) with a dated folder name; run it at the end of every phase.
5. Move ENVIRONMENT_REPORT.md and tools\ into the repo's docs\ and tools\ folders.

8. Terminal commands

Where to run: PowerShell.

```
# What: creates the repository. Success: 'Initialized empty Git repository'.
mkdir C:\dev\MapX
cd C:\dev\MapX
git init -b main
mkdir docs, tools
git add -A
git commit --allow-empty -m "chore: empty repository (step 02.1.1)"
git tag done-02.1.1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).

Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 02.1.1 - Create the Repository, .gitignore and Backup Routine
TASK: Write .gitignore and .gitattributes for a Kotlin Android Gradle project on Windows, plus a PowerShell
  backup.ps1 that copies the repo without build outputs to a dated folder on another drive.
CONTEXT TO ATTACH: Windows 11, PowerShell 5.1.
FILES TO CREATE OR MODIFY: .gitignore, .gitattributes, tools/backup.ps1
RULES: Exclude keystores, local.properties, build outputs, .gradle. Native Android only: Kotlin, Jetpack
  Compose, Material 3, MapLibre Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels,
  manual DI via AppContainer. Do not add a dependency that is not already in the version catalog; if one is
  needed, stop and say so. Store geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command.
  No hard-coded user-visible strings (use string resources, English and Arabic). Touch targets at least 48 dp.
  Use start/end, never left/right. Write or update unit tests. Do not weaken or delete existing tests. Never
  claim something works unless it was built and tested.
OUTPUT: The three complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-02.1.1. If the result fails verification: git restore . (or git reset --hard before-02.1.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result
Repo created; backup script runs.

12. Verification procedure
git status is clean; backup folder appears with a dated name.

13. Acceptance criteria
[ ] Signing keys can never be committed.
[ ] Backup tested once.

14. Common errors
- 'fatal: not a git repository' -> you are in the wrong folder; cd C:\dev\MapX.

15. Troubleshooting
1) Re-run the command exactly as printed. 2) Compare your output with the 'Expected result'. 3) Paste command and output to the AI with 'Interpret this; do not suggest reinstalling anything unless it is proven missing.' 4) Record UNVERIFIED if still unclear.

16. Dependencies
Depends on step 01.3.4; leads to the next step in the phase order.

17. Estimated download / storage
No download.

STEP 02.1.2 — Install the Two Source Documents in docs/
Provenance: [SPEC 0.3]

1. Step ID / 2. Title
02.1.2 — Install the Two Source Documents in docs/ (type: document)

3. Objective
Copy MapX_Master_Project_Specification.md and MapX_UI_UX_Design_Guide.pdf into docs/ as read-only references.

4. Why it is necessary
These are the authorities (Spec first; Guide second; Layout Blueprint inside the Guide). They must travel with the code.

5. Prerequisites
Step 02.1.1 done and verified (tracker ticked). Phase Gate: a clean debug build installs and starts on both devices; the six documents exist; architecture tests run.

6. Required software
File Explorer, Git.

7. Exact instructions
1. Copy both files into C:\dev\MapX\docs\source\.
2. Create docs\source\README.md stating the document priority: Spec > UI/UX Guide > Layout Blueprint (as amended by BA-01..BA-17).
3. Create docs\source\SPEC_SECTIONS.md listing section numbers you will quote (5.1, 5.5, 6.3, 9.1, 12.4, 16.1) to make context packs easy.
4. Commit.

8. Terminal commands
Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
# No command required. Commit when finished:
git add -A
git commit -m "docs: step done"
```

9. Official download links
None required for this step.

10. Complete AI implementation prompt
Which AI: Primary: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 02.1.2 - Install the Two Source Documents in docs/
TASK: Create a README stating the document priority and conflict-resolution rule for a project that has a
  specification, a UI/UX guide and an embedded layout blueprint.
CONTEXT TO ATTACH: Priority list from the master prompt.
FILES TO CREATE OR MODIFY: Only the files needed for this step; list them before writing code.
RULES: Max 150 words. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: README.md text.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-02.1.2. If the result fails verification: git restore . (or git reset --hard before-02.1.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Source documents are versioned in the repo.

12. Verification procedure

git log shows the commit; files open.

13. Acceptance criteria

- [ ] Both documents in docs\source\.
- [ ] Priority README present.

14. Common errors

- AI invents facts -> require it to quote the specification section it relied on.

15. Troubleshooting

1) Compare the document with the listed sources. 2) Ask the AI to list contradictions between the document and the specification. 3) Fix and commit.

16. Dependencies

Depends on step 02.1.1; leads to the next step in the phase order.

17. Estimated download / storage

About 1-2 MB of documents.

STEP 02.1.3 — Create ARCHITECTURE.md (Initial Version)

Provenance: [SPEC 18.2]

1. Step ID / 2. Title

02.1.3 — Create ARCHITECTURE.md (Initial Version) (type: document)

3. Objective

Write the first ARCHITECTURE.md from Spec 18.2 and Spec 7, including the format support matrix with an Evidence column that starts empty.

4. Why it is necessary

The Spec requires ARCHITECTURE.md as the engineering truth. It is updated at the end of every phase.

5. Prerequisites

Step 02.1.2 done and verified (tracker ticked). Phase Gate: a clean debug build installs and starts on both devices; the six documents exist; architecture tests run.

6. Required software

VS Code or any text editor, Git, one AI tool from the AI field.

7. Exact instructions

1. Use the AI prompt below to generate the initial file.
2. It must contain: product definition; platform; the four device modes; layout contract (laws L1-L12 summary); the 15 engineering rules verbatim from Spec 18.1; package structure and dependency rule (Spec 7.2); seven state domains (Spec 7.3); map abstraction; presentation/display/companion architecture; Format Support Matrix (Spec 6.3) with columns Format, Expected, Actual, Sample file used, Evidence; rules for modifying systems.
3. Keep 'Actual' and 'Evidence' cells empty: nothing is Supported until tested with a real sample (Spec 6.3).
4. Review against Spec 7 and fix omissions. Commit.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
# No command required. Commit when finished:
git add -A
git commit -m "docs: step done"
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 02.1.3 - Create ARCHITECTURE.md (Initial Version)
TASK: Write ARCHITECTURE.md for MapX exactly as specified.
CONTEXT TO ATTACH: Attach Spec sections 7.1, 7.2, 7.3, 6.3, 18.1, 18.2 and Guide section 19 (use the context
  pack generator).
FILES TO CREATE OR MODIFY: docs/ARCHITECTURE.md
RULES: Do not add technologies not in Spec 7.1. Do not mark any format Supported. Keep class names exactly as in
  Spec 7.2. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room, DataStore,
  kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a dependency that
  is not already in the version catalog; if one is needed, stop and say so. Store geographic data as WGS84
  lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use string
  resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or update
  unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: One Markdown file.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-02.1.3. If the result fails verification: git restore . (or git reset --hard before-02.1.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

ARCHITECTURE.md committed.

12. Verification procedure

Spot-check: 15 rules present; 7 state domains present; matrix has the Evidence column.

13. Acceptance criteria

- [ ] All required class names listed.
- [ ] No format marked Supported.

14. Common errors

- AI shortens the 15 rules -> ask it to paste them verbatim from the context.

15. Troubleshooting

1) Compare the document with the listed sources. 2) Ask the AI to list contradictions between the document and the specification. 3) Fix and commit.

16. Dependencies

Depends on step 02.1.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 02.1.4 — Create the Other Five Engineering Documents

Provenance: [SPEC 18.2] [ASSUMED C-01, C-08]

1. Step ID / 2. Title

02.1.4 — Create the Other Five Engineering Documents (type: document)

3. Objective

Create MAPX_DECISIONS.md, MAPX_HANDOFF.md, MAPX_STATUS.md, SYMBOLS_LICENSE.md and SECURITY.md in initial form.

4. Why it is necessary

Spec 18.2 requires all six documents. They are the cross-AI memory.

5. Prerequisites

Step 02.1.3 done and verified (tracker ticked). Phase Gate: a clean debug build installs and starts on both devices; the six documents exist; architecture tests run.

6. Required software

VS Code or any text editor, Git, one AI tool from the AI field.

7. Exact instructions

1. MAPX_DECISIONS.md: decision log table (ID, date, decision, reason, alternatives, consequences). Seed it with Spec D-01..D-15 and S1 resolutions, DN-1..DN-10 defaults (from the master prompt), and these roadmap decisions: D-R01 no emulator by default; D-R02 targetSdk strategy (decided in 02.1.5); D-R03 lightweight state-based navigation instead of Navigation library; D-R04 no GitHub or cloud; D-R05 applicationId app.mapx (assumption).
2. MAPX_STATUS.md: one row per feature area with status columns Implemented / Built / Automated-test verified / Device verified / Not tested / Deferred, all starting 'Not built'.

3. MAPX_HANDOFF.md: what MapX is; nothing built yet; what must NOT change; recovery procedure (git tags); next action.
4. SYMBOLS_LICENSE.md: header and an empty table (symbol id, name, author, licence, source).
5. SECURITY.md: skeleton headings for account/password design, lock behaviour, pairing protocol, permission table (filled in Phases 06, 16, 17).
6. Record the result of DN-9 from 01.1.9 and any palette decisions: alert red #E3201B is authoritative (register C-02); phone portrait dock = Select, Pan, Draw, Layers, More (register C-01, flagged for owner confirmation; this roadmap uses the Guide's order as the documented default).

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
# No command required. Commit when finished:
git add -A
git commit -m "docs: step done"
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 02.1.4 - Create the Other Five Engineering Documents
TASK: Create the five engineering documents with correct skeletons and seed MAPX_DECISIONS.md with the listed
  decisions.
CONTEXT TO ATTACH: Spec 3.1, 3.2, 17.1, 18.2; master prompt section 10 defaults; Guide register C-01, C-02,
  C-08, C-09.
FILES TO CREATE OR MODIFY: docs/MAPX_DECISIONS.md, docs/MAPX_HANDOFF.md, docs/MAPX_STATUS.md,
  docs/SYMBOLS_LICENSE.md, docs/SECURITY.md
RULES: Do not claim anything is built. Keep every status 'Not built'. Native Android only: Kotlin, Jetpack
  Compose, Material 3, MapLibre Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels,
  manual DI via AppContainer. Do not add a dependency that is not already in the version catalog; if one is
  needed, stop and say so. Store geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command.
  No hard-coded user-visible strings (use string resources, English and Arabic). Touch targets at least 48 dp.
  Use start/end, never left/right. Write or update unit tests. Do not weaken or delete existing tests. Never
  claim something works unless it was built and tested.
OUTPUT: Five Markdown files, separated by file-path headings.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-02.1.4. If the result fails verification: git restore . (or git reset --hard before-02.1.4), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

All six engineering documents exist.

12. Verification procedure

Open each file; MAPX_STATUS has all rows 'Not built'.

13. Acceptance criteria

- [ ] Six documents present (with ARCHITECTURE.md).
- [ ] DN-1..DN-10 defaults recorded.
- [ ] Open UI items C-01/C-08/C-09 recorded as assumptions.

14. Common errors

- AI invents facts -> require it to quote the specification section it relied on.

15. Troubleshooting

1) Compare the document with the listed sources. 2) Ask the AI to list contradictions between the document and the specification. 3) Fix and commit.

16. Dependencies

Depends on step 02.1.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 02.1.5 — Run the Version Gate (Verify and Pin Dependency Versions)

Provenance: [VERIFIED 4 Nov 2026, to be rechecked] [REC]

1. Step ID / 2. Title

02.1.5 — Run the Version Gate (Verify and Pin Dependency Versions) (type: decision)

3. Objective

Look up the current stable versions of every dependency from official sources, check compatibility, and record the pinned set in MAPX_DECISIONS.md with purpose, size, licence and alternatives.

4. Why it is necessary

The master prompt forbids copying old versions. Compatibility between Android Studio, AGP, Gradle, Kotlin, Compose, Room and MapLibre must be checked together before coding.

5. Prerequisites

Step 02.1.4 done and verified (tracker ticked). Phase Gate: a clean debug build installs and starts on both devices; the six documents exist; architecture tests run.

6. Required software

A web browser for official pages, VS Code, Git, one AI tool from the AI field.

7. Exact instructions

1. Open each official page below and write the current STABLE version (not alpha/beta/RC): Android Studio and AGP; Gradle; Kotlin; Compose BOM; Material 3; Room; DataStore; Lifecycle/ViewModel; Activity; kotlinx.serialization; kotlinx.coroutines; CameraX; MapLibre Native Android (org.maplibre.gl:android-sdk); Java-WebSocket; ZXing-core; LeakCanary.
2. Research findings on 4 Nov 2026 [VERIFIED, recheck]: MapLibre Android has moved well past the 11.x line cited in the specification (versions in the 13.x series appear on Maven Central, with 13.7.1 listed for a related artifact on 16 Oct 2026); 16 KB page-size support was added in MapLibre Android 11.5.0 per its release notes; Java-WebSocket's latest published release is 1.6.0 (MIT); Android 17 (API 37) reached stable on 16 June 2026.
3. Rule for MapLibre: pin the newest STABLE release that passes spikes 03.1.1-03.1.4. The Spec requires 11.7.0 or newer for PMTiles; a newer major version may change APIs, so the spike decides, not this document.
4. targetSdk decision [REC]: compileSdk = newest stable platform installed (36 or 37). targetSdk = 36 for the first build. Reason: targeting 37 makes ACCESS_LOCAL_NETWORK a mandatory runtime permission for the phone remote and removes large-screen orientation opt-outs. MapX is private-use (no Play deadline). Raising to 37 is a deliberate later task in Phase 21 (Step 21.3.2). Record as D-R02.
5. Let Android Studio's New Project template (02.2.1) generate its own AGP/Kotlin/Compose versions, then compare with what you found. Where they differ, prefer the template's AGP, Gradle and Kotlin (they are co-tested) and the BOM.
6. For each non-AndroidX dependency fill the table: purpose, why required, size (from Maven Central page if shown, else 'unknown'), licence, alternatives, compatibility notes. Dependencies allowed by Spec 11 only; add none.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
# No command required. Record the decision in docs\MAPX_DECISIONS.md and commit.
```

9. Official download links

- Android Studio and AGP: https://developer.android.com/studio/releases
- Gradle: https://gradle.org/releases/
- Kotlin: https://kotlinlang.org/docs/releases.html
- Compose BOM: https://developer.android.com/jetpack/compose/bom
- Room: https://developer.android.com/jetpack/androidx/releases/room
- DataStore: https://developer.android.com/jetpack/androidx/releases/datastore
- CameraX: https://developer.android.com/jetpack/androidx/releases/camera
- MapLibre Native: https://github.com/maplibre/maplibre-native
- Java-WebSocket: https://github.com/TooTallNate/Java-WebSocket
- 16 KB page sizes: https://developer.android.com/guide/practices/page-sizes
- Local network permission: https://developer.android.com/privacy-and-security/local-network-permission

10. Complete AI implementation prompt

Which AI: Primary: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) Fallback: Google AI Studio free tier (best for large, multi-file generation because of its long context) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 02.1.5 - Run the Version Gate (Verify and Pin Dependency Versions)
TASK: Using only the version numbers I paste (copied from the official pages today), check the combination for
  compatibility problems and fill the dependency table for MAPX_DECISIONS.md.
CONTEXT TO ATTACH: Paste my version list and the dependency table template.
FILES TO CREATE OR MODIFY: Only the files needed for this step; list them before writing code.
RULES: Do not invent versions. If a version is missing, say 'verify'. Native Android only: Kotlin, Jetpack
  Compose, Material 3, MapLibre Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels,
  manual DI via AppContainer. Do not add a dependency that is not already in the version catalog; if one is
  needed, stop and say so. Store geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command.
  No hard-coded user-visible strings (use string resources, English and Arabic). Touch targets at least 48 dp.
  Use start/end, never left/right. Write or update unit tests. Do not weaken or delete existing tests. Never
  claim something works unless it was built and tested.
OUTPUT: A compatibility verdict and a filled table (purpose, why, size, licence, alternatives, compatibility).
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-02.1.5. If the result fails verification: git restore . (or git reset --hard before-02.1.5), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Pinned set recorded as decision D-R06; targetSdk decision D-R02 recorded.

12. Verification procedure

Every dependency row has a version copied from an official page and a date.

13. Acceptance criteria

[ ] No dependency lacks purpose/licence/alternative.
[ ] MapLibre version marked 'provisional until spike passes'.
[ ] targetSdk decision recorded.

14. Common errors

- Official page unreachable on slow connection -> use Android Studio's template versions and mark 'unverified'.

15. Troubleshooting

1) Reopen the official page and copy the exact text or version. 2) If two sources disagree, prefer the specification, then the official page, and record the conflict in MAPX_DECISIONS.md.

16. Dependencies

Depends on step 02.1.4; leads to the next step in the phase order.

17. Estimated download / storage

Web pages only.

STAGE 02.2 — Create the New Android Project

STEP 02.2.1 — Generate the Compose Project in Android Studio

Provenance: [SPEC 7.1] [REC]

1. Step ID / 2. Title

02.2.1 — Generate the Compose Project in Android Studio (type: device test)

3. Objective

Create a new single-module Compose project with minSdk 28 in C:\dev\MapX.

4. Why it is necessary

Starting from Studio's own template gives a known-good set of co-tested versions.

5. Prerequisites

Step 02.1.5 done and verified (tracker ticked). Phase Gate: a clean debug build installs and starts on both devices; the six documents exist; architecture tests run.

6. Required software

Android Studio, Android SDK.

7. Exact instructions

1. File > New > New Project > 'Empty Activity' (Compose / Material 3). If the template shows several, choose the plain Compose one.
2. Name: MapX. Package name: app.mapx [ASSUMED, D-R05]. Save location: C:\dev\MapX (the existing repo folder; allow it). Language Kotlin. Minimum SDK: API 28.
3. Build configuration language: Kotlin DSL (build.gradle.kts).
4. Wait for the first Gradle sync (needs internet; downloads the cache). Do not interrupt it.
5. Do not add any library yet. Open Build > Make Project to confirm it compiles.
6. Commit: 'chore: generate Compose project (02.2.1)' and tag done-02.2.1.

8. Terminal commands

Where to run: Android Studio.

```
.\gradlew.bat installDebug --offline
# Then run the app on the device named in the verification procedure and note the result.
```

9. Official download links

- https://developer.android.com/studio

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 02.2.1 - Generate the Compose Project in Android Studio
TASK: My Android Studio New Project wizard shows these options. Tell me exactly what to choose for a native
  Kotlin Compose, single-module, minSdk 28 app.
CONTEXT TO ATTACH: Paste any wizard text or error.
FILES TO CREATE OR MODIFY: Only the files needed for this step; list them before writing code.
RULES: Single module only; no Navigation, Hilt or other libraries. Native Android only: Kotlin, Jetpack Compose,
  Material 3, MapLibre Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI
  via AppContainer. Do not add a dependency that is not already in the version catalog; if one is needed, stop
  and say so. Store geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-
  coded user-visible strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use
  start/end, never left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim
  something works unless it was built and tested.
OUTPUT: Step-by-step choices.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-02.2.1. If the result fails verification: git restore . (or git reset --hard before-02.2.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Project builds; template app runs later.

12. Verification procedure

Build > Make Project finishes with 'BUILD SUCCESSFUL'.

13. Acceptance criteria

- [ ] minSdk 28.
- [ ] Single app module.
- [ ] Kotlin DSL.
- [ ] No extra libraries.

14. Common errors

- Sync fails with 'Could not resolve' -> internet dropped; click Try Again; do not delete caches.
- 'SDK location not found' -> set sdk.dir in local.properties (Studio does this) or choose SDK in the wizard.

15. Troubleshooting

1) Run: adb devices -l and confirm 'device'. 2) Uninstall and reinstall: adb uninstall app.mapx then installDebug. 3) Capture the crash: adb logcat -d | Select-String 'FATAL|AndroidRuntime' and give the lines to the AI. 4) Retest on the other device to separate device-specific from general faults.

16. Dependencies

Depends on step 02.1.5; leads to the next step in the phase order.

17. Estimated download / storage

Gradle distribution plus first dependencies: roughly 1-3 GB class on first sync (estimate).

STEP 02.2.2 — Convert to Version Catalog and Pin Versions

Provenance: [SPEC 11] [REC]

1. Step ID / 2. Title

02.2.2 — Convert to Version Catalog and Pin Versions (type: AI-assisted coding)

3. Objective

Move all versions into gradle/libs.versions.toml and add only the dependencies needed for Phases 03-06 now (later phases add theirs with a justification each).

4. Why it is necessary

A version catalog keeps pins in one place so AI edits cannot scatter versions. Adding dependencies only when needed limits download size.

5. Prerequisites

Step 02.2.1 done and verified (tracker ticked). Phase Gate: a clean debug build installs and starts on both devices; the six documents exist; architecture tests run.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Open gradle/libs.versions.toml (generated by the template). Keep Studio's generated AGP/Kotlin/Compose entries.
2. Add (using pinned versions from 02.1.5): Room runtime/ktx/compiler with KSP, DataStore preferences, kotlinx-serialization-json, kotlinx-coroutines-android and test, lifecycle viewmodel-compose, JUnit, androidx test, compose ui-test-junit4, LeakCanary (debug only).
3. Add gradle.properties options from 01.2.2.
4. Do NOT add MapLibre yet (added in the spike 03.1.1 on a branch), Java-WebSocket, CameraX, ZXing.
5. Sync, build, run unit test task: .\gradlew.bat testDebugUnitTest. Commit and tag.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
# What: builds and runs unit tests. Success: BUILD SUCCESSFUL.
cd C:\dev\MapX
.\gradlew.bat assembleDebug testDebugUnitTest
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 02.2.2 - Convert to Version Catalog and Pin Versions
TASK: Edit libs.versions.toml and app/build.gradle.kts to add the dependencies listed, using version.ref entries
  and KSP for Room. Keep every existing line that works.
CONTEXT TO ATTACH: Attach the current three files and the pinned versions from MAPX_DECISIONS D-R06.
FILES TO CREATE OR MODIFY: gradle/libs.versions.toml, app/build.gradle.kts, gradle.properties
RULES: No dependency outside the list. Do not downgrade AGP/Kotlin. Show a diff-style answer plus the full
  changed files. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room, DataStore,
  kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a dependency that
  is not already in the version catalog; if one is needed, stop and say so. Store geographic data as WGS84
  lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use string
  resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or update
  unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete replacement files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-02.2.2. If the result fails verification: git restore . (or git reset --hard before-02.2.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Project builds with the catalog.

12. Verification procedure

BUILD SUCCESSFUL for assembleDebug and testDebugUnitTest.

13. Acceptance criteria

- [ ] All versions live in the catalog.
- [ ] No MapLibre/WebSocket/CameraX yet.
- [ ] Pins match decision D-R06.

14. Common errors

- KSP version mismatch with Kotlin -> choose the KSP release that matches your Kotlin version (see KSP release notes).

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 02.2.1; leads to the next step in the phase order.

17. Estimated download / storage

Library downloads: tens of MB.

STEP 02.2.3 — Harden the Manifest (Backup, Permissions, Features)

Provenance: [SPEC 8, 10.4]

1. Step ID / 2. Title

02.2.3 — Harden the Manifest (Backup, Permissions, Features) (type: AI-assisted coding)

3. Objective

Apply the Spec's permission and backup rules from day one.

4. Why it is necessary

allowBackup must be false and unused permissions must never exist (Spec 8, 10.4).

5. Prerequisites

Step 02.2.2 done and verified (tracker ticked). Phase Gate: a clean debug build installs and starts on both devices; the six documents exist; architecture tests run.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Set android:allowBackup="false" and add data-extraction rules that exclude everything (res/xml/data_extraction_rules.xml and res/xml/backup_rules.xml).
2. Declare only: INTERNET, ACCESS_NETWORK_STATE, ACCESS_WIFI_STATE now. Location, camera and biometric permissions are added in the phases that use them.
3. Declare uses-feature android.hardware.location.gps, android.hardware.camera, android.hardware.touchscreen with required=false (added when each feature arrives, but write the touchscreen one now).
4. Do not set a screenOrientation lock; set resizeableActivity="true"; keep configChanges minimal (Compose handles it).
5. Add a lint check (Lint config) that fails the build on READ_EXTERNAL_STORAGE or MANAGE_EXTERNAL_STORAGE.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Google AI Studio free tier (best for large, multi-file generation because of its long context) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 02.2.3 - Harden the Manifest (Backup, Permissions, Features)
TASK: Write the AndroidManifest.xml, data_extraction_rules.xml and backup_rules.xml for MapX per the rules.
CONTEXT TO ATTACH: Spec 8 (platform backup), 10.4 (permissions), 12.2 (resizable, never lock orientation).
FILES TO CREATE OR MODIFY: app/src/main/AndroidManifest.xml, app/src/main/res/xml/data_extraction_rules.xml,
  app/src/main/res/xml/backup_rules.xml
RULES: No other permissions. No cleartext traffic except later loopback configuration (not yet). Native Android
  only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room, DataStore, kotlinx.serialization,
  Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a dependency that is not already in the
  version catalog; if one is needed, stop and say so. Store geographic data as WGS84 lat/lon, never screen
  pixels. Every edit is a command. No hard-coded user-visible strings (use string resources, English and
  Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or update unit tests. Do not
  weaken or delete existing tests. Never claim something works unless it was built and tested.
OUTPUT: Three complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-02.2.3. If the result fails verification: git restore . (or git reset --hard before-02.2.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Manifest conforms.

12. Verification procedure

Studio's Merged Manifest tab shows only the three permissions; build succeeds.

13. Acceptance criteria

[ ] allowBackup=false.
[ ] No storage permissions.
[ ] No orientation lock.

14. Common errors

- Library merges extra permissions -> use tools:node="remove" for permissions not allowed by the Spec.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 02.2.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 02.2.4 — First Build and Install on the Lenovo Tab P12

Provenance: [REC]

1. Step ID / 2. Title

02.2.4 — First Build and Install on the Lenovo Tab P12 (type: device test)

3. Objective

Install the template app on the tablet and confirm the debug workflow.

4. Why it is necessary

Proves the toolchain end-to-end before any real code exists.

5. Prerequisites

Step 02.2.3 done and verified (tracker ticked). Phase Gate: a clean debug build installs and starts on both devices; the six documents exist; architecture tests run.

6. Required software

Android Studio, adb, the Lenovo Tab P12 and/or Samsung Galaxy A16 (USB debugging on), Git.

7. Exact instructions

1. Connect the tablet (01.1.9). In Android Studio choose the device in the device list and press Run.
2. Confirm the app opens. Rotate the tablet; the app must rotate.
3. Run: adb shell dumpsys package app.mapx | findstr versionName to confirm install.
4. Build once more with --offline to prove the cache works (01.2.2).
5. Commit and tag.

8. Terminal commands

Where to run: PowerShell in C:\dev\MapX.

```
.\gradlew.bat installDebug --offline
adb shell am start -n app.mapx/.MainActivity
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 02.2.4 - First Build and Install on the Lenovo Tab P12
TASK: My app failed to install with this message. Explain the cause and the fix.
CONTEXT TO ATTACH: Paste the error.
FILES TO CREATE OR MODIFY: Only the files needed for this step; list them before writing code.
RULES: Native Android only. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Cause and one fix.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-02.2.4. If the result fails verification: git restore . (or git reset --hard before-02.2.4), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Template app visible on tablet.

12. Verification procedure

App launches; rotation works.

13. Acceptance criteria

[ ] Installed on Lenovo.
[ ] Offline rebuild succeeded.

14. Common errors

- INSTALL_FAILED_USER_RESTRICTED -> in Developer options enable 'Install via USB'.
- Device 'unauthorized' -> repeat 01.1.9 authorization.

15. Troubleshooting

1) Run: adb devices -l and confirm 'device'. 2) Uninstall and reinstall: adb uninstall app.mapx then installDebug. 3) Capture the crash: adb logcat -d | Select-String 'FATAL|AndroidRuntime' and give the lines to the AI. 4) Retest on the other device to separate device-specific from general faults.

16. Dependencies

Depends on step 02.2.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 02.2.5 — First Install on the Samsung Galaxy A16

Provenance: [REC]

1. Step ID / 2. Title
02.2.5 — First Install on the Samsung Galaxy A16 (type: device test)

3. Objective
Repeat the install on the phone.

4. Why it is necessary
Both device classes must work from the first build.

5. Prerequisites
Step 02.2.4 done and verified (tracker ticked). Phase Gate: a clean debug build installs and starts on both devices; the six documents exist; architecture tests run.

6. Required software
Android Studio, adb, the Lenovo Tab P12 and/or Samsung Galaxy A16 (USB debugging on), Git.

7. Exact instructions
1. Connect only the phone, or select it in the device list.
2. Run the app; rotate; note the screen size in dp (Developer: Settings > Developer options > 'Smallest width' shows dp).
3. Record in MAPX_STATUS that the toolchain is device-verified on both devices.

8. Terminal commands
Where to run: PowerShell.

```
.\gradlew.bat installDebug --offline
```

9. Official download links
None required for this step.

10. Complete AI implementation prompt
Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 02.2.5 - First Install on the Samsung Galaxy A16
TASK: Explain this Samsung-specific install problem.
CONTEXT TO ATTACH: Paste the error.
FILES TO CREATE OR MODIFY: Only the files needed for this step; list them before writing code.
RULES: No emulator. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room, DataStore,
  kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a dependency that
  is not already in the version catalog; if one is needed, stop and say so. Store geographic data as WGS84
  lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use string
  resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or update
  unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Fix.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-02.2.5. If the result fails verification: git restore . (or git reset --hard before-02.2.5), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result
Template app on phone.

12. Verification procedure
App launches.

13. Acceptance criteria
[ ] Installed on Samsung.

14. Common errors
- Samsung 'Auto Blocker' blocks USB installs -> turn Auto Blocker off in Settings > Security and privacy.

15. Troubleshooting
1) Run: adb devices -l and confirm 'device'. 2) Uninstall and reinstall: adb uninstall app.mapx then installDebug. 3) Capture the crash: adb logcat -d | Select-String 'FATAL|AndroidRuntime' and give the lines to the AI. 4) Retest on the other device to separate device-specific from general faults.

16. Dependencies
Depends on step 02.2.4; leads to the next step in the phase order.

17. Estimated download / storage
No download.

STEP 02.2.6 — Create and Back Up the Signing Key
Provenance: [SPEC 14]

1. Step ID / 2. Title
02.2.6 — Create and Back Up the Signing Key (type: environment check)

3. Objective
Create a release signing keystore and back it up in two places.

4. Why it is necessary
Spec 14: a lost key prevents updates over an installed copy. Do it now, not at release.

5. Prerequisites
Step 02.2.5 done and verified (tracker ticked). Phase Gate: a clean debug build installs and starts on both devices; the six documents exist; architecture tests run.

6. Required software
Windows PowerShell, adb (Platform-Tools), a text editor; an AI tool from the AI field for interpretation.

7. Exact instructions
1. Create the keystore with Studio (Build > Generate Signed App Bundle/APK > Create new) or keytool. Store it OUTSIDE the repo, for example C:\dev\keys\mapx-release.jks.
2. Use a strong password; store the password in a password manager or on paper, not in the repo.
3. Copy the keystore to a second location (USB or other drive).
4. Add a signingConfig that reads paths and passwords from a local, untracked keystore.properties file.
5. Record in SECURITY.md: location of backups (not the password).

8. Terminal commands
Where to run: PowerShell.

```
# What: creates a key valid ~27 years. Success: the .jks file exists.
mkdir C:\dev\keys
& "C:\Program Files\Android\Android Studio\jbr\bin\keytool.exe" -genkeypair -v -keystore C:\dev\keys\mapx-
  release.jks -alias mapx -keyalg RSA -keysize 4096 -validity 10000
```

9. Official download links
None required for this step.

10. Complete AI implementation prompt
Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Google AI Studio free tier (best for large, multi-file generation because of its long context) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 02.2.6 - Create and Back Up the Signing Key
TASK: Write the Gradle Kotlin DSL signingConfigs block that reads keystore.properties (untracked) and falls back
  to unsigned when the file is missing.
CONTEXT TO ATTACH: Attach current app/build.gradle.kts.
FILES TO CREATE OR MODIFY: app/build.gradle.kts, keystore.properties.example
RULES: Never print passwords. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Changed file sections.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-02.2.6. If the result fails verification: git restore . (or git reset --hard before-02.2.6), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result
Key exists in two places.

12. Verification procedure
Both copies have the same file size.

13. Acceptance criteria
- [ ] Key outside repo.
- [ ] Second copy exists.
- [ ] .gitignore blocks keystore.properties.

14. Common errors
- Lost password -> unrecoverable; create a new key (only possible before first team install).

15. Troubleshooting
1) Re-run the command exactly as printed. 2) Compare your output with the 'Expected result'. 3) Paste command and output to the AI with 'Interpret this; do not suggest reinstalling anything unless it is proven missing.' 4) Record UNVERIFIED if still unclear.

16. Dependencies

Depends on step 02.2.5; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STAGE 02.3 — Skeleton, Localization and Theme

STEP 02.3.1 — Create Package Structure and AppContainer

Provenance: [SPEC 7.2]

1. Step ID / 2. Title

02.3.1 — Create Package Structure and AppContainer (type: AI-assisted coding)

3. Objective

Create the package-by-feature structure from Spec 7.2 and the manual AppContainer.

4. Why it is necessary

All later code lands in predictable places; manual DI avoids an extra library.

5. Prerequisites

Step 02.2.6 done and verified (tracker ticked). Phase Gate: a clean debug build installs and starts on both devices; the six documents exist; architecture tests run.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Create packages under app.mapx: core.geo, core.model, core.command, core.coords, data, map, ui.layout, ui.theme, feature.{auth,editor,projects,mapsources,georef,drawing,layers,media,symbols,scenes,presentation,gis,exchange,display,companion,location,settings}.
2. Add placeholder package-info files with a one-line purpose each.
3. Create AppContainer (holds repositories and managers; created in the Application class).
4. Create MapXApplication; register it in the manifest.
5. Build; run on a device; commit.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 02.3.1 - Create Package Structure and AppContainer
TASK: Create the package skeleton and AppContainer for MapX per the structure.
CONTEXT TO ATTACH: Spec 7.2 package list and class names.
FILES TO CREATE OR MODIFY: app/src/main/java/app/mapx/**/package-info.kt, MapXApplication.kt, AppContainer.kt
RULES: No DI library. No logic yet. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native,
  Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-02.3.1. If the result fails verification: git restore . (or git reset --hard before-02.3.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Skeleton builds.

12. Verification procedure

BUILD SUCCESSFUL; app starts.

13. Acceptance criteria

- [ ] Packages match Spec 7.2.
- [ ] AppContainer exists.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 02.2.6; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 02.3.2 — Add Architecture Tests and the Local check.ps1 Script

Provenance: [SPEC 16.2]

1. Step ID / 2. Title

02.3.2 — Add Architecture Tests and the Local check.ps1 Script (type: AI-assisted coding)

3. Objective

Create tests that enforce the dependency rule and a one-command quality check.

4. Why it is necessary

Spec 16.2: the domain layer may not import Android UI or MapLibre; persistent models may not hold screen-space fields. Enforcing it by test stops AI drift.

5. Prerequisites

Step 02.3.1 done and verified (tracker ticked). Phase Gate: a clean debug build installs and starts on both devices; the six documents exist; architecture tests run.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Create JVM tests that scan source files: (1) files under core/** must not import android.*, androidx.compose.*, org.maplibre.*; (2) files under core/model must not declare fields named x, y, px, screen, offset; (3) a test that no file outside the map package imports org.maplibre; (4) a test that user-visible string literals do not appear inside Text(calls in feature/** and ui/** (scan for Text(" patterns).
2. Create tools/check.ps1: runs lint, unit tests and the architecture tests; prints PASS/FAIL.
3. Write the tests so they pass on an empty skeleton, then they protect all later code.

8. Terminal commands

Where to run: PowerShell in C:\dev\MapX.

```
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 02.3.2 - Add Architecture Tests and the Local check.ps1 Script
TASK: Write JUnit4 tests that scan Kotlin source files by path and enforce the listed rules, plus check.ps1.
CONTEXT TO ATTACH: Spec 16.2 architecture tests; package structure.
FILES TO CREATE OR MODIFY: app/src/test/java/app/mapx/ArchitectureTest.kt, tools/check.ps1
RULES: No new dependency; use java.io.File and project-relative paths. Native Android only: Kotlin, Jetpack
  Compose, Material 3, MapLibre Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels,
  manual DI via AppContainer. Do not add a dependency that is not already in the version catalog; if one is
  needed, stop and say so. Store geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command.
  No hard-coded user-visible strings (use string resources, English and Arabic). Touch targets at least 48 dp.
  Use start/end, never left/right. Write or update unit tests. Do not weaken or delete existing tests. Never
  claim something works unless it was built and tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-02.3.2. If the result fails verification: git restore . (or git reset --hard before-02.3.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result
Tests pass.

12. Verification procedure
check.ps1 prints PASS; deliberately add 'import android.util.Log' to a core file and see FAIL, then remove it.

13. Acceptance criteria
[ ] Four rules enforced.
[ ] check.ps1 works.

14. Common errors
- Tests cannot find the source folder -> use System.getProperty("user.dir") and walk up to the repo root.

15. Troubleshooting
1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies
Depends on step 02.3.1; leads to the next step in the phase order.

17. Estimated download / storage
No download.

STEP 02.3.3 — Localization Infrastructure (English/Arabic, No Hard-Coded Strings)
Provenance: [SPEC FR-SET-01]

1. Step ID / 2. Title
02.3.3 — Localization Infrastructure (English/Arabic, No Hard-Coded Strings) (type: AI-assisted coding)

3. Objective
Set up values/ and values-ar/, a language setting that applies without restart, and RTL support.

4. Why it is necessary
Arabic and English are first-class languages (Spec 5.15, FR-SET-01). Retrofitting later is expensive.

5. Prerequisites
Step 02.3.2 done and verified (tracker ticked). Phase Gate: a clean debug build installs and starts on both devices; the six documents exist; architecture tests run.

6. Required software
Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions
1. Enable RTL support (android:supportsRtl="true"). Create res/values/strings.xml and res/values-ar/strings.xml with the first 20 keys (app_name, new_project, open_project, recent_projects, settings, layers, cancel, delete...).
2. Implement an in-app language switch using the platform per-app language API (AppCompat-free approach: Configuration/Context wrapper at the root); persist later in DataStore (05.1.3).
3. Provide the Compose root with LayoutDirection once (Guide section 19).
4. Add pseudo-locale support in debug (en-XA, ar-XB) via build flag.
5. Draft Arabic strings with AI, then mark them 'needs native review' in a comment file (Guide section 15 requires native review).

8. Terminal commands
Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links
None required for this step.

10. Complete AI implementation prompt
Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 02.3.3 - Localization Infrastructure (English/Arabic, No Hard-Coded Strings)
TASK: Create the localization infrastructure and the first 20 English and Arabic strings.
CONTEXT TO ATTACH: Spec FR-SET-01, Guide section 15 (Arabic samples).
FILES TO CREATE OR MODIFY: res/values/strings.xml, res/values-ar/strings.xml, ui/layout/LocaleController.kt
RULES: Western digits by default. No hard-coded strings in code. Native Android only: Kotlin, Jetpack Compose,
  Material 3, MapLibre Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI
  via AppContainer. Do not add a dependency that is not already in the version catalog; if one is needed, stop
  and say so. Store geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-
  coded user-visible strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use
  start/end, never left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim
  something works unless it was built and tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-02.3.3. If the result fails verification: git restore . (or git reset --hard before-02.3.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Language can be switched.

12. Verification procedure

Switch to Arabic on the tablet: the layout direction flips.

13. Acceptance criteria

[ ] Both locales load.

[ ] LayoutDirection provided at root once.

14. Common errors

- Context wrapper not applied -> recreate the activity after changing language.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 02.3.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 02.3.4 — Theme, Design Tokens and the Cairo Font

Provenance: [SPEC 12.2] [REC]

1. Step ID / 2. Title

02.3.4 — Theme, Design Tokens and the Cairo Font (type: AI-assisted coding)

3. Objective

Implement the design tokens and bundle the Cairo font (OFL).

4. Why it is necessary

The visual identity must be consistent from the first screen (Spec 12.2, Guide sections 03-05).

5. Prerequisites

Step 02.3.3 done and verified (tracker ticked). Phase Gate: a clean debug build installs and starts on both devices; the six documents exist; architecture tests run.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Download Cairo from Google Fonts (variable font TTF) and place it in res/font/. Keep the OFL licence text in docs/licenses/Cairo-OFL.txt.
2. Create tokens: Navy900 #0A1A3A, NavyText #14264F, InteractionBlue #3B6FD6, AccessibleBlueFill #2457B8, AlertRed #E3201B (authoritative, register C-02), AccessibleAlertFill #C62828, TileSurface #5E574D at 80%, MarkerAmber #F6B800, light surface #F2F5F9, panel white, divider #C8D2E2.
3. Create dark and light themes; no cyan/teal; radii 2-4 dp; gaps 2-4 dp.
4. Add a unit test that checks contrast ratios of the documented text/background pairs are >= 4.5 (except the documented 'never use' pairs).
5. Create a theme preview screen reachable in debug builds to eyeball both themes.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

- Cairo (OFL): https://fonts.google.com/specimen/Cairo

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 02.3.4 - Theme, Design Tokens and the Cairo Font
TASK: Write Compose theme files with the given color tokens, Cairo typography roles from Guide section 05, and a
  contrast-ratio unit test.
CONTEXT TO ATTACH: Guide Part A consolidated tokens and contrast table; section 05 typography.
FILES TO CREATE OR MODIFY: ui/theme/Color.kt, Type.kt, Theme.kt, test ContrastTest.kt
RULES: Use #E3201B for alert red. White on #E3201B is 4.68:1 (passes narrowly for normal text). Never use
  #3B6FD6 or #E3201B as small text on navy. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre
  Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do
  not add a dependency that is not already in the version catalog; if one is needed, stop and say so. Store
  geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible
  strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never
  left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim something works
  unless it was built and tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-02.3.4. If the result fails verification: git restore . (or git reset --hard before-02.3.4), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Both themes render; contrast test passes.

12. Verification procedure

ContrastTest passes; screens on device look navy/blue with no teal.

13. Acceptance criteria

- [ ] Tokens match the Guide.
- [ ] Cairo renders Arabic and Latin.
- [ ] OFL file included.

14. Common errors

- Font fails to load -> file name must be lowercase with underscores.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 02.3.3; leads to the next step in the phase order.

17. Estimated download / storage

Cairo font: under 1 MB class (verify).

PHASE 03 — TECHNICAL RISK SPIKES

Disposable experiments, run early on a git branch, that prove or disprove the riskiest assumptions before the product is built on them: MapLibre with 16 KB pages, PMTiles, MBTiles, Arabic map labels, TLS WebSocket pairing, external display and GNSS hardware.

Estimated relative effort: M. Gate: every spike has a recorded result and decision (spike code is discarded; only decisions and tested snippets are kept).

STAGE 03.1 — Map Engine Feasibility

Work on branch spike/map. Never merge spike code into main; copy only the verified snippets.

STEP 03.1.1 — MapLibre Hello-Map and 16 KB Alignment Check

Provenance: [SPEC 14] [VERIFIED 16 KB support from 11.5.0]

1. Step ID / 2. Title

03.1.1 — MapLibre Hello-Map and 16 KB Alignment Check (type: AI-assisted coding)

3. Objective

Add MapLibre Native Android, show an empty map from a local style, and prove the native libraries are 16 KB compatible.

4. Why it is necessary

Spec 14 and master prompt: native-library 16 KB page-size compatibility must be checked at pin time. The map engine is the largest dependency.

5. Prerequisites

Step 02.3.4 done and verified (tracker ticked). Phase Gate: every spike has a recorded result and decision (spike code is discarded; only decisions and tested snippets are kept).

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. git checkout -b spike/map. Add org.maplibre.gl:android-sdk at the provisional version from D-R06.
2. Create a screen with a MapView (AndroidView) loading a tiny style.json from assets (background colour only).
3. Build, install on both devices, pan and zoom.
4. Check 16 KB alignment: build a release-like APK and run zipalign -c -P 16 -v 4 app-debug.apk (zipalign is in build-tools); also open the APK in Android Studio (Build > Analyze APK) and look for 16 KB alignment warnings on lib/arm64-v8a/*.so.
5. Record APK size increase and MapLibre AAR size in MAPX_DECISIONS.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
& "$env:LOCALAPPDATA\Android\Sdk\build-tools\<version>\zipalign.exe" -c -P 16 -v 4
  app\build\outputs\apk\debug\app-debug.apk
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 03.1.1 - MapLibre Hello-Map and 16 KB Alignment Check
TASK: Create a minimal Compose screen hosting a MapLibre MapView loading asset://style.json with a background
  layer, handling lifecycle (onStart/onStop/onDestroy).
CONTEXT TO ATTACH: Spec 5.2, 7.1; MapLibre docs for the pinned version.
FILES TO CREATE OR MODIFY: spike/MapSpikeScreen.kt, assets/style.json
RULES: Spike only; no architecture. Report exact Maven coordinate used. Native Android only: Kotlin, Jetpack
  Compose, Material 3, MapLibre Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels,
  manual DI via AppContainer. Do not add a dependency that is not already in the version catalog; if one is
  needed, stop and say so. Store geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command.
  No hard-coded user-visible strings (use string resources, English and Arabic). Touch targets at least 48 dp.
  Use start/end, never left/right. Write or update unit tests. Do not weaken or delete existing tests. Never
  claim something works unless it was built and tested.
OUTPUT: Complete files and gradle line.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-03.1.1. If the result fails verification: git restore . (or git reset --hard before-03.1.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Blank map renders; alignment check reports pass or fail.

12. Verification procedure
Map visible and pannable on both devices; zipalign output saved.

13. Acceptance criteria
[ ] 16 KB result recorded (pass/fail).
[ ] Exact MapLibre version recorded.
[ ] Both devices show the map.

14. Common errors
- API differs from older examples -> use the docs for your pinned version.
- zipalign -P unsupported -> update build-tools.

15. Troubleshooting
1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies
Depends on step 02.3.4; leads to the next step in the phase order.

17. Estimated download / storage
MapLibre AAR: tens of MB class (verify).

STEP 03.1.2 — PMTiles Local Source Spike
Provenance: [VERIFIED Spec 3.3]

1. Step ID / 2. Title
03.1.2 — PMTiles Local Source Spike (type: AI-assisted coding)

3. Objective
Display a real local PMTiles file using pmtiles://file:///... on both devices.

4. Why it is necessary
Spec 3.3 says PMTiles is supported from MapLibre Android 11.7.0 with the URL fully specified; this proves it on your pinned version and files.

5. Prerequisites
Step 03.1.1 done and verified (tracker ticked). Phase Gate: every spike has a recorded result and decision (spike code is discarded; only decisions and tested snippets are kept).

6. Required software
Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions
1. Obtain a small raster or vector PMTiles sample (for example a small regional extract; keep it under 50 MB; check the licence). Docs: Protomaps PMTiles.
2. Copy it to the app's filesDir with adb (adb push then run-as copy) or via a debug file picker.
3. Define a source with url pmtiles://file:///data/user/0/app.mapx/files/sample.pmtiles and a layer in the style.
4. Confirm tiles appear offline (airplane mode).
5. Record Evidence for the Format Support Matrix: file name, size, result.

8. Terminal commands
Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links
- PMTiles docs: https://docs.protomaps.com/pmtiles/
- Spec-cited MapLibre example: maplibre.org/maplibre-native/android/examples/data/PMTiles

10. Complete AI implementation prompt
Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 03.1.2 - PMTiles Local Source Spike
TASK: Add a PMTiles source to the spike style and show how to build the file URL from filesDir.
CONTEXT TO ATTACH: Spec 6.3 PMTiles row and 3.3.
FILES TO CREATE OR MODIFY: spike code
RULES: URL inside pmtiles:// must be fully specified. Native Android only: Kotlin, Jetpack Compose, Material 3,
  MapLibre Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via
  AppContainer. Do not add a dependency that is not already in the version catalog; if one is needed, stop and
  say so. Store geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded
  user-visible strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use start/end,
  never left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim something
  works unless it was built and tested.
OUTPUT: Changed files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-03.1.2. If the result fails verification: git restore . (or git reset --hard before-03.1.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

PMTiles tiles render.

12. Verification procedure

Visible map in airplane mode.

13. Acceptance criteria

[ ] PMTiles Supported/Partial decision recorded with sample.

14. Common errors

- Blank map -> wrong absolute path or sample is vector without style layers.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 03.1.1; leads to the next step in the phase order.

17. Estimated download / storage

Sample file size (you choose).

STEP 03.1.3 — MBTiles Direct-Read Spike and Loopback Fallback Decision

Provenance: [SPEC 3.3, 6.3]

1. Step ID / 2. Title

03.1.3 — MBTiles Direct-Read Spike and Loopback Fallback Decision (type: AI-assisted coding)

3. Objective

Test whether the pinned MapLibre can read a local MBTiles file; if not, prototype the loopback tile reader.

4. Why it is necessary

Official documentation for local MBTiles was not found (Spec 3.3), so MBTiles is 'expected, unconfirmed'. The choice changes Phase 09 design.

5. Prerequisites

Step 03.1.2 done and verified (tracker ticked). Phase Gate: every spike has a recorded result and decision (spike code is discarded; only decisions and tested snippets are kept).

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Get a small raster MBTiles sample (for example exported from MOBAC or QGIS).
2. Test a source using mbtiles:// with a local path; note whether tiles render.
3. If it fails: implement a ~100-line loopback server (ServerSocket on 127.0.0.1, random port, random path token) that reads tiles with SQLiteDatabase read-only and answers /{token}/{z}/{x}/{y}; remember MBTiles stores TMS y: y_tms = 2^z - 1 - y.
4. Add network_security_config allowing cleartext only for 127.0.0.1.
5. Decide: Direct or Loopback; record as D-R07 with evidence.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

- MOBAC (map exporter): https://mobac.sourceforge.io/
- QGIS: https://qgis.org/download/

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 03.1.3 - MBTiles Direct-Read Spike and Loopback Fallback Decision
TASK: Write a minimal loopback HTTP tile server for MBTiles in Kotlin using only platform classes, bound to
  127.0.0.1 with a random token path, handling the TMS row flip.
CONTEXT TO ATTACH: Spec 6.3 fallback paragraph; 10.4 cleartext rule.
FILES TO CREATE OR MODIFY: spike/LoopbackTileServer.kt, res/xml/network_security_config.xml
RULES: Never bind to a non-loopback address. No new dependency. Native Android only: Kotlin, Jetpack Compose,
  Material 3, MapLibre Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI
  via AppContainer. Do not add a dependency that is not already in the version catalog; if one is needed, stop
  and say so. Store geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-
  coded user-visible strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use
  start/end, never left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim
  something works unless it was built and tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-03.1.3. If the result fails verification: git restore . (or git reset --hard before-03.1.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

MBTiles rendered by one path.

12. Verification procedure

Raster MBTiles visible offline.

13. Acceptance criteria

[ ] Decision D-R07 recorded.
[ ] Y-flip verified (tiles not upside down).

14. Common errors

- Tiles upside-down -> missing TMS flip.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 03.1.2; leads to the next step in the phase order.

17. Estimated download / storage

Sample file.

STEP 03.1.4 — Arabic Label Rendering Spike

Provenance: [SPEC 7.5]

1. Step ID / 2. Title

03.1.4 — Arabic Label Rendering Spike (type: AI-assisted coding)

3. Objective

Prove Arabic labels render correctly through MapLibre text and through MapX's own bitmap path, and generate Arabic-capable glyph files.

4. Why it is necessary

Spec 7.5 mandates a reference Arabic label set test; MapLibre's shaping is basic (Spec 3.3). Glyph PBFs must contain the Arabic Unicode blocks.

5. Prerequisites

Step 03.1.3 done and verified (tracker ticked). Phase Gate: every spike has a recorded result and decision (spike code is discarded; only decisions and tested snippets are kept).

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Generate glyph PBF ranges from Cairo (Latin + U+0600-06FF, U+0750-077F, U+FB50-FDFF, U+FE70-FEFF). Options: a browser-based MapLibre font generator (confirm it exists), or node-fontnik under WSL if available. Record the tool used.

2. Bundle glyphs in assets/glyphs/Cairo Regular/ and reference glyphs: asset://glyphs/{fontstack}/{range}.pbf in the style (verify asset:// is accepted for glyphs; if not, use the loopback server).
3. Create a GeoJSON with the reference labels (connected forms, lam-alef, diacritics, mixed digits and Latin).
4. Also render the same strings with StaticLayout to a Bitmap (Path B) and show both on screen.
5. Photograph or screenshot results for the evidence file.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 03.1.4 - Arabic Label Rendering Spike
TASK: Create a style and GeoJSON that draw the Arabic reference labels via a symbol layer, and a StaticLayout
  bitmap renderer for the same strings.
CONTEXT TO ATTACH: Spec 7.5.
FILES TO CREATE OR MODIFY: spike code, assets/ref_labels.geojson
RULES: Spike only. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room, DataStore,
  kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a dependency that
  is not already in the version catalog; if one is needed, stop and say so. Store geographic data as WGS84
  lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use string
  resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or update
  unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-03.1.4. If the result fails verification: git restore . (or git reset --hard before-03.1.4), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Both paths shown side by side.

12. Verification procedure

Native Arabic reader judges joining and order.

13. Acceptance criteria

[ ] Path B correct.
[ ] Path C correct or fallback (pre-shaped basemap names) chosen and recorded.

14. Common errors

- Disconnected letters -> glyph file lacks presentation forms or text engine limitation; apply Spec fallback.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 03.1.3; leads to the next step in the phase order.

17. Estimated download / storage

Glyph files: a few MB.

STAGE 03.2 — Networking and Display Feasibility

Branch spike/net.

STEP 03.2.1 — WebSocket + TLS + AndroidKeyStore Certificate Spike

Provenance: [SPEC DN-3, 10.5]

1. Step ID / 2. Title

03.2.1 — WebSocket + TLS + AndroidKeyStore Certificate Spike (type: AI-assisted coding)

3. Objective

Prove a TLS WebSocket server on the tablet and client on the phone using Java-WebSocket, a Keystore-generated self-signed certificate and fingerprint pinning.

4. Why it is necessary

Decision DN-3 and Spec 10.5: confirms the one unavoidable dependency works on both devices before building the remote.

5. Prerequisites

Step 03.1.4 done and verified (tracker ticked). Phase Gate: every spike has a recorded result and decision (spike code is discarded; only decisions and tested snippets are kept).

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Add Java-WebSocket (pinned) on spike/net only.
2. Generate an RSA/EC key pair in AndroidKeyStore with KeyGenParameterSpec and certificate subject; read its self-signed certificate; compute SHA-256 fingerprint.
3. Build an SSLContext from the Keystore key and start WebSocketServer on a random port with DefaultSSLWebSocketServerFactory.
4. On the phone, connect with a custom X509TrustManager that accepts only the certificate with the expected fingerprint.
5. Both devices on the same Wi-Fi: send ping/echo; measure round-trip time.
6. If Keystore certificate or TLS fails, prototype the documented fallback: platform TLS sockets with line-delimited JSON; record decision D-R08.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C). Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 03.2.1 - WebSocket + TLS + AndroidKeyStore Certificate Spike
TASK: Write a spike: AndroidKeyStore self-signed key, Java-WebSocket TLS server on the tablet, client with
  fingerprint-pinned trust manager on the phone, echo test.
CONTEXT TO ATTACH: Spec 10.5.
FILES TO CREATE OR MODIFY: spike/TlsServer.kt, spike/PinnedClient.kt
RULES: No certificate library unless Keystore cannot produce the certificate; if so say so. Native Android only:
  Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow,
  ViewModels, manual DI via AppContainer. Do not add a dependency that is not already in the version catalog; if
  one is needed, stop and say so. Store geographic data as WGS84 lat/lon, never screen pixels. Every edit is a
  command. No hard-coded user-visible strings (use string resources, English and Arabic). Touch targets at least
  48 dp. Use start/end, never left/right. Write or update unit tests. Do not weaken or delete existing tests.
  Never claim something works unless it was built and tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-03.2.1. If the result fails verification: git restore . (or git reset --hard before-03.2.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Echo works over wss with pinning.

12. Verification procedure

Wrong fingerprint is rejected; right one passes.

13. Acceptance criteria

- [ ] Library decision recorded (Java-WebSocket or fallback).
- [ ] Certificate library need decided.

14. Common errors

- SSL handshake error -> key algorithm/signature not supported by the TLS stack; try EC or RSA with SHA256.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 03.1.4; leads to the next step in the phase order.

17. Estimated download / storage

Java-WebSocket jar: small (verify).

STEP 03.2.2 — External Display Capability Probe on the Lenovo Tab P12

Provenance: [SPEC 5.13, 17.2] [VERIFIED research on specs: unconfirmed]

1. Step ID / 2. Title

03.2.2 — External Display Capability Probe on the Lenovo Tab P12 (type: device test)

3. Objective

Find out which external display routes the tablet truly offers.

4. Why it is necessary

Research found USB-C 2.0 and no DisplayPort listing; a wired display may be impossible. The Spec requires honest reporting and an Audience Preview fallback.

5. Prerequisites

Step 03.2.1 done and verified (tracker ticked). Phase Gate: every spike has a recorded result and decision (spike code is discarded; only decisions and tested snippets are kept).

6. Required software

Android Studio, adb, the Lenovo Tab P12 and/or Samsung Galaxy A16 (USB debugging on), Git.

7. Exact instructions

1. Run adb shell dumpsys display and note displays present.
2. Check Settings for 'Cast', 'Wireless display' or 'Smart View'-type options and test with any available TV or Miracast receiver.
3. Use Developer options > Simulate secondary displays to create an overlay display; write a tiny test that lists displays via DisplayManager.getDisplays(DisplayManager.DISPLAY_CATEGORY_PRESENTATION) and opens an android.app.Presentation on it.
4. Classify result: Hardware verified / Simulated only / Not available. Never record simulated as hardware-verified.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat installDebug --offline
# Then run the app on the device named in the verification procedure and note the result.
```

9. Official download links

- Presentation: https://developer.android.com/reference/android/app/Presentation
- DisplayManager: https://developer.android.com/reference/android/hardware/display/DisplayManager

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 03.2.2 - External Display Capability Probe on the Lenovo Tab P12
TASK: Write a spike Activity that lists displays and shows a Presentation with a coloured screen on the first
  presentation display, with a log of display flags.
CONTEXT TO ATTACH: Spec 5.13.
FILES TO CREATE OR MODIFY: spike/DisplayProbe.kt
RULES: Spike only. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room, DataStore,
  kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a dependency that
  is not already in the version catalog; if one is needed, stop and say so. Store geographic data as WGS84
  lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use string
  resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or update
  unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete file.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-03.2.2. If the result fails verification: git restore . (or git reset --hard before-03.2.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Capability class recorded.

12. Verification procedure

Presentation window appears on a simulated display.

13. Acceptance criteria

[ ] Display capability recorded as simulated or real.

14. Common errors

- No presentation display found -> record 'none'; MapX will use Audience Preview.

15. Troubleshooting

1) Run: adb devices -l and confirm 'device'. 2) Uninstall and reinstall: adb uninstall app.mapx then installDebug. 3) Capture the crash: adb logcat -d | Select-String 'FATAL|AndroidRuntime' and give the lines to the AI. 4) Retest on the other device to separate device-specific from general faults.

16. Dependencies

Depends on step 03.2.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 03.2.3 — GNSS Hardware Probe (Lenovo and Samsung)

Provenance: [SPEC DN-9]

1. Step ID / 2. Title

03.2.3 — GNSS Hardware Probe (Lenovo and Samsung) (type: device test)

3. Objective

Decide which device performs GPS tests.

4. Why it is necessary

DN-9: some Wi-Fi tablets lack GNSS.

5. Prerequisites

Step 03.2.2 done and verified (tracker ticked). Phase Gate: every spike has a recorded result and decision (spike code is discarded; only decisions and tested snippets are kept).

6. Required software

Android Studio, adb, the Lenovo Tab P12 and/or Samsung Galaxy A16 (USB debugging on), Git.

7. Exact instructions

1. From 01.1.9 you know whether gps appears in the feature list.
2. Write a spike using LocationManager (GPS_PROVIDER) printing provider availability and, outdoors with a sky view, a fix with accuracy.
3. If the Lenovo has no fix or provider, designate the Samsung Galaxy A16 as the GNSS test device and record DN-9.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat installDebug --offline
# Then run the app on the device named in the verification procedure and note the result.
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 03.2.3 - GNSS Hardware Probe (Lenovo and Samsung)
TASK: Write a spike showing LocationManager providers and a single GPS fix with accuracy, with runtime
  permission handling.
CONTEXT TO ATTACH: Spec 5.3.
FILES TO CREATE OR MODIFY: spike/GnssProbe.kt
RULES: Foreground only; no Play Services. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre
  Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do
  not add a dependency that is not already in the version catalog; if one is needed, stop and say so. Store
  geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible
  strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never
  left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim something works
  unless it was built and tested.
OUTPUT: Complete file.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-03.2.3. If the result fails verification: git restore . (or git reset --hard before-03.2.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

DN-9 closed.

12. Verification procedure

Fix received outdoors on the designated device.

13. Acceptance criteria

[ ] GPS test device named in decisions.

14. Common errors

- No fix indoors -> test near a window or outdoors.

15. Troubleshooting

1) Run: adb devices -l and confirm 'device'. 2) Uninstall and reinstall: adb uninstall app.mapx then installDebug. 3) Capture the crash: adb logcat -d | Select-String 'FATAL|AndroidRuntime' and give the lines to the AI. 4) Retest on the other device to separate device-specific from general faults.

16. Dependencies

Depends on step 03.2.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STAGE 03.3 — Spike Decision Gate

STEP 03.3.1 — Record Spike Results and Update Decisions

Provenance: [SPEC 6.3]

1. Step ID / 2. Title

03.3.1 — Record Spike Results and Update Decisions (type: document)

3. Objective

Write all spike evidence into MAPX_DECISIONS.md and ARCHITECTURE.md, merge snippets and discard spike branches.

4. Why it is necessary

Evidence-based decisions prevent rework; the Spec forbids marking support without samples.

5. Prerequisites

Step 03.2.3 done and verified (tracker ticked). Phase Gate: every spike has a recorded result and decision (spike code is discarded; only decisions and tested snippets are kept).

6. Required software

VS Code or any text editor, Git, one AI tool from the AI field.

7. Exact instructions

1. For each spike write: result, evidence, decision ID (D-R06..D-R10), consequences.
2. Update ARCHITECTURE.md Format Support Matrix 'Evidence' cells for MBTiles and PMTiles with the actual sample used.
3. Delete branches: git branch -D spike/map spike/net (after copying notes).
4. Update MAPX_HANDOFF.md and the tracker.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
# No command required. Commit when finished:
git add -A
git commit -m "docs: step done"
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 03.3.1 - Record Spike Results and Update Decisions
TASK: Turn my spike notes into decision-log entries and matrix updates.
CONTEXT TO ATTACH: Paste notes.
FILES TO CREATE OR MODIFY: Only the files needed for this step; list them before writing code.
RULES: Do not claim anything beyond the evidence. Native Android only: Kotlin, Jetpack Compose, Material 3,
  MapLibre Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via
  AppContainer. Do not add a dependency that is not already in the version catalog; if one is needed, stop and
  say so. Store geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded
  user-visible strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use start/end,
  never left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim something
  works unless it was built and tested.
OUTPUT: Markdown rows.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-03.3.1. If the result fails verification: git restore . (or git reset --hard before-03.3.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Decisions recorded.

12. Verification procedure

Each spike has a decision row.

13. Acceptance criteria

[ ] Spike branches deleted.
[ ] Matrix updated.

14. Common errors

- AI invents facts -> require it to quote the specification section it relied on.

15. Troubleshooting

1) Compare the document with the listed sources. 2) Ask the AI to list contradictions between the document and the specification. 3) Fix and commit.

16. Dependencies

Depends on step 03.2.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

PHASE 04 — CORE DOMAIN LOGIC

Pure Kotlin logic with no Android or MapLibre imports: geography, coordinates, objects, commands, serialization. Everything is unit-tested on the PC with no device.
Estimated relative effort: L. Gate: all JVM tests in Spec 16.2 for this layer pass; architecture tests pass.

STAGE 04.1 — Geography and Coordinates

STEP 04.1.1 — GeoPoint and Geodesy Utilities

Provenance: [SPEC FR-CRD-01, FR-DRW-03]

1. Step ID / 2. Title

04.1.1 — GeoPoint and Geodesy Utilities (type: AI-assisted coding)

3. Objective

Implement GeoPoint (validated), distance, bearing, destination point, geodesic circle polygon and polygon area/perimeter in metres.

4. Why it is necessary

Geographic truth (Spec 7.3) needs one tested geodesy core.

5. Prerequisites

Step 03.3.1 done and verified (tracker ticked). Phase Gate: all JVM tests in Spec 16.2 for this layer pass; architecture tests pass.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Implement GeoPoint(lat, lon) validating lat in [-90,90], lon in [-180,180].
2. Haversine or Vincenty-style distance; initial bearing; destination from point+bearing+distance.
3. Geodesic circle: N-vertex ring (default 72) at radius metres.
4. Polygon area (spherical excess or ellipsoidal approximation) in m2 and perimeter.
5. Unit tests with known city-pair distances (compute expected values independently from a published calculator and note the source).

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 04.1.1 - GeoPoint and Geodesy Utilities
TASK: Write GeoPoint and Geodesy.kt with the listed functions plus JUnit tests.
CONTEXT TO ATTACH: Spec 7.3, FR-CRD-01, FR-DRW-03.
FILES TO CREATE OR MODIFY: core/geo/GeoPoint.kt, Geodesy.kt, test GeodesyTest.kt
RULES: Pure Kotlin; no Android imports; doubles; metres. Native Android only: Kotlin, Jetpack Compose, Material
  3, MapLibre Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via
  AppContainer. Do not add a dependency that is not already in the version catalog; if one is needed, stop and
  say so. Store geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded
  user-visible strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use start/end,
  never left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim something
  works unless it was built and tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-04.1.1. If the result fails verification: git restore . (or git reset --hard before-04.1.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Tests pass.

12. Verification procedure

gradlew testDebugUnitTest passes.

13. Acceptance criteria

[ ] lat 91 rejected.
[ ] Circle radius error under 0.5% for 1-100 km.

14. Common errors

- Antimeridian: circle near +-180 wraps -> handled in 08.2.3.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 03.3.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 04.1.2 — DD/DDM/DMS Formatting and Parsing

Provenance: [SPEC 5.4]

1. Step ID / 2. Title

04.1.2 — DD/DDM/DMS Formatting and Parsing (type: AI-assisted coding)

3. Objective

Implement CoordinateUtils for DD, DDM, DMS output and tolerant parsing (separators, signs, N/S/E/W, Arabic-Indic digits).

4. Why it is necessary

FR-CRD-02, 05, 06.

5. Prerequisites

Step 04.1.1 done and verified (tracker ticked). Phase Gate: all JVM tests in Spec 16.2 for this layer pass; architecture tests pass.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Formatters with configurable precision; hemisphere letters always Latin.
2. Parser accepts '35.1234 N 47.8821 E', '35°7.404′N', '35 7 24.2 N', '٣٥٫١٢٣٤', comma/space/semicolon separators.
3. Return a Result with a clear error message key (for localisation) when invalid.
4. Tests for round trips and invalid input.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 04.1.2 - DD/DDM/DMS Formatting and Parsing
TASK: Write CoordinateUtils for DD/DDM/DMS formatting and parsing with tests.
CONTEXT TO ATTACH: FR-CRD-02..06.
FILES TO CREATE OR MODIFY: core/coords/CoordinateUtils.kt, CoordinateParser.kt, tests
RULES: Pure Kotlin; error keys not literal UI text. Native Android only: Kotlin, Jetpack Compose, Material 3,
  MapLibre Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via
  AppContainer. Do not add a dependency that is not already in the version catalog; if one is needed, stop and
  say so. Store geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded
  user-visible strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use start/end,
  never left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim something
  works unless it was built and tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-04.1.2. If the result fails verification: git restore . (or git reset --hard before-04.1.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Tests pass.

12. Verification procedure
All parse cases pass.

13. Acceptance criteria
[ ] Round trip within 1e-6 deg.
[ ] Arabic-Indic digits parsed.

14. Common errors
- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting
1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies
Depends on step 04.1.1; leads to the next step in the phase order.

17. Estimated download / storage
No download.

STEP 04.1.3 — UTM Conversion (80S-84N, Zone Exceptions)
Provenance: [SPEC FR-CRD-03, 07]

1. Step ID / 2. Title
04.1.3 — UTM Conversion (80S-84N, Zone Exceptions) (type: AI-assisted coding)

3. Objective
Implement WGS84 lat/lon to UTM and back, including Norway and Svalbard zone exceptions.

4. Why it is necessary
FR-CRD-03 requires correctness verified against published vectors.

5. Prerequisites
Step 04.1.2 done and verified (tracker ticked). Phase Gate: all JVM tests in Spec 16.2 for this layer pass; architecture tests pass.

6. Required software
Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions
1. Implement Krueger series or Karney-style formulas on WGS84.
2. Handle zone 32V and 31X-37X exceptions; latitude bands C-X; false easting/northing; southern hemisphere.
3. Outside 80S-84N return NotSupported (UPS deferred).
4. Obtain published reference vectors (for example from an official geodetic reference or a trusted calculator) and put them in the test as constants with source notes. Do not let the AI invent expected values.

8. Terminal commands
Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links
None required for this step.

10. Complete AI implementation prompt
Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 04.1.3 - UTM Conversion (80S-84N, Zone Exceptions)
TASK: Write UTM forward and inverse conversions with exceptions and tests that use the reference vectors I
  provide.
CONTEXT TO ATTACH: Paste your reference vectors.
FILES TO CREATE OR MODIFY: core/coords/Utm.kt, UtmTest.kt
RULES: Fail rather than guess for polar latitudes. Native Android only: Kotlin, Jetpack Compose, Material 3,
  MapLibre Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via
  AppContainer. Do not add a dependency that is not already in the version catalog; if one is needed, stop and
  say so. Store geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded
  user-visible strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use start/end,
  never left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim something
  works unless it was built and tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-04.1.3. If the result fails verification: git restore . (or git reset --hard before-04.1.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

UTM matches vectors within 1 mm to 1 m (decide tolerance and write it).

12. Verification procedure

Tests pass with your independent vectors.

13. Acceptance criteria

- [ ] Round trip < 1 mm at test points.
- [ ] Zone exceptions tested.

14. Common errors

- AI supplies fabricated expected values -> reject; use independent sources.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 04.1.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 04.1.4 — MGRS Encode/Decode with Precision

Provenance: [SPEC FR-CRD-04]

1. Step ID / 2. Title

04.1.4 — MGRS Encode/Decode with Precision (type: AI-assisted coding)

3. Objective

Implement MGRS from UTM with precision 10 km to 1 m (default 1 m) and parsing.

4. Why it is necessary

FR-CRD-04.

5. Prerequisites

Step 04.1.3 done and verified (tracker ticked). Phase Gate: all JVM tests in Spec 16.2 for this layer pass; architecture tests pass.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. 100 km square letters by zone set; row letter offset for even zones.
2. Precision selector 10 km..1 m; truncation, not rounding.
3. Parser tolerant of spaces and lowercase.
4. Tests with independent reference vectors including zone boundaries and 80S/84N limits.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 04.1.4 - MGRS Encode/Decode with Precision
TASK: Write MGRS encode/decode on top of Utm.kt with tests.
CONTEXT TO ATTACH: Paste reference vectors.
FILES TO CREATE OR MODIFY: core/coords/Mgrs.kt, MgrsTest.kt
RULES: Reject polar bands (A,B,Y,Z). Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native,
  Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-04.1.4. If the result fails verification: git restore . (or git reset --hard before-04.1.4), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Round trips pass.

12. Verification procedure

Tests pass.

13. Acceptance criteria

[ ] Every precision level tested.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 04.1.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 04.1.5 — Scale Bar and Metric Formatting

Provenance: [SPEC FR-MAP-04]

1. Step ID / 2. Title

04.1.5 — Scale Bar and Metric Formatting (type: AI-assisted coding)

3. Objective

Compute scale bar length from latitude and zoom, and format metric distances/areas.

4. Why it is necessary

FR-MAP-04, FR-CRD-08.

5. Prerequisites

Step 04.1.4 done and verified (tracker ticked). Phase Gate: all JVM tests in Spec 16.2 for this layer pass; architecture tests pass.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. metres per pixel = 156543.03392 * cos(lat) / 2^zoom for 512-px tiles adjust per MapLibre tile size (verify in the spike).
2. Round to 1-2-5 steps; format m, km, m2, ha, km2.
3. Tests at latitudes 0, 45, 60, 80.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Google AI Studio free tier (best for large, multi-file generation because of its long context) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 04.1.5 - Scale Bar and Metric Formatting
TASK: Write ScaleBar.kt and MetricFormat.kt with tests.
CONTEXT TO ATTACH: FR-CRD-08.
FILES TO CREATE OR MODIFY: core/geo/ScaleBar.kt, MetricFormat.kt
RULES: Metric only; numbers as Western digits. Native Android only: Kotlin, Jetpack Compose, Material 3,
  MapLibre Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via
  AppContainer. Do not add a dependency that is not already in the version catalog; if one is needed, stop and
  say so. Store geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded
  user-visible strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use start/end,
  never left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim something
  works unless it was built and tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-04.1.5. If the result fails verification: git restore . (or git reset --hard before-04.1.5), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Tests pass.

12. Verification procedure

Tests pass.

13. Acceptance criteria

[ ] Scale tested at 4 latitudes.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 04.1.4; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STAGE 04.2 — Model, Commands and Serialization

STEP 04.2.1 — ProjectModels and MapObject Types

Provenance: [SPEC 5.5, 7.3]

1. Step ID / 2. Title

04.2.1 — ProjectModels and MapObject Types (type: AI-assisted coding)

3. Objective

Define ProjectModels, layers, scenes data, MapObject sealed hierarchy and CoordinateSpace.

4. Why it is necessary

FR-OBJ-01, FR-PRJ-04, 7.3 geometry table.

5. Prerequisites

Step 04.1.5 done and verified (tracker ticked). Phase Gate: all JVM tests in Spec 16.2 for this layer pass; architecture tests pass.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Types: Marker, Label, Symbol, Image, Drawing (11 kinds), ImportedFeature.
2. Common: id (UUID string), type, layerId, visible, locked, style, coordinateSpace.
3. Geometry per Spec 7.3 table; metres for radius/width.
4. Project document: formatVersion, id, name, description, createdAt, modifiedAt, camera, mapSourceRef(id+fingerprint), layers, objects, scenes, graphics, mediaRefs.
5. No screen-space fields (architecture test).

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 04.2.1 - ProjectModels and MapObject Types
TASK: Write the model classes with @Serializable annotations.
CONTEXT TO ATTACH: Spec 5.1, 5.5, 7.3.
FILES TO CREATE OR MODIFY: core/model/ProjectModels.kt, MapObject.kt, Geometry.kt
RULES: Class names exactly as Spec 7.2. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre
  Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do
  not add a dependency that is not already in the version catalog; if one is needed, stop and say so. Store
  geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible
  strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never
  left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim something works
  unless it was built and tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-04.2.1. If the result fails verification: git restore . (or git reset --hard before-04.2.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Model compiles.

12. Verification procedure

Build + architecture tests pass.

13. Acceptance criteria

[ ] Every object type from Spec 5.5 present.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 04.1.5; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 04.2.2 — MapObjectRegistry and Definitions

Provenance: [SPEC FR-OBJ-02]

1. Step ID / 2. Title

04.2.2 — MapObjectRegistry and Definitions (type: AI-assisted coding)

3. Objective

Create the registry with one definition per type (serialize, deserialize, hit-test hook, renderer hook, editor hook).

4. Why it is necessary

FR-OBJ-02: nothing hard-codes types elsewhere; unknown types preserved.

5. Prerequisites

Step 04.2.1 done and verified (tracker ticked). Phase Gate: all JVM tests in Spec 16.2 for this layer pass; architecture tests pass.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. MapObjectDefinition interface; registry lookup by type string.
2. Unknown type: keep raw JSON element and round-trip unchanged.
3. Architecture test: every registered type has serializer, renderer and editor entries (renderer/editor entries are registered later; test initially lists expected pending types).

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 04.2.2 - MapObjectRegistry and Definitions
TASK: Write MapObjectRegistry with unknown-type preservation and tests.
CONTEXT TO ATTACH: FR-OBJ-02.
FILES TO CREATE OR MODIFY: core/model/MapObjectRegistry.kt, MapObjectDefinition.kt
RULES: Domain only. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room, DataStore,
  kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a dependency that
  is not already in the version catalog; if one is needed, stop and say so. Store geographic data as WGS84
  lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use string
  resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or update
  unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-04.2.2. If the result fails verification: git restore . (or git reset --hard before-04.2.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Round trip of unknown type works.

12. Verification procedure

Test passes.

13. Acceptance criteria

[ ] Unknown type survives save/load.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 04.2.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 04.2.3 — CommandManager (begin/update/commit, 200 steps)

Provenance: [SPEC FR-OBJ-06]

1. Step ID / 2. Title

04.2.3 — CommandManager (begin/update/commit, 200 steps) (type: AI-assisted coding)

3. Objective

Implement MapCommand, CommandManager with undo/redo, gesture sessions and a 200-step history.

4. Why it is necessary

FR-OBJ-06, FR-PRJ-14: one gesture = one undo step.

5. Prerequisites

Step 04.2.2 done and verified (tracker ticked). Phase Gate: all JVM tests in Spec 16.2 for this layer pass; architecture tests pass.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Commands: Create, Delete, Move, Update, layer, scene, conversion and import commands (stubs for later).
2. begin/update/commit coalesces a gesture into one command.
3. History limit 200 (oldest dropped); redo cleared on new command.
4. Commands operate on a ProjectState reducer; UI never mutates state.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 04.2.3 - CommandManager (begin/update/commit, 200 steps)
TASK: Write CommandManager and tests for apply/revert, coalescing, limit and redo clearing.
CONTEXT TO ATTACH: FR-OBJ-06, 16.2.
FILES TO CREATE OR MODIFY: core/command/*.kt, tests
RULES: Pure Kotlin. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room, DataStore,
  kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a dependency that
  is not already in the version catalog; if one is needed, stop and say so. Store geographic data as WGS84
  lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use string
  resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or update
  unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-04.2.3. If the result fails verification: git restore . (or git reset --hard before-04.2.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Tests pass.

12. Verification procedure

Tests pass.

13. Acceptance criteria

- [ ] History cap 200 tested.
- [ ] Gesture coalescing tested.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 04.2.2; leads to the next step in the phase order.

17. Estimated download / storage
No download.

STEP 04.2.4 — Geometry Utilities (Douglas-Peucker, Curves, Rectangle)

Provenance: [SPEC FR-DRW-05]

1. Step ID / 2. Title
04.2.4 — Geometry Utilities (Douglas-Peucker, Curves, Rectangle) (type: AI-assisted coding)

3. Objective
Implement simplification with vertex cap, quadratic curve sampling, rectangle from centre/size/bearing, hit-distance helpers.

4. Why it is necessary
FR-DRW-05.

5. Prerequisites
Step 04.2.3 done and verified (tracker ticked). Phase Gate: all JVM tests in Spec 16.2 for this layer pass; architecture tests pass.

6. Required software
Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions
1. Douglas-Peucker with tolerance parameter in metres at a zoom (the UI passes ~2 dp converted)
2. Hard vertex cap (for example 500, documented).
3. Quadratic sampling for curved arrows; rectangle corners via destination points.
4. Tests.

8. Terminal commands
Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links
None required for this step.

10. Complete AI implementation prompt
Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Google AI Studio free tier (best for large, multi-file generation because of its long context) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 04.2.4 - Geometry Utilities (Douglas-Peucker, Curves, Rectangle)
TASK: Write GeometryOps.kt with those functions and tests.
CONTEXT TO ATTACH: Spec 7.3.
FILES TO CREATE OR MODIFY: core/geo/GeometryOps.kt
RULES: Pure Kotlin. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room, DataStore,
  kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a dependency that
  is not already in the version catalog; if one is needed, stop and say so. Store geographic data as WGS84
  lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use string
  resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or update
  unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-04.2.4. If the result fails verification: git restore . (or git reset --hard before-04.2.4), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result
Tests pass.

12. Verification procedure
Tests pass.

13. Acceptance criteria
[ ] Vertex cap enforced.

14. Common errors
- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 04.2.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 04.2.5 — ProjectSerializer, formatVersion and Migration Framework

Provenance: [SPEC FR-PRJ-10]

1. Step ID / 2. Title

04.2.5 — ProjectSerializer, formatVersion and Migration Framework (type: AI-assisted coding)

3. Objective

JSON serialization with explicit formatVersion, migrations keyed by version and golden fixtures.

4. Why it is necessary

FR-PRJ-10: existing projects must never break.

5. Prerequisites

Step 04.2.4 done and verified (tracker ticked). Phase Gate: all JVM tests in Spec 16.2 for this layer pass; architecture tests pass.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. ProjectSerializer using kotlinx.serialization; canonical JSON (sorted keys) for the round-trip guarantee FR-EXP-06.
2. ProjectMigration chain; version 1 is the first.
3. Create fixtures/v1/*.json golden files; one test per released version (rule continues forever).
4. Reject newer formatVersion with a distinct error.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 04.2.5 - ProjectSerializer, formatVersion and Migration Framework
TASK: Write ProjectSerializer, canonical JSON helper and ProjectMigration with golden-fixture tests.
CONTEXT TO ATTACH: FR-PRJ-10, FR-EXP-06.
FILES TO CREATE OR MODIFY: core/model/ProjectSerializer.kt, ProjectMigration.kt, test fixtures
RULES: Do not change a fixture after release. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre
  Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do
  not add a dependency that is not already in the version catalog; if one is needed, stop and say so. Store
  geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible
  strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never
  left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim something works
  unless it was built and tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-04.2.5. If the result fails verification: git restore . (or git reset --hard before-04.2.5), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Round trip equal.

12. Verification procedure

Tests pass.

13. Acceptance criteria

[ ] Canonical JSON equal after round trip for all object types.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 04.2.4; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 04.2.6 — Layer, Scene and RenderOrder Domain

Provenance: [SPEC 5.7, 5.11]

1. Step ID / 2. Title

04.2.6 — Layer, Scene and RenderOrder Domain (type: AI-assisted coding)

3. Objective

Layer rules, scene data and the RenderOrder service.

4. Why it is necessary

FR-LYR-01..04, FR-SCN-01.

5. Prerequisites

Step 04.2.5 done and verified (tracker ticked). Phase Gate: all JVM tests in Spec 16.2 for this layer pass; architecture tests pass.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Layer kind USER/IMPORTED; default layer undeletable; semantic rules (hidden, locked, readOnly, opacity multiplies).
2. Scene = view state referencing data (camera, visible layers, overrides, graphics state).
3. RenderOrder: layer order then object order; documented mapping to style layers.
4. Tests for layer commands.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 04.2.6 - Layer, Scene and RenderOrder Domain
TASK: Write layer/scene domain and RenderOrder with tests.
CONTEXT TO ATTACH: Spec 5.7, 5.11.
FILES TO CREATE OR MODIFY: core/model/Layer.kt, Scene.kt, core/command/RenderOrder.kt
RULES: Domain only. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room, DataStore,
  kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a dependency that
  is not already in the version catalog; if one is needed, stop and say so. Store geographic data as WGS84
  lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use string
  resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or update
  unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-04.2.6. If the result fails verification: git restore . (or git reset --hard before-04.2.6), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Tests pass.

12. Verification procedure
Tests pass.

13. Acceptance criteria
[ ] Default layer cannot be deleted.

14. Common errors
- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting
1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies
Depends on step 04.2.5; leads to the next step in the phase order.

17. Estimated download / storage
No download.

STEP 04.2.7 — Spatial Index and Hit-Test Core
Provenance: [SPEC 7.4]

1. Step ID / 2. Title
04.2.7 — Spatial Index and Hit-Test Core (type: AI-assisted coding)

3. Objective
Implement an in-memory spatial index and screen-space hit-testing via a projection interface.

4. Why it is necessary
FR-OBJ-10, Spec 7.4 item 4.

5. Prerequisites
Step 04.2.6 done and verified (tracker ticked). Phase Gate: all JVM tests in Spec 16.2 for this layer pass; architecture tests pass.

6. Required software
Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions
1. Grid or R-tree on bounding boxes (own code, no library).
2. Hit-test: distance in screen space using a Projection interface (toScreen) so tests use a fake projection.
3. Hidden layers excluded.
4. Test with 5,000 objects: lookup under a few ms.

8. Terminal commands
Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links
None required for this step.

10. Complete AI implementation prompt
Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 04.2.7 - Spatial Index and Hit-Test Core
TASK: Write SpatialIndex.kt and HitTester.kt with a fake-projection test.
CONTEXT TO ATTACH: Spec 7.4.
FILES TO CREATE OR MODIFY: core/geo/SpatialIndex.kt, HitTester.kt
RULES: No libraries. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room, DataStore,
  kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a dependency that
  is not already in the version catalog; if one is needed, stop and say so. Store geographic data as WGS84
  lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use string
  resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or update
  unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-04.2.7. If the result fails verification: git restore . (or git reset --hard before-04.2.7), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Tests pass.

12. Verification procedure

Timing test passes.

13. Acceptance criteria

[ ] 5,000-object query fast on PC.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 04.2.6; leads to the next step in the phase order.

17. Estimated download / storage

No download.

PHASE 05 — DATA LAYER AND PROJECT PERSISTENCE

Room, DataStore, file layout, autosave, atomic save, last-known-good, duplicate and migration tests.
Estimated relative effort: L. Gate: Room tests and persistence tests pass; kill-process restore proven later in 10.3.4.

STAGE 05.1 — Storage Foundation

STEP 05.1.1 — Room Database and Entities

Provenance: [SPEC 8]

1. Step ID / 2. Title

05.1.1 — Room Database and Entities (type: AI-assisted coding)

3. Objective

Create MapXDatabase, ProjectEntity, ProjectDao, accounts and map-source tables.

4. Why it is necessary

Spec 8: Room for structured data only.

5. Prerequisites

Step 04.2.7 done and verified (tracker ticked). Phase Gate: Room tests and persistence tests pass; kill-process restore proven later in 10.3.4.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Entities: accounts, projects (library row + document JSON), map sources, last-known-good revision.
2. Document stored as JSON text; no large binaries.
3. Export schema JSON to the repo for migrations.
4. In-memory Room tests.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 05.1.1 - Room Database and Entities
TASK: Write Room entities, DAOs and database with exported schemas.
CONTEXT TO ATTACH: Spec 8, FR-PRJ-04/12.
FILES TO CREATE OR MODIFY: data/db/*.kt
RULES: No binaries in Room. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-05.1.1. If the result fails verification: git restore . (or git reset --hard before-05.1.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

DB compiles.

12. Verification procedure

Instrumented or Robolectric-free in-memory tests pass on device.

13. Acceptance criteria

[ ] Schema exported.

14. Common errors

- KSP errors -> versions mismatch with Kotlin.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 04.2.7; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 05.1.2 — File Storage Layout and Account-Scoped Paths

Provenance: [SPEC 8]

1. Step ID / 2. Title

05.1.2 — File Storage Layout and Account-Scoped Paths (type: AI-assisted coding)

3. Objective

Implement the storage layout from Spec 8.

4. Why it is necessary

Per-account isolation and safe cleanup.

5. Prerequisites

Step 05.1.1 done and verified (tracker ticked). Phase Gate: Room tests and persistence tests pass; kill-process restore proven later in 10.3.4.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. files/accounts/<accountId>/media/<projectId>/<mediaId>; files/accounts/<accountId>/mapsources/<sourceId>/; cache/exports/.
2. Path builder class rejects '..' and absolute input.
3. Free-space helper (StatFs) for later import checks.
4. Cache cleaned on start.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 05.1.2 - File Storage Layout and Account-Scoped Paths
TASK: Write AppPaths.kt and FreeSpace.kt with tests of path safety.
CONTEXT TO ATTACH: Spec 8.
FILES TO CREATE OR MODIFY: data/files/AppPaths.kt
RULES: App-private storage only. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native,
  Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-05.1.2. If the result fails verification: git restore . (or git reset --hard before-05.1.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Paths created.

12. Verification procedure

Unit tests reject traversal.

13. Acceptance criteria

[ ] Traversal attempts rejected.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 05.1.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 05.1.3 — PreferencesManager (DataStore)

Provenance: [SPEC 5.15]

1. Step ID / 2. Title

05.1.3 — PreferencesManager (DataStore) (type: AI-assisted coding)

3. Objective

Create PreferencesManager for language, coordinate format (per account), theme, motion, auto-lock, favourites and recents.

4. Why it is necessary

Spec 8: settings in DataStore.

5. Prerequisites

Step 05.1.2 done and verified (tracker ticked). Phase Gate: Room tests and persistence tests pass; kill-process restore proven later in 10.3.4.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Typed keys; Flow API.
2. Coordinate format saved per account.
3. Defaults: Smooth motion, dark theme, MGRS precision 1 m, auto-lock 5 min.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 05.1.3 - PreferencesManager (DataStore)
TASK: Write PreferencesManager with DataStore Preferences.
CONTEXT TO ATTACH: FR-SET-02, FR-CRD-02.
FILES TO CREATE OR MODIFY: data/prefs/PreferencesManager.kt
RULES: No secrets in DataStore. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete file.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-05.1.3. If the result fails verification: git restore . (or git reset --hard before-05.1.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Values persist.

12. Verification procedure

Restart app: values remain.

13. Acceptance criteria

[ ] Per-account coordinate format.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 05.1.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 05.1.4 — Offline Guard (Debug)

Provenance: [SPEC 6.6]

1. Step ID / 2. Title

05.1.4 — Offline Guard (Debug) (type: AI-assisted coding)

3. Objective

Add a debug-only guard that reports any outbound network request.

4. Why it is necessary

AC-OFF-1 requires zero outbound requests; INTERNET permission exists only for local sockets.

5. Prerequisites

Step 05.1.3 done and verified (tracker ticked). Phase Gate: Room tests and persistence tests pass; kill-process restore proven later in 10.3.4.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Debug build: install a global ProxySelector/URL handler and socket factory wrapper that records any non-loopback, non-local-network connection and shows a visible warning.
2. Expose counters on a debug screen.
3. Document limits: native MapLibre network cannot be fully seen; additionally verify with airplane mode and with adb shell dumpsys netstats or a router log (record method used).

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 05.1.4 - Offline Guard (Debug)
TASK: Write a debug OfflineGuard that logs non-loopback connections via ProxySelector and a counter screen.
CONTEXT TO ATTACH: Spec 6.6, AC-OFF-1.
FILES TO CREATE OR MODIFY: debug/OfflineGuard.kt
RULES: Debug source set only. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-05.1.4. If the result fails verification: git restore . (or git reset --hard before-05.1.4), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Guard reports 0.

12. Verification procedure

Trigger a test URL fetch: the guard flags it.

13. Acceptance criteria

[ ] Guard detects a deliberate request.

14. Common errors

- Native requests invisible -> also use airplane mode runs.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 05.1.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STAGE 05.2 — Project Repository

STEP 05.2.1 — ProjectRepository CRUD and Library Index

Provenance: [SPEC 5.1]

1. Step ID / 2. Title

05.2.1 — ProjectRepository CRUD and Library Index (type: AI-assisted coding)

3. Objective

Create, open, rename, delete (with confirmation handled by UI) and list projects for an account.

4. Why it is necessary

FR-PRJ-03, 12.

5. Prerequisites

Step 05.1.4 done and verified (tracker ticked). Phase Gate: Room tests and persistence tests pass; kill-process restore proven later in 10.3.4.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Library row: id, name, description, createdAt, modifiedAt, objectCount, mapSourceId, thumbnail ref.
2. Queries scoped by accountId.
3. Flow-based recents.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 05.2.1 - ProjectRepository CRUD and Library Index
TASK: Write ProjectRepository and tests for account isolation.
CONTEXT TO ATTACH: FR-PRJ-03, FR-PRJ-12.
FILES TO CREATE OR MODIFY: data/repo/ProjectRepository.kt
RULES: Repository is the only writer. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native,
  Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-05.2.1. If the result fails verification: git restore . (or git reset --hard before-05.2.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Tests pass.

12. Verification procedure

Two accounts cannot see each other's projects.

13. Acceptance criteria

[ ] Account isolation test passes.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 05.1.4; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 05.2.2 — Autosave Engine and Atomic Save

Provenance: [SPEC 5.1]

1. Step ID / 2. Title

05.2.2 — Autosave Engine and Atomic Save (type: AI-assisted coding)

3. Objective

Debounced autosave (1-2 s), single writer per project, atomic transaction, save-state machine.

4. Why it is necessary

FR-PRJ-05/06/07.

5. Prerequisites

Step 05.2.1 done and verified (tracker ticked). Phase Gate: Room tests and persistence tests pass; kill-process restore proven later in 10.3.4.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Save states Saved/Saving/Unsaved/Error exposed as Flow.
2. Write new revision and keep previous until read-back verifies.
3. Triggers: debounce after commands, onStop, close, low memory.
4. Never on main thread.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 05.2.2 - Autosave Engine and Atomic Save
TASK: Write AutosaveEngine with a fake clock and tests for debounce and failure.
CONTEXT TO ATTACH: FR-PRJ-05..07.
FILES TO CREATE OR MODIFY: data/repo/AutosaveEngine.kt
RULES: Coroutines; one writer per project via Mutex. Native Android only: Kotlin, Jetpack Compose, Material 3,
  MapLibre Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via
  AppContainer. Do not add a dependency that is not already in the version catalog; if one is needed, stop and
  say so. Store geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded
  user-visible strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use start/end,
  never left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim something
  works unless it was built and tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-05.2.2. If the result fails verification: git restore . (or git reset --hard before-05.2.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Tests pass.

12. Verification procedure

Simulated write failure yields Error state and keeps data.

13. Acceptance criteria

[ ] Read-back verification implemented.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 05.2.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 05.2.3 — Last-Known-Good Recovery and Corruption Handling

Provenance: [SPEC FR-PRJ-08]

1. Step ID / 2. Title

05.2.3 — Last-Known-Good Recovery and Corruption Handling (type: AI-assisted coding)

3. Objective

Detect unreadable documents and offer restore.

4. Why it is necessary

FR-PRJ-08: never crash or white-screen.

5. Prerequisites

Step 05.2.2 done and verified (tracker ticked). Phase Gate: Room tests and persistence tests pass; kill-process restore proven later in 10.3.4.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Load returns Result with LoadFailure type.

2. RestoreLastKnownGood operation.
3. Fixture of a corrupted JSON in tests.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 05.2.3 - Last-Known-Good Recovery and Corruption Handling
TASK: Write recovery logic and tests using a corrupt fixture.
CONTEXT TO ATTACH: FR-PRJ-08.
FILES TO CREATE OR MODIFY: data/repo/Recovery.kt
RULES: No exceptions escape. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-05.2.3. If the result fails verification: git restore . (or git reset --hard before-05.2.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Tests pass.

12. Verification procedure

Corrupt fixture restores previous revision.

13. Acceptance criteria

[ ] Corrupt document never crashes.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 05.2.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 05.2.4 — Duplicate Project (Deep Copy)

Provenance: [SPEC 5.1]

1. Step ID / 2. Title

05.2.4 — Duplicate Project (Deep Copy) (type: AI-assisted coding)

3. Objective

Deep copy with regenerated IDs and copied media.

4. Why it is necessary

FR-PRJ-03.

5. Prerequisites

Step 05.2.3 done and verified (tracker ticked). Phase Gate: Room tests and persistence tests pass; kill-process restore proven later in 10.3.4.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Regenerate every id (objects, layers, scenes, media) and fix internal references.
2. Copy media files; keep source fingerprint.
3. Test: no id shared with the original.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 05.2.4 - Duplicate Project (Deep Copy)
TASK: Write duplicate logic with an id-uniqueness test.
CONTEXT TO ATTACH: FR-PRJ-03.
FILES TO CREATE OR MODIFY: data/repo/DuplicateProject.kt
RULES: Pure logic plus file copy. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native,
  Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-05.2.4. If the result fails verification: git restore . (or git reset --hard before-05.2.4), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Tests pass.

12. Verification procedure

No shared ids.

13. Acceptance criteria

[ ] Independence verified.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 05.2.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 05.2.5 — Room and Migration Tests with Golden Fixtures

Provenance: [SPEC 16.2]

1. Step ID / 2. Title

05.2.5 — Room and Migration Tests with Golden Fixtures (type: AI-assisted coding)

3. Objective

Complete the Room test suite required by Spec 16.2.

4. Why it is necessary

Regression protection.

5. Prerequisites

Step 05.2.4 done and verified (tracker ticked). Phase Gate: Room tests and persistence tests pass; kill-process restore proven later in 10.3.4.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. CRUD, duplicate independence, delete, corrupt document, last-known-good, migration fixtures, account isolation.
2. Run on the Lenovo via connectedDebugAndroidTest.

8. Terminal commands

Where to run: PowerShell, device connected.

```
.\gradlew.bat connectedDebugAndroidTest
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 05.2.5 - Room and Migration Tests with Golden Fixtures
TASK: Write the missing Room tests listed.
CONTEXT TO ATTACH: Spec 16.2 Room tests.
FILES TO CREATE OR MODIFY: app/src/androidTest/**
RULES: Never weaken tests. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-05.2.5. If the result fails verification: git restore . (or git reset --hard before-05.2.5), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

All pass.

12. Verification procedure

BUILD SUCCESSFUL.

13. Acceptance criteria

[ ] All Spec 16.2 Room tests present and passing.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 05.2.4; leads to the next step in the phase order.

17. Estimated download / storage

No download.

PHASE 06 — AUTHENTICATION AND SECURITY

Local accounts, password hashing bound to AndroidKeyStore, lockout, auto-lock, recovery and login UI. Login is built before the editor shell so no screen is ever reachable without it.

Estimated relative effort: L. Gate: AC-SEC-1 passes on devices except items needing later features.

STAGE 06.1 — Accounts and Credentials

STEP 06.1.1 — Password Hashing with AndroidKeyStore Binding

Provenance: [SPEC FR-SEC-04]

1. Step ID / 2. Title

06.1.1 — Password Hashing with AndroidKeyStore Binding (type: AI-assisted coding)

3. Objective

PBKDF2-HMAC-SHA256 calibrated to about 0.3-0.5 s, then keyed with a non-exportable Keystore HMAC key.

4. Why it is necessary

FR-SEC-04.

5. Prerequisites

Step 05.2.5 done and verified (tracker ticked). Phase Gate: AC-SEC-1 passes on devices except items needing later features.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Calibrate iterations at first run on the device (minimum 600,000).
2. Random per-account salt (SecureRandom).
3. Keystore HMAC-SHA256 key (non-exportable); stored value = HMAC(PBKDF2 output).
4. Constant-time compare (MessageDigest.isEqual).
5. Never log or store plain passwords.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 06.1.1 - Password Hashing with AndroidKeyStore Binding
TASK: Write PasswordHasher with calibration and Keystore HMAC binding.
CONTEXT TO ATTACH: FR-SEC-04.
FILES TO CREATE OR MODIFY: data/auth/PasswordHasher.kt
RULES: javax.crypto only. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete file.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-06.1.1. If the result fails verification: git restore . (or git reset --hard before-06.1.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Hash verifies.

12. Verification procedure

Instrumented test: right password passes, wrong fails; stored data contains no plain text.

13. Acceptance criteria

[ ] Iteration count >= 600,000.
[ ] Keystore key non-exportable.

14. Common errors

- Keystore unavailable on emulator -> use the device.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 05.2.5; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 06.1.2 — AuthRepository, Roles and Lockout

Provenance: [SPEC 10.1]

1. Step ID / 2. Title

06.1.2 — AuthRepository, Roles and Lockout (type: AI-assisted coding)

3. Objective

Accounts (Admin/Member), brute-force delays and recovery code logic.

4. Why it is necessary

FR-SEC-02,03,05,07,08.

5. Prerequisites

Step 06.1.1 done and verified (tracker ticked). Phase Gate: AC-SEC-1 passes on devices except items needing later features.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. First account is Admin; no default password.
2. Lockout: 5 failures then 30 s doubling to 15 min; counter persisted.
3. Recovery code generated once (high entropy), stored hashed.
4. Admin resets Member passwords; erase-all operation.
5. Minimum length 8 with strength indicator logic.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 06.1.2 - AuthRepository, Roles and Lockout
TASK: Write AuthRepository with a fake clock for lockout tests.
CONTEXT TO ATTACH: FR-SEC-02..08.
FILES TO CREATE OR MODIFY: data/auth/AuthRepository.kt
RULES: No cloud. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room, DataStore,
  kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a dependency that
  is not already in the version catalog; if one is needed, stop and say so. Store geographic data as WGS84
  lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use string
  resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or update
  unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-06.1.2. If the result fails verification: git restore . (or git reset --hard before-06.1.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Tests pass.

12. Verification procedure

Lockout persists across process restart (instrumented).

13. Acceptance criteria

[ ] Delay doubling tested.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 06.1.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 06.1.3 — SessionManager, Auto-lock and Process-death Behaviour

Provenance: [SPEC 10.1]

1. Step ID / 2. Title

06.1.3 — SessionManager, Auto-lock and Process-death Behaviour (type: AI-assisted coding)

3. Objective

Session state, auto-lock timer, Lock now, presentation exemption hook.

4. Why it is necessary

FR-SEC-06, S1-21.

5. Prerequisites

Step 06.1.2 done and verified (tracker ticked). Phase Gate: AC-SEC-1 passes on devices except items needing later features.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Lock after background/idle (default 5 min; 1/5/15/30/Never).
2. Process death returns to locked state.
3. Hook: presentation active suspends auto-lock (wired in Phase 14).
4. Locking ends companion sessions (wired in Phase 16).

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 06.1.3 - SessionManager, Auto-lock and Process-death Behaviour
TASK: Write SessionManager with fake clock tests.
CONTEXT TO ATTACH: FR-SEC-06.
FILES TO CREATE OR MODIFY: feature/auth/SessionManager.kt
RULES: Domain-testable. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete file.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-06.1.3. If the result fails verification: git restore . (or git reset --hard before-06.1.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Tests pass.

12. Verification procedure

Kill app: login required.

13. Acceptance criteria

[ ] Never fires during presentation (test with flag).

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 06.1.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STAGE 06.2 — Login UI

STEP 06.2.1 — S0 Lock/Login Screen and First-run Admin

Provenance: [SPEC FR-SEC-01,02]

1. Step ID / 2. Title

06.2.1 — S0 Lock/Login Screen and First-run Admin (type: AI-assisted coding)

3. Objective

Build S0 per Guide section 08.

4. Why it is necessary

BA-02; nothing visible before login.

5. Prerequisites

Step 06.1.3 done and verified (tracker ticked). Phase Gate: AC-SEC-1 passes on devices except items needing later features.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Centered card on tablet; full-width on phone; no rail, panels or context bar.
2. Fields account and password, visibility toggle, Sign in; first-run Create Admin with confirm and strength; clear incorrect-password and lockout states.
3. No project names or thumbnails before login.
4. Strings in both languages.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 06.2.1 - S0 Lock/Login Screen and First-run Admin
TASK: Build the Lock/Login and first-run screens in Compose Material 3 with the theme tokens.
CONTEXT TO ATTACH: Guide section 08 and Spec 10.1.
FILES TO CREATE OR MODIFY: feature/auth/LoginScreen.kt, ViewModel
RULES: No hard-coded strings; 48 dp targets. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre
  Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do
  not add a dependency that is not already in the version catalog; if one is needed, stop and say so. Store
  geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible
  strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never
  left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim something works
  unless it was built and tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-06.2.1. If the result fails verification: git restore . (or git reset --hard before-06.2.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Login works.

12. Verification procedure

On both devices: create Admin, lock, log in, wrong password delay shown.

13. Acceptance criteria

[ ] Cold start shows S0 first.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 06.1.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 06.2.2 — Recovery Code and Erase-all Flow

Provenance: [SPEC FR-SEC-07]

1. Step ID / 2. Title

06.2.2 — Recovery Code and Erase-all Flow (type: AI-assisted coding)

3. Objective

Recovery-code display and the heavily confirmed erase-all flow.

4. Why it is necessary

FR-SEC-07.

5. Prerequisites

Step 06.2.1 done and verified (tracker ticked). Phase Gate: AC-SEC-1 passes on devices except items needing later features.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Show code once with copy guidance and 'I saved it' confirmation.

2. Erase-all: explicit warning, typed confirmation, wipes accounts and app data.
3. Dialog states plainly that lost Admin password plus code means starting over.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 06.2.2 - Recovery Code and Erase-all Flow
TASK: Build recovery code and erase-all dialogs.
CONTEXT TO ATTACH: FR-SEC-07, Guide section 08 dialog table.
FILES TO CREATE OR MODIFY: feature/auth/RecoveryDialogs.kt
RULES: Cancel is default focus on destructive dialog. Native Android only: Kotlin, Jetpack Compose, Material 3,
  MapLibre Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via
  AppContainer. Do not add a dependency that is not already in the version catalog; if one is needed, stop and
  say so. Store geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded
  user-visible strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use start/end,
  never left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim something
  works unless it was built and tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-06.2.2. If the result fails verification: git restore . (or git reset --hard before-06.2.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Flows work.

12. Verification procedure

Erase-all leaves a clean first-run state.

13. Acceptance criteria

[ ] Code never logged.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 06.2.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 06.2.3 — S10 Accounts Screen

Provenance: [SPEC 10.1]

1. Step ID / 2. Title

06.2.3 — S10 Accounts Screen (type: AI-assisted coding)

3. Objective

Add/remove accounts, change password, rename, Admin reset.

4. Why it is necessary

FR-SEC-03, 08.

5. Prerequisites

Step 06.2.2 done and verified (tracker ticked). Phase Gate: AC-SEC-1 passes on devices except items needing later features.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Admin-only add/remove/reset; every user can change own password.
2. Deleting an account confirms and deletes only that account's data.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 06.2.3 - S10 Accounts Screen
TASK: Build the Accounts screen.
CONTEXT TO ATTACH: Guide S10.
FILES TO CREATE OR MODIFY: feature/auth/AccountsScreen.kt
RULES: Same rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room, DataStore,
  kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a dependency that
  is not already in the version catalog; if one is needed, stop and say so. Store geographic data as WGS84
  lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use string
  resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or update
  unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-06.2.3. If the result fails verification: git restore . (or git reset --hard before-06.2.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Works.

12. Verification procedure

Delete Member removes their projects only.

13. Acceptance criteria

[ ] Isolation holds after delete.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 06.2.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 06.2.4 — Screen Protection and Device-Lock Warning

Provenance: [SPEC 10.2]

1. Step ID / 2. Title

06.2.4 — Screen Protection and Device-Lock Warning (type: AI-assisted coding)

3. Objective

FLAG_SECURE on login; optional Protect screen; warn when no device screen lock exists.

4. Why it is necessary

FR-SEC-01, 10.2.

5. Prerequisites

Step 06.2.3 done and verified (tracker ticked). Phase Gate: AC-SEC-1 passes on devices except items needing later features.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Login always secure; Settings option for editor (default off); never on audience window.
2. KeyguardManager.isDeviceSecure check with first-run warning.
3. Verify recent-apps thumbnail is blank on login.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 06.2.4 - Screen Protection and Device-Lock Warning
TASK: Implement secure-window handling and the device-lock warning.
CONTEXT TO ATTACH: Spec 10.2.
FILES TO CREATE OR MODIFY: feature/auth/ScreenProtection.kt
RULES: Not applied to Presentation windows. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre
  Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do
  not add a dependency that is not already in the version catalog; if one is needed, stop and say so. Store
  geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible
  strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never
  left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim something works
  unless it was built and tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-06.2.4. If the result fails verification: git restore . (or git reset --hard before-06.2.4), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Works.

12. Verification procedure

Screenshot attempt on login is blocked.

13. Acceptance criteria

[ ] Warning shown when no lock.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 06.2.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 06.2.5 — Security Tests and SECURITY.md Update

Provenance: [SPEC AC-SEC-1]

1. Step ID / 2. Title

06.2.5 — Security Tests and SECURITY.md Update (type: AI-assisted coding)

3. Objective
Complete AC-SEC-1 tests and document design.

4. Why it is necessary
Evidence rule.

5. Prerequisites
Step 06.2.4 done and verified (tracker ticked). Phase Gate: AC-SEC-1 passes on devices except items needing later features.

6. Required software
Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions
1. Tests: gate after process death, lockout persistence, no plain password in stored data (inspect DB via adb run-as), auto-lock logic.
2. Fill SECURITY.md account/password/lock sections.

8. Terminal commands
Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links
None required for this step.

10. Complete AI implementation prompt
Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 06.2.5 - Security Tests and SECURITY.md Update
TASK: Write remaining security tests and the SECURITY.md sections.
CONTEXT TO ATTACH: AC-SEC-1.
FILES TO CREATE OR MODIFY: tests, docs/SECURITY.md
RULES: Do not weaken tests. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-06.2.5. If the result fails verification: git restore . (or git reset --hard before-06.2.5), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result
Pass.

12. Verification procedure
AC-SEC-1 items marked with evidence in MAPX_STATUS.

13. Acceptance criteria
[ ] AC-SEC-1 verified except presentation interaction (Phase 14).

14. Common errors
- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting
1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies
Depends on step 06.2.4; leads to the next step in the phase order.

17. Estimated download / storage
No download.

PHASE 07 — LAYOUT SYSTEM AND APPLICATION SHELL

The four layout modes, region tree, panels, Home and navigation, built from the Guide and Blueprint with sizes taken from one LayoutSpec. Chrome is always a sibling in a Row/Column; never offsets or z-index.

Estimated relative effort: L. Gate: regionsNeverOverlap passes in four modes and two languages; stress sizes checked on devices.

STAGE 07.1 — Layout Engine

STEP 07.1.1 — LayoutMode Classifier and LayoutSpec

Provenance: [SPEC 12.2]

1. Step ID / 2. Title

07.1.1 — LayoutMode Classifier and LayoutSpec (type: AI-assisted coding)

3. Objective

Implement the one classifier (window-based) and LayoutSpec with every dp value.

4. Why it is necessary

Laws L9, L11.

5. Prerequisites

Step 06.2.5 done and verified (tracker ticked). Phase Gate: regionsNeverOverlap passes in four modes and two languages; stress sizes checked on devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Smallest side >= 600 dp = tablet else phone; width > height = landscape.
2. LayoutSpec values: top bar 48/48/40/56; rail 72/72/56/none; dock none/none/none/64; context 56/56/44/48; side panel 320 (280-360)/none/280/none; sheet peek 80, half 40% (tablet portrait), peek 72, half 50% (phone portrait).
3. Re-evaluate on rotation, split-screen, resize and font scale.
4. Pure unit tests for classification at the ten stress sizes.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 07.1.1 - LayoutMode Classifier and LayoutSpec
TASK: Write LayoutMode, LayoutClassifier and LayoutSpec with tests at the ten stress sizes.
CONTEXT TO ATTACH: Guide section 14 table; Blueprint section 4.
FILES TO CREATE OR MODIFY: ui/layout/LayoutMode.kt, LayoutSpec.kt
RULES: All dp in LayoutSpec only. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native,
  Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-07.1.1. If the result fails verification: git restore . (or git reset --hard before-07.1.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Tests pass.

12. Verification procedure

1280x800 = tablet landscape; 844x390 = phone landscape.

13. Acceptance criteria

[ ] Ten sizes classified correctly.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 06.2.5; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 07.1.2 — MapXLayoutRoot with Region Tree (Tablet Landscape)

Provenance: [SPEC 12.2]

1. Step ID / 2. Title

07.1.2 — MapXLayoutRoot with Region Tree (Tablet Landscape) (type: AI-assisted coding)

3. Objective

Build the root: insets once, TopStatusBar, ToolRail (LazyColumn, 48 dp items), central Column (MapWorkspace + BottomContextBar), optional SidePanel; ModalLayer last.

4. Why it is necessary

Laws L1-L5.

5. Prerequisites

Step 07.1.1 done and verified (tracker ticked). Phase Gate: regionsNeverOverlap passes in four modes and two languages; stress sizes checked on devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Use the Blueprint skeleton (section 14) as the structure; testTags region_top/rail/dock/map/context/panel.
2. MapWorkspace is a placeholder colour box for now (real map in Phase 08).
3. Opening panel makes map narrower; never overlays.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 07.1.2 - MapXLayoutRoot with Region Tree (Tablet Landscape)
TASK: Implement MapXLayoutRoot for TABLET_LANDSCAPE following the skeleton.
CONTEXT TO ATTACH: Blueprint section 14 skeleton and laws; Guide section 09.
FILES TO CREATE OR MODIFY: ui/layout/MapXLayoutRoot.kt, regions/*.kt
RULES: No offset(), zIndex() or absolute positioning. Native Android only: Kotlin, Jetpack Compose, Material 3,
  MapLibre Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via
  AppContainer. Do not add a dependency that is not already in the version catalog; if one is needed, stop and
  say so. Store geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded
  user-visible strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use start/end,
  never left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim something
  works unless it was built and tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-07.1.2. If the result fails verification: git restore . (or git reset --hard before-07.1.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Screen matches wireframe proportions.

12. Verification procedure

On Lenovo landscape the regions look like Blueprint Figure 1.

13. Acceptance criteria

[ ] Regions are siblings.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 07.1.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 07.1.3 — Remaining Three Modes (Tablet Portrait, Phone Landscape, Phone Portrait)

Provenance: [ASSUMED C-08]

1. Step ID / 2. Title

07.1.3 — Remaining Three Modes (Tablet Portrait, Phone Landscape, Phone Portrait) (type: AI-assisted coding)

3. Objective

Add the other modes including portrait bottom sheet host and the five-slot dock.

4. Why it is necessary

Four modes are mandatory.

5. Prerequisites

Step 07.1.2 done and verified (tracker ticked). Phase Gate: regionsNeverOverlap passes in four modes and two languages; stress sizes checked on devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Tablet portrait: 72 dp rail, sheet peek 80/half 40%/full; phone landscape: 56 dp rail, side panel 280; phone portrait: dock Select, Pan, Draw, Layers, More (C-01 default).
2. Phone landscape rail five slots + More: assumption pending owner (C-08): Select, Pan, Draw, Layers, Scenes then More; record as D-R09.
3. Sheet covers map only; map controls inset follows sheet height.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 07.1.3 - Remaining Three Modes (Tablet Portrait, Phone Landscape, Phone Portrait)
TASK: Implement the remaining LayoutModes.
CONTEXT TO ATTACH: Guide section 14; Blueprint sections 5.2-5.4.
FILES TO CREATE OR MODIFY: ui/layout/*.kt
RULES: Same laws. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room, DataStore,
  kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a dependency that
  is not already in the version catalog; if one is needed, stop and say so. Store geographic data as WGS84
  lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use string
  resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or update
  unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-07.1.3. If the result fails verification: git restore . (or git reset --hard before-07.1.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

All four modes render.

12. Verification procedure

Rotate both devices.

13. Acceptance criteria

[ ] Exactly one rail or dock per mode.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 07.1.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 07.1.4 — PanelManager and PanelContent Frame

Provenance: [SPEC 12.2]

1. Step ID / 2. Title

07.1.4 — PanelManager and PanelContent Frame (type: AI-assisted coding)

3. Objective

One ActivePanel, reusable PanelContent (header 44, scroll body, footer 48).

4. Why it is necessary

Law L6; Guide section 11.

5. Prerequisites

Step 07.1.3 done and verified (tracker ticked). Phase Gate: regionsNeverOverlap passes in four modes and two languages; stress sizes checked on devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. ActivePanel: None, Properties, Layers, Symbols, Scenes, MapSources, GisImport, SceneGraphics.
2. Tap same item closes; another replaces; selection opens Properties only if None or Properties.
3. Rotation preserves panel, scroll and input (rememberSaveable).

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 07.1.4 - PanelManager and PanelContent Frame
TASK: Write PanelManager and the PanelContent frame with placeholder content.
CONTEXT TO ATTACH: Blueprint section 8.
FILES TO CREATE OR MODIFY: ui/layout/PanelManager.kt, PanelContent.kt
RULES: Exactly one panel. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-07.1.4. If the result fails verification: git restore . (or git reset --hard before-07.1.4), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Works.

12. Verification procedure

Rotate with text typed: preserved.

13. Acceptance criteria

[ ] State survives rotation.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 07.1.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 07.1.5 — Layout Debug Mode

Provenance: [SPEC 12.2]

1. Step ID / 2. Title

07.1.5 — Layout Debug Mode (type: AI-assisted coding)

3. Objective

Permanent debug overlay showing mode, size and region outlines.

4. Why it is necessary

Spec 12.2.

5. Prerequisites

Step 07.1.4 done and verified (tracker ticked). Phase Gate: regionsNeverOverlap passes in four modes and two languages; stress sizes checked on devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Hidden toggle (debug builds or developer setting).
2. Draw outlines using the region test tags' bounds.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Google AI Studio free tier (best for large, multi-file generation because of its long context) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 07.1.5 - Layout Debug Mode
TASK: Write Layout Debug overlay.
CONTEXT TO ATTACH: Spec 12.2.
FILES TO CREATE OR MODIFY: ui/layout/LayoutDebug.kt
RULES: Not drawn in release by default. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre
  Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do
  not add a dependency that is not already in the version catalog; if one is needed, stop and say so. Store
  geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible
  strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never
  left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim something works
  unless it was built and tested.
OUTPUT: Complete file.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-07.1.5. If the result fails verification: git restore . (or git reset --hard before-07.1.5), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Works.

12. Verification procedure

Shows mode name.

13. Acceptance criteria

[ ] Region outlines visible.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 07.1.4; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 07.1.6 — regionsNeverOverlap Test and Stress-Size Harness

Provenance: [SPEC AC-UI-1]

1. Step ID / 2. Title

07.1.6 — regionsNeverOverlap Test and Stress-Size Harness (type: AI-assisted coding)

3. Objective

Compose UI test for every mode in English and Arabic plus the ten stress sizes.

4. Why it is necessary

AC-UI-1.

5. Prerequisites

Step 07.1.5 done and verified (tracker ticked). Phase Gate: regionsNeverOverlap passes in four modes and two languages; stress sizes checked on devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Use test tags; assert no two region rectangles intersect; map controls inside region_map; exactly one rail or dock.

2. Parameterise over modes x locales; use device config overrides.
3. Run on Lenovo and Samsung.

8. Terminal commands

Where to run: PowerShell, device connected.

```
.\gradlew.bat connectedDebugAndroidTest
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 07.1.6 - regionsNeverOverlap Test and Stress-Size Harness
TASK: Write the regionsNeverOverlap test per the Blueprint example, parameterised.
CONTEXT TO ATTACH: Blueprint section 14.
FILES TO CREATE OR MODIFY: androidTest/RegionsNeverOverlapTest.kt
RULES: Never relax assertions. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete file.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-07.1.6. If the result fails verification: git restore . (or git reset --hard before-07.1.6), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Zero failures.

12. Verification procedure

Test report green.

13. Acceptance criteria

[ ] AC-UI-1 test passes for 4 modes x 2 languages.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 07.1.5; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STAGE 07.2 — Shell Screens and Navigation

STEP 07.2.1 — Navigation Host and Back-stack Priority

Provenance: [SPEC 12.5]

1. Step ID / 2. Title

07.2.1 — Navigation Host and Back-stack Priority (type: AI-assisted coding)

3. Objective

Lightweight state-based navigation (D-R03) with the Spec back order.

4. Why it is necessary

Spec 12.5.

5. Prerequisites

Step 07.1.6 done and verified (tracker ticked). Phase Gate: regionsNeverOverlap passes in four modes and two languages; stress sizes checked on devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Routes: Login, Home, NewProject, Editor, Presenter, Settings, Georeference, Remote.
2. Back order: dialog/popup, panel/sheet, tool/selection, leave editor after autosave; Presenter asks confirmation.
3. Auto-lock return shows Login.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 07.2.1 - Navigation Host and Back-stack Priority
TASK: Write the navigator and back handler.
CONTEXT TO ATTACH: Spec 12.5.
FILES TO CREATE OR MODIFY: ui/nav/*.kt
RULES: No Navigation library. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-07.2.1. If the result fails verification: git restore . (or git reset --hard before-07.2.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Works.

12. Verification procedure

Back behaviour as specified on device.

13. Acceptance criteria

[ ] Back priority tested.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 07.1.6; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 07.2.2 — S1 Home Screen (Adaptive)

Provenance: [SPEC FR-PRJ-01]

1. Step ID / 2. Title

07.2.2 — S1 Home Screen (Adaptive) (type: AI-assisted coding)

3. Objective

Home with New Project, Open Project (overflow Import project package...), Recent Projects, Settings, account chip with Lock.

4. Why it is necessary

FR-PRJ-01, BA-11.

5. Prerequisites

Step 07.2.1 done and verified (tracker ticked). Phase Gate: regionsNeverOverlap passes in four modes and two languages; stress sizes checked on devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Phone: vertical list; tablet: card grid.
2. Cards: thumbnail, name, modified date, object count; actions Open, Rename, Duplicate, Delete (confirmed).
3. No rail, panels or context bar.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 07.2.2 - S1 Home Screen (Adaptive)
TASK: Build the Home screen.
CONTEXT TO ATTACH: Guide section 07; FR-PRJ-01.
FILES TO CREATE OR MODIFY: feature/projects/HomeScreen.kt
RULES: Exactly the listed actions. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native,
  Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-07.2.2. If the result fails verification: git restore . (or git reset --hard before-07.2.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Works.

12. Verification procedure

Create/delete project works.

13. Acceptance criteria

[ ] Home contains exactly the specified items.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 07.2.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 07.2.3 — S2 New Project Flow and Library Actions

Provenance: [SPEC FR-PRJ-02]

1. Step ID / 2. Title

07.2.3 — S2 New Project Flow and Library Actions (type: AI-assisted coding)

3. Objective

Two-step wizard; project creation opens the editor.

4. Why it is necessary

FR-PRJ-02.

5. Prerequisites

Step 07.2.2 done and verified (tracker ticked). Phase Gate: regionsNeverOverlap passes in four modes and two languages; stress sizes checked on devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Step 1 name required; step 2 choose existing source or import (import wired in Phase 09; bundled basemap option now).
2. Create enabled only when valid.
3. Missing source later: warning and Relink.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 07.2.3 - S2 New Project Flow and Library Actions
TASK: Build the New Project wizard dialogs.
CONTEXT TO ATTACH: Guide section 08.
FILES TO CREATE OR MODIFY: feature/projects/NewProject.kt
RULES: Single ModalLayer dialog. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native,
  Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-07.2.3. If the result fails verification: git restore . (or git reset --hard before-07.2.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Works.

12. Verification procedure

Project appears in Recent.

13. Acceptance criteria

[ ] Create validates.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies
Depends on step 07.2.2; leads to the next step in the phase order.

17. Estimated download / storage
No download.

STEP 07.2.4 — Top Bar, Tool Rail, Dock and Context Bar Content

Provenance: [SPEC 12.4]

1. Step ID / 2. Title
07.2.4 — Top Bar, Tool Rail, Dock and Context Bar Content (type: AI-assisted coding)

3. Objective
Populate bars per Guide sections 07-10 including collapse strategy.

4. Why it is necessary
BA-05, L8.

5. Prerequisites
Step 07.2.3 done and verified (tracker ticked). Phase Gate: regionsNeverOverlap passes in four modes and two languages; stress sizes checked on devices.

6. Required software
Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions
1. Top bar: Back, name, save state, centre readout, Undo, Redo, Present, More, Settings with collapse order.
2. Rail/dock items with icons (outline default, filled active).
3. Context bar situations table (idle hint, place tool, drawing, selection, imported, image, preview, GPS, georeference).
4. Delete is last-but-one; Properties last.

8. Terminal commands
Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links
None required for this step.

10. Complete AI implementation prompt
Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 07.2.4 - Top Bar, Tool Rail, Dock and Context Bar Content
TASK: Build the bars and their state-driven content.
CONTEXT TO ATTACH: Guide sections 04, 07, 09, 10.
FILES TO CREATE OR MODIFY: ui/layout/bars/*.kt
RULES: LazyRow/LazyColumn; never wrap. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre
  Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do
  not add a dependency that is not already in the version catalog; if one is needed, stop and say so. Store
  geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible
  strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never
  left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim something works
  unless it was built and tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-07.2.4. If the result fails verification: git restore . (or git reset --hard before-07.2.4), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result
Works.

12. Verification procedure
200% font scale: no second row.

13. Acceptance criteria
[ ] Bars scroll or truncate.

14. Common errors
- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 07.2.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 07.2.5 — ModalLayer (One at a Time)

Provenance: [SPEC 12.4]

1. Step ID / 2. Title

07.2.5 — ModalLayer (One at a Time) (type: AI-assisted coding)

3. Objective

Single host for dialogs, popups, menus and snackbars.

4. Why it is necessary

Law L4; Guide section 18.

5. Prerequisites

Step 07.2.4 done and verified (tracker ticked). Phase Gate: regionsNeverOverlap passes in four modes and two languages; stress sizes checked on devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. One item at a time; focus trapped; Back dismisses; popups max 60% height and scroll.
2. Snackbars auto-hide 4 s.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 07.2.5 - ModalLayer (One at a Time)
TASK: Write ModalLayer host and request queue.
CONTEXT TO ATTACH: Guide section 18.
FILES TO CREATE OR MODIFY: ui/layout/ModalLayer.kt
RULES: Never stack two dialogs. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-07.2.5. If the result fails verification: git restore . (or git reset --hard before-07.2.5), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Works.

12. Verification procedure

Open Draw popup then dialog: second replaces.

13. Acceptance criteria

[ ] LC-4 satisfied.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.

- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 07.2.4; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 07.2.6 — RTL Mirroring Pass on the Shell

Provenance: [SPEC 12.4]

1. Step ID / 2. Title

07.2.6 — RTL Mirroring Pass on the Shell (type: AI-assisted coding)

3. Objective

Verify and fix mirroring of every shell region.

4. Why it is necessary

BA-15.

5. Prerequisites

Step 07.2.5 done and verified (tracker ticked). Phase Gate: regionsNeverOverlap passes in four modes and two languages; stress sizes checked on devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Rail to end edge, panel to start, top bar reversed, controls to start edge.
2. Flip Back and Undo/Redo; do not flip compass, play, numbers.
3. Run regionsNeverOverlap in Arabic.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 07.2.6 - RTL Mirroring Pass on the Shell
TASK: Review and fix RTL issues in the shell given this checklist.
CONTEXT TO ATTACH: Guide section 15.
FILES TO CREATE OR MODIFY: ui/**
RULES: Use start/end only. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Changed files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-07.2.6. If the result fails verification: git restore . (or git reset --hard before-07.2.6), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Mirrored.

12. Verification procedure

Arabic on both devices.

13. Acceptance criteria

[ ] LC-2 satisfied for shell.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 07.2.5; leads to the next step in the phase order.

17. Estimated download / storage

No download.

PHASE 08 — MAP ENGINE AND BUNDLED BASEMAP

MapEngine abstraction over MapLibre, map controls, coordinate readout and search, and the bundled offline basemap with bilingual labels.

Estimated relative effort: L. Gate: a new project shows the offline world map in airplane mode on both devices.

STAGE 08.1 — MapEngine Abstraction

STEP 08.1.1 — MapEngine Interface and MapLibreMapView

Provenance: [SPEC FR-MAP-01]

1. Step ID / 2. Title

08.1.1 — MapEngine Interface and MapLibreMapView (type: AI-assisted coding)

3. Objective

Define MapEngine and implement it with MapLibre in the map package only.

4. Why it is necessary

FR-MAP-01; architecture test limits MapLibre imports to the map package.

5. Prerequisites

Step 07.2.6 done and verified (tracker ticked). Phase Gate: a new project shows the offline world map in airplane mode on both devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Interface: camera get/set/animate, toScreen/fromScreen, add/update/remove sources and layers, events, snapshot.
2. MapLibreMapView composable with lifecycle.
3. Add MapLibre dependency at the version from D-R06/D-R07.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 08.1.1 - MapEngine Interface and MapLibreMapView
TASK: Implement MapEngine and MapLibreMapView.
CONTEXT TO ATTACH: Spec 5.2, 7.1; spike decisions.
FILES TO CREATE OR MODIFY: map/MapEngine.kt, MapLibreMapView.kt
RULES: Only the map package imports org.maplibre. Native Android only: Kotlin, Jetpack Compose, Material 3,
  MapLibre Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via
  AppContainer. Do not add a dependency that is not already in the version catalog; if one is needed, stop and
  say so. Store geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded
  user-visible strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use start/end,
  never left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim something
  works unless it was built and tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-08.1.1. If the result fails verification: git restore . (or git reset --hard before-08.1.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Map shows in MapWorkspace.

12. Verification procedure

Architecture test passes.

13. Acceptance criteria

- [ ] No MapLibre import outside map.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 07.2.6; leads to the next step in the phase order.

17. Estimated download / storage

MapLibre AAR (see spike).

STEP 08.1.2 — Camera, Projection and Event Coalescing

Provenance: [SPEC FR-MAP-03]

1. Step ID / 2. Title

08.1.2 — Camera, Projection and Event Coalescing (type: AI-assisted coding)

3. Objective

Camera model (centre, zoom, bearing, pitch), projection round-trip, one update per frame.

4. Why it is necessary

FR-MAP-03; Spec 7.4.

5. Prerequisites

Step 08.1.1 done and verified (tracker ticked). Phase Gate: a new project shows the offline world map in airplane mode on both devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Pan, pinch, rotate, tilt gestures; optional two-finger pan.
2. Projection round trip within 1 px (instrumented test).
3. Coalesce camera events with Choreographer.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 08.1.2 - Camera, Projection and Event Coalescing
TASK: Implement camera and projection with coalescing.
CONTEXT TO ATTACH: FR-MAP-03.
FILES TO CREATE OR MODIFY: map/Camera.kt, ProjectionImpl.kt
RULES: No per-event recomposition. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native,
  Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-08.1.2. If the result fails verification: git restore . (or git reset --hard before-08.1.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Works.

12. Verification procedure

Instrumented projection test passes.

13. Acceptance criteria

[ ] Projection round-trip < 1 px.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 08.1.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 08.1.3 — Map Controls (Compass, Zoom, Scale, Attribution)

Provenance: [SPEC FR-MAP-06]

1. Step ID / 2. Title

08.1.3 — Map Controls (Compass, Zoom, Scale, Attribution) (type: AI-assisted coding)

3. Objective

Controls inside MapWorkspace at 10 dp, with RTL placement and sheet inset.

4. Why it is necessary

FR-MAP-06; BA-06.

5. Prerequisites

Step 08.1.2 done and verified (tracker ticked). Phase Gate: a new project shows the offline world map in airplane mode on both devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Stack at end edge: compass, zoom in, zoom out, Locate (Locate wired in Phase 17).
2. Scale bar start-bottom; attribution end-bottom; controls ride above sheet edge.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 08.1.3 - Map Controls (Compass, Zoom, Scale, Attribution)
TASK: Build map controls.
CONTEXT TO ATTACH: Guide section 09.
FILES TO CREATE OR MODIFY: feature/editor/MapControls.kt
RULES: Constrained inside map bounds. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native,
  Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-08.1.3. If the result fails verification: git restore . (or git reset --hard before-08.1.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result
Works.

12. Verification procedure
Open sheet: controls move up.

13. Acceptance criteria
[ ] Controls inside region_map.

14. Common errors
- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting
1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies
Depends on step 08.1.2; leads to the next step in the phase order.

17. Estimated download / storage
No download.

STEP 08.1.4 — Coordinate Readout, Format Switcher and Coordinate Search

Provenance: [SPEC FR-MAP-05]

1. Step ID / 2. Title
08.1.4 — Coordinate Readout, Format Switcher and Coordinate Search (type: AI-assisted coding)

3. Objective
Top-bar readout (coordinate, zoom, scale) tappable to switch DD/DDM/DMS/UTM/MGRS; search by coordinates.

4. Why it is necessary
FR-MAP-04/05; BA-05.

5. Prerequisites
Step 08.1.3 done and verified (tracker ticked). Phase Gate: a new project shows the offline world map in airplane mode on both devices.

6. Required software
Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions
1. Format saved per account.
2. Search auto-detects format; invalid input explained; polar UTM falls back to DD with a short note.

8. Terminal commands
Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links
None required for this step.

10. Complete AI implementation prompt
Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 08.1.4 - Coordinate Readout, Format Switcher and Coordinate Search
TASK: Build readout and coordinate search.
CONTEXT TO ATTACH: FR-MAP-04/05.
FILES TO CREATE OR MODIFY: feature/editor/Readout.kt, CoordinateSearch.kt
RULES: Hemisphere letters Latin. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native,
  Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-08.1.4. If the result fails verification: git restore . (or git reset --hard before-08.1.4), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Works.

12. Verification procedure

Enter an MGRS string: map centres correctly.

13. Acceptance criteria

[ ] Round trip across all five formats.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 08.1.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STAGE 08.2 — Bundled Offline Basemap

STEP 08.2.1 — Build the Basemap Data from Natural Earth

Provenance: [SPEC FR-MAP-02]

1. Step ID / 2. Title

08.2.1 — Build the Basemap Data from Natural Earth (type: environment check)

3. Objective

Prepare small simplified Natural Earth GeoJSON (coastlines/countries/populated places with Arabic names).

4. Why it is necessary

FR-MAP-02: bundled basemap, budget at most 60 MB; target under 15 MB.

5. Prerequisites

Step 08.1.4 done and verified (tracker ticked). Phase Gate: a new project shows the offline world map in airplane mode on both devices.

6. Required software

Windows PowerShell, adb (Platform-Tools), a text editor; an AI tool from the AI field for interpretation.

7. Exact instructions

1. Download Natural Earth 1:110m and 1:50m admin-0 countries and populated places (official site; sizes are small; check each before downloading).
2. Natural Earth attributes include name and Arabic name fields; confirm in the file.
3. Convert and simplify with mapshaper via npx (Node is installed): geometry simplification, keep attributes name, name_ar.
4. Write attribution (Natural Earth is public domain; still record source) in docs/licenses.

8. Terminal commands

Where to run: PowerShell in a temp folder.

```
# What: simplifies a shapefile to GeoJSON. Success: output file created.
npx mapshaper ne_50m_admin_0_countries.shp -simplify 10% -o format=geojson countries.json
```

9. Official download links

- Natural Earth: https://www.naturalearthdata.com/

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 08.2.1 - Build the Basemap Data from Natural Earth
TASK: Give me mapshaper commands to convert and simplify Natural Earth countries, coastline and populated places
  to small GeoJSON keeping name and Arabic-name fields.
CONTEXT TO ATTACH: File list.
FILES TO CREATE OR MODIFY: Only the files needed for this step; list them before writing code.
RULES: Keep total under 15 MB. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Command list.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-08.2.1. If the result fails verification: git restore . (or git reset --hard before-08.2.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

GeoJSON assets ready.

12. Verification procedure

Files open in a viewer; sizes recorded.

13. Acceptance criteria

[ ] Total size recorded.

[ ] Arabic name field verified present.

14. Common errors

- npx tries to download mapshaper: allow (small).

15. Troubleshooting

1) Re-run the command exactly as printed. 2) Compare your output with the 'Expected result'. 3) Paste command and output to the AI with 'Interpret this; do not suggest reinstalling anything unless it is proven missing.' 4) Record UNVERIFIED if still unclear.

16. Dependencies

Depends on step 08.1.4; leads to the next step in the phase order.

17. Estimated download / storage

Natural Earth zips: a few to tens of MB; mapshaper small (verify).

STEP 08.2.2 — Glyphs, Style and Bilingual Labels

Provenance: [SPEC 6.7]

1. Step ID / 2. Title

08.2.2 — Glyphs, Style and Bilingual Labels (type: AI-assisted coding)

3. Objective

Create the offline style using GeoJSON sources and Cairo glyphs; label expression picks Arabic or English by app language.

4. Why it is necessary

Spec 6.7, 7.5.

5. Prerequisites

Step 08.2.1 done and verified (tracker ticked). Phase Gate: a new project shows the offline world map in airplane mode on both devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. style.json with country fills, borders, coastline, city circles and symbol layers.
2. Text field expression chooses name_ar when language is Arabic, else name_en, else name.
3. Glyph files from the spike (03.1.4).
4. Switch style text on language change without reload flicker.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 08.2.2 - Glyphs, Style and Bilingual Labels
TASK: Write the basemap style.json and Kotlin code switching label language.
CONTEXT TO ATTACH: Spec 6.7, 7.5.
FILES TO CREATE OR MODIFY: assets/basemap/style.json, map/BasemapStyle.kt
RULES: No online URLs. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-08.2.2. If the result fails verification: git restore . (or git reset --hard before-08.2.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Offline world map.

12. Verification procedure

Airplane mode: map and labels render.

13. Acceptance criteria

[ ] No network access (offline guard 0).

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 08.2.1; leads to the next step in the phase order.

17. Estimated download / storage

Assets from 08.2.1.

STEP 08.2.3 — Antimeridian Handling and Arabic Reference Test

Provenance: [SPEC FR-MAP-07]

1. Step ID / 2. Title

08.2.3 — Antimeridian Handling and Arabic Reference Test (type: AI-assisted coding)

3. Objective

Verify geometry across +-180 and the Arabic reference label set on the basemap.

4. Why it is necessary

FR-MAP-07, Spec 7.5 mandatory verification.

5. Prerequisites

Step 08.2.2 done and verified (tracker ticked). Phase Gate: a new project shows the offline world map in airplane mode on both devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Longitude unwrapping for objects crossing the antimeridian (or documented limitation with a test).
2. Run the reference Arabic label set on the reference tablet; record screenshots.
3. Apply the fallback if unacceptable.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 08.2.3 - Antimeridian Handling and Arabic Reference Test
TASK: Write longitude unwrapping utilities and tests.
CONTEXT TO ATTACH: FR-MAP-07.
FILES TO CREATE OR MODIFY: core/geo/Antimeridian.kt
RULES: Domain pure. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room, DataStore,
  kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a dependency that
  is not already in the version catalog; if one is needed, stop and say so. Store geographic data as WGS84
  lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use string
  resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or update
  unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-08.2.3. If the result fails verification: git restore . (or git reset --hard before-08.2.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Pass.

12. Verification procedure

Draw a line from Fiji to Samoa region: correct path.

13. Acceptance criteria

[ ] AC-GEO-1 antimeridian item verified or documented.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 08.2.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

PHASE 09 — MAP SOURCES AND IMPORT PIPELINE

Source library, safe streaming import, classification, MBTiles/PMTiles/XYZ, image sources and the Maps panel, with a format evidence run.

Estimated relative effort: XL. Gate: every format in the matrix is classified Supported/Partial/Deferred with a real sample.

STAGE 09.1 — Import Foundation

STEP 09.1.1 — Source Library Data Model and Storage

Provenance: [SPEC 6.1]

1. Step ID / 2. Title

09.1.1 — Source Library Data Model and Storage (type: AI-assisted coding)

3. Objective

Library tables and metadata per FR-SRC-*.

4. Why it is necessary

FR-SRC-01..05.

5. Prerequisites

Step 08.2.3 done and verified (tracker ticked). Phase Gate: every format in the matrix is classified Supported/Partial/Deferred with a real sample.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Fields: id, name, original file name, format, size, dimensions, bounds, CRS, zoom range, tile type, import date, location, attribution, SHA-256.
2. MapSourceRef in projects stores id plus fingerprint.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 09.1.1 - Source Library Data Model and Storage
TASK: Write source library entities, DAO and repository.
CONTEXT TO ATTACH: Spec 6.1.
FILES TO CREATE OR MODIFY: data/sources/*.kt
RULES: Per-account folders. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-09.1.1. If the result fails verification: git restore . (or git reset --hard before-09.1.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Tests pass.

12. Verification procedure

Room tests pass.

13. Acceptance criteria

[ ] Deleting in-use source is blocked or warned.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 08.2.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 09.1.2 — Import Pipeline (SAF, Free Space, Stream Copy, SHA-256, Cancel)

Provenance: [SPEC 6.2]

1. Step ID / 2. Title

09.1.2 — Import Pipeline (SAF, Free Space, Stream Copy, SHA-256, Cancel) (type: AI-assisted coding)

3. Objective

Implement the pipeline from Spec 6.2.

4. Why it is necessary

Large files must never enter RAM.

5. Prerequisites

Step 09.1.1 done and verified (tracker ticked). Phase Gate: every format in the matrix is classified Supported/Partial/Deferred with a real sample.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. OpenDocument picker; free-space check first; stream copy into staging with SHA-256 computed while copying; progress; cancel deletes partial files.
2. Duplicate detection by hash.
3. Atomic move from staging to library after validation; pause/cancel when backgrounded (no foreground service).

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 09.1.2 - Import Pipeline (SAF, Free Space, Stream Copy, SHA-256, Cancel)
TASK: Write ImportPipeline with fakes for storage and tests for cancel and low space.
CONTEXT TO ATTACH: Spec 6.2.
FILES TO CREATE OR MODIFY: feature/mapsources/ImportPipeline.kt
RULES: Streams only. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room, DataStore,
  kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a dependency that
  is not already in the version catalog; if one is needed, stop and say so. Store geographic data as WGS84
  lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use string
  resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or update
  unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-09.1.2. If the result fails verification: git restore . (or git reset --hard before-09.1.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Tests pass.

12. Verification procedure

Cancel leaves no files.

13. Acceptance criteria

[ ] No partial files after cancel.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 09.1.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 09.1.3 — Validation and SourceNormalizer

Provenance: [SPEC 6.2]

1. Step ID / 2. Title

09.1.3 — Validation and SourceNormalizer (type: AI-assisted coding)

3. Objective

Magic-byte detection and classification into MBTILES, PMTILES, GEOREF_IMAGE, IMAGE_LOCAL, STYLE_ONLY.

4. Why it is necessary

Spec 6 principle.

5. Prerequisites

Step 09.1.2 done and verified (tracker ticked). Phase Gate: every format in the matrix is classified Supported/Partial/Deferred with a real sample.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. SQLite header + MBTiles tables; PMTiles header; PNG/JPEG/WebP signatures; ZIP central directory.
2. Clear errors for invalid, unsupported, corrupt.
3. Unsupported formats reported with nearest alternative from Spec 6.3.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 09.1.3 - Validation and SourceNormalizer
TASK: Write SourceNormalizer with signature tests using small fixtures.
CONTEXT TO ATTACH: Spec 6.2, 6.3.
FILES TO CREATE OR MODIFY: feature/mapsources/SourceNormalizer.kt
RULES: No whole-file reads. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-09.1.3. If the result fails verification: git restore . (or git reset --hard before-09.1.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the

backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Tests pass.

12. Verification procedure

Renamed .txt as .mbtiles rejected.

13. Acceptance criteria

[ ] Each type detected.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 09.1.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STAGE 09.2 — Tile Sources

STEP 09.2.1 — PMTiles Source Support

Provenance: [SPEC 6.3]

1. Step ID / 2. Title

09.2.1 — PMTiles Source Support (type: AI-assisted coding)

3. Objective

Render PMTiles sources in projects.

4. Why it is necessary

P0.

5. Prerequisites

Step 09.1.3 done and verified (tracker ticked). Phase Gate: every format in the matrix is classified Supported/Partial/Deferred with a real sample.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Use pmtiles:// file:/// with the library path; read metadata (bounds, zoom, attribution).
2. Attribution displayed.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 09.2.1 - PMTiles Source Support
TASK: Implement PMTiles source loading and metadata parsing.
CONTEXT TO ATTACH: Spec 3.3, 6.3.
FILES TO CREATE OR MODIFY: map/PmtilesSource.kt
RULES: Read-only. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room, DataStore,
  kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a dependency that
  is not already in the version catalog; if one is needed, stop and say so. Store geographic data as WGS84
  lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use string
  resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or update
  unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-09.2.1. If the result fails verification: git restore . (or git reset --hard before-09.2.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Displays.

12. Verification procedure

Real sample offline on both devices.

13. Acceptance criteria

[ ] Evidence recorded.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 09.1.3; leads to the next step in the phase order.

17. Estimated download / storage

Sample.

STEP 09.2.2 — MBTiles Source Support (Direct or Loopback)

Provenance: [SPEC 6.3]

1. Step ID / 2. Title

09.2.2 — MBTiles Source Support (Direct or Loopback) (type: AI-assisted coding)

3. Objective

Implement the path chosen in D-R07.

4. Why it is necessary

P0.

5. Prerequisites

Step 09.2.1 done and verified (tracker ticked). Phase Gate: every format in the matrix is classified Supported/Partial/Deferred with a real sample.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Direct or loopback with random port and token; read-only SQLite.
2. Metadata table parsing; raster and vector tile detection.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 09.2.2 - MBTiles Source Support (Direct or Loopback)
TASK: Implement MBTiles source per decision D-R07.
CONTEXT TO ATTACH: Spec 6.3.
FILES TO CREATE OR MODIFY: map/MbtilesSource.kt
RULES: Loopback only if chosen; never non-loopback bind. Native Android only: Kotlin, Jetpack Compose, Material
  3, MapLibre Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via
  AppContainer. Do not add a dependency that is not already in the version catalog; if one is needed, stop and
  say so. Store geographic data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded
  user-visible strings (use string resources, English and Arabic). Touch targets at least 48 dp. Use start/end,
  never left/right. Write or update unit tests. Do not weaken or delete existing tests. Never claim something
  works unless it was built and tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-09.2.2. If the result fails verification: git restore . (or git reset --hard before-09.2.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Displays.

12. Verification procedure

Real raster and vector MBTiles samples.

13. Acceptance criteria

[ ] Both samples tested.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 09.2.1; leads to the next step in the phase order.

17. Estimated download / storage

Samples.

STEP 09.2.3 — XYZ/TMS ZIP Conversion

Provenance: [SPEC 6.3]

1. Step ID / 2. Title

09.2.3 — XYZ/TMS ZIP Conversion (type: AI-assisted coding)

3. Objective

Convert tile-folder ZIPs to MBTiles at import.

4. Why it is necessary

Matrix row P1.

5. Prerequisites

Step 09.2.2 done and verified (tracker ticked). Phase Gate: every format in the matrix is classified Supported/Partial/Deferred with a real sample.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Detect z/x/y structure; zip-slip and bomb limits; write MBTiles with correct y flip; progress and cancel.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 09.2.3 - XYZ/TMS ZIP Conversion
TASK: Write XyzZipConverter.
CONTEXT TO ATTACH: Spec 6.3.
FILES TO CREATE OR MODIFY: feature/mapsources/XyzZipConverter.kt
RULES: Streaming. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room, DataStore,
  kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a dependency that
  is not already in the version catalog; if one is needed, stop and say so. Store geographic data as WGS84
  lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use string
  resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or update
  unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-09.2.3. If the result fails verification: git restore . (or git reset --hard before-09.2.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Converted.

12. Verification procedure

MOBAC XYZ sample displays.

13. Acceptance criteria

[ ] Converted file opens.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 09.2.2; leads to the next step in the phase order.

17. Estimated download / storage

Sample.

STEP 09.2.4 — Vector Style Handling (OpenMapTiles and Generic)

Provenance: [SPEC 6.7]

1. Step ID / 2. Title

09.2.4 — Vector Style Handling (OpenMapTiles and Generic) (type: AI-assisted coding)

3. Objective

Bundled OpenMapTiles-schema style and generic fallback style.

4. Why it is necessary

Spec 6.7.

5. Prerequisites

Step 09.2.3 done and verified (tracker ticked). Phase Gate: every format in the matrix is classified Supported/Partial/Deferred with a real sample.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Generic style draws unknown source-layers by geometry type.
2. Details screen states which style is used.
3. Imported online URLs refused or rewritten.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 09.2.4 - Vector Style Handling (OpenMapTiles and Generic)
TASK: Write the two styles and selector.
CONTEXT TO ATTACH: Spec 6.7.
FILES TO CREATE OR MODIFY: assets/styles/*.json, map/StyleSelector.kt
RULES: Local resources only. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-09.2.4. If the result fails verification: git restore . (or git reset --hard before-09.2.4), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Vector sample displays.

12. Verification procedure

Vector MBTiles sample renders.

13. Acceptance criteria

[ ] Style noted in details.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 09.2.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STAGE 09.3 — Image Sources and Maps Panel

STEP 09.3.1 — IMAGE_LOCAL Canvas

Provenance: [SPEC DN-7]

1. Step ID / 2. Title

09.3.1 — IMAGE_LOCAL Canvas (type: AI-assisted coding)

3. Objective

Image-space canvas for ungeoreferenced images.

4. Why it is necessary

FR-GRF-04; DN-7.

5. Prerequisites

Step 09.2.4 done and verified (tracker ticked). Phase Gate: every format in the matrix is classified Supported/Partial/Deferred with a real sample.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Objects use image coordinates and are never labelled geographic.
2. Pan/zoom image; export keeps IMAGE_LOCAL flag.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 09.3.1 - IMAGE_LOCAL Canvas
TASK: Implement IMAGE_LOCAL canvas mode.
CONTEXT TO ATTACH: Spec FR-GRF-04.
FILES TO CREATE OR MODIFY: map/ImageLocalEngine.kt
RULES: No invented coordinates. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-09.3.1. If the result fails verification: git restore . (or git reset --hard before-09.3.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Works.

12. Verification procedure

Ordinary image gets no lat/lon.

13. Acceptance criteria

[ ] No geographic coordinates assigned.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 09.2.4; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 09.3.2 — World-file Georeferenced Images

Provenance: [SPEC 6.5]

1. Step ID / 2. Title

09.3.2 — World-file Georeferenced Images (type: AI-assisted coding)

3. Objective

Import image + .pgw/.jgw/.wld with CRS EPSG:4326/3857/UTM.

4. Why it is necessary

Matrix P1.

5. Prerequisites

Step 09.3.1 done and verified (tracker ticked). Phase Gate: every format in the matrix is classified Supported/Partial/Deferred with a real sample.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Parse six-line world file; apply affine; refuse other CRS clearly.
2. Small extents via image source; large extents later via pyramid (Phase 20).

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 09.3.2 - World-file Georeferenced Images
TASK: Write world-file parser and GEOREF_IMAGE source.
CONTEXT TO ATTACH: Spec 6.3, 6.5.
FILES TO CREATE OR MODIFY: feature/mapsources/WorldFile.kt
RULES: Honest accuracy note. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-09.3.2. If the result fails verification: git restore . (or git reset --hard before-09.3.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Works.

12. Verification procedure

Known sample aligns within expected metres.

13. Acceptance criteria

[ ] Unsupported CRS refused.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 09.3.1; leads to the next step in the phase order.

17. Estimated download / storage

Sample.

STEP 09.3.3 — Maps Panel UI (Map Sources Tab, Details, Relink)

Provenance: [SPEC BA-08]

1. Step ID / 2. Title

09.3.3 — Maps Panel UI (Map Sources Tab, Details, Relink) (type: AI-assisted coding)

3. Objective

Panel with import chooser and plain-language hints, progress rows, details and actions.

4. Why it is necessary

BA-08, BA-10.

5. Prerequisites

Step 09.3.2 done and verified (tracker ticked). Phase Gate: every format in the matrix is classified Supported/Partial/Deferred with a real sample.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Import chooser with 'Where do I find this file?' hints for AlpineQuest and Google Earth.
2. Details: bounds, zoom, CRS, tile type, attribution; Rename, Delete, Export source, Georeference.
3. Relink dialog by choice or fingerprint.
4. Empty state 'No map sources - Import map source'.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 09.3.3 - Maps Panel UI (Map Sources Tab, Details, Relink)
TASK: Build the Maps panel.
CONTEXT TO ATTACH: Guide sections 11, 13.
FILES TO CREATE OR MODIFY: feature/mapsources/MapsPanel.kt
RULES: Progress inside panel rows. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native,
  Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-09.3.3. If the result fails verification: git restore . (or git reset --hard before-09.3.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Works.

12. Verification procedure

Import real sample with progress and cancel.

13. Acceptance criteria

[ ] No full-screen spinner.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 09.3.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 09.3.4 — Format Support Matrix Evidence Run

Provenance: [SPEC AC-SRC-1]

1. Step ID / 2. Title
09.3.4 — Format Support Matrix Evidence Run (type: device test)

3. Objective
Test each format with a real sample and fill the matrix.

4. Why it is necessary
Never claim support without a sample.

5. Prerequisites
Step 09.3.3 done and verified (tracker ticked). Phase Gate: every format in the matrix is classified Supported/Partial/Deferred with a real sample.

6. Required software
Android Studio, adb, the Lenovo Tab P12 and/or Samsung Galaxy A16 (USB debugging on), Git.

7. Exact instructions
1. For every matrix row: sample file name, size, result Supported/Partial/Deferred, screenshot.
2. Cases: cancel, invalid, duplicate, low storage, in-use deletion, airplane mode.
3. Update ARCHITECTURE.md and MAPX_STATUS.

8. Terminal commands
Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat installDebug --offline
# Then run the app on the device named in the verification procedure and note the result.
```

9. Official download links
None required for this step.

10. Complete AI implementation prompt
Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 09.3.4 - Format Support Matrix Evidence Run
TASK: Create a test-evidence table template for the matrix.
CONTEXT TO ATTACH: Spec 6.3.
FILES TO CREATE OR MODIFY: Only the files needed for this step; list them before writing code.
RULES: No marks without evidence. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native,
  Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Table.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-09.3.4. If the result fails verification: git restore . (or git reset --hard before-09.3.4), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result
Matrix complete.

12. Verification procedure
Every Supported row has a sample.

13. Acceptance criteria
[ ] AC-SRC-1 evidence present.

14. Common errors
- App crashes on start -> read logcat FATAL lines.
- Works on one device only -> compare screen class (tablet vs phone) and Android version.
- Install blocked -> see 02.2.4 and 02.2.5 notes.

15. Troubleshooting
1) Run: adb devices -l and confirm 'device'. 2) Uninstall and reinstall: adb uninstall app.mapx then installDebug. 3) Capture the crash: adb logcat -d | Select-String 'FATAL|AndroidRuntime' and give the lines to the AI. 4) Retest on the other device to separate device-specific from general faults.

16. Dependencies
Depends on step 09.3.3; leads to the next step in the phase order.

17. Estimated download / storage
Samples.

PHASE 10 — EDITOR CORE: OBJECTS, DRAWING, LAYERS

Map-native rendering, selection, handles, labels, markers, the 11 drawing tools, properties and layers.
Estimated relative effort: XL. Gate: AC-GEO-1, AC-DRW-1, AC-LYR-1 pass on devices.

STAGE 10.1 — Rendering and Interaction Foundation

STEP 10.1.1 — Map-native Object Rendering

Provenance: [SPEC 7.4]

1. Step ID / 2. Title

10.1.1 — Map-native Object Rendering (type: AI-assisted coding)

3. Objective

Render objects as GeoJSON sources per layer x geometry class with line/fill/circle/symbol layers.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 09.3.4 done and verified (tracker ticked). Phase Gate: AC-GEO-1, AC-DRW-1, AC-LYR-1 pass on devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. One source per (layer x geometry class); update only changed sources.
2. RenderOrder mapped to style layers; hidden layers not rendered.
3. Layer opacity multiplies object opacity.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 10.1.1 - Map-native Object Rendering
TASK: Write the object renderer.
CONTEXT TO ATTACH: Spec 7.4.
FILES TO CREATE OR MODIFY: map/ObjectRenderer.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-10.1.1. If the result fails verification: git restore . (or git reset --hard before-10.1.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Add 1,000 objects: ~60 fps on tablet.

13. Acceptance criteria

[ ] Anchoring intrinsic.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.

- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 09.3.4; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 10.1.2 — Selection, Hit-testing and Drag-move

Provenance: [SPEC FR-OBJ-07]

1. Step ID / 2. Title

10.1.2 — Selection, Hit-testing and Drag-move (type: AI-assisted coding)

3. Objective

Tap selects, empty tap deselects, multi-select, drag moves without panning map.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 10.1.1 done and verified (tracker ticked). Phase Gate: AC-GEO-1, AC-DRW-1, AC-LYR-1 pass on devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Blue outline; hit-test via spatial index and projection.
2. One drag = one undo step.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 10.1.2 - Selection, Hit-testing and Drag-move
TASK: Write selection and drag logic.
CONTEXT TO ATTACH: FR-OBJ-07, FR-OBJ-10.
FILES TO CREATE OR MODIFY: feature/editor/Selection.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-10.1.2. If the result fails verification: git restore . (or git reset --hard before-10.1.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Select and move objects.

13. Acceptance criteria

[ ] Hidden layers not hit-testable.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 10.1.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 10.1.3 — HandleLayer

Provenance: [SPEC FR-OBJ-08]

1. Step ID / 2. Title

10.1.3 — HandleLayer (type: AI-assisted coding)

3. Objective

One reusable screen-space handle layer for vertex, endpoint, move, resize, shape, rotation handles.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 10.1.2 done and verified (tracker ticked). Phase Gate: AC-GEO-1, AC-DRW-1, AC-LYR-1 pass on devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Compose overlay inside MapWorkspace only; never stored.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 10.1.3 - HandleLayer
TASK: Write HandleLayer.
CONTEXT TO ATTACH: FR-OBJ-08.
FILES TO CREATE OR MODIFY: feature/editor/HandleLayer.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-10.1.3. If the result fails verification: git restore . (or git reset --hard before-10.1.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Handles follow object during map pan.

13. Acceptance criteria

[ ] Handles never saved (architecture test).

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 10.1.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 10.1.4 — Label Tag (Map-native Bitmaps)

Provenance: [SPEC FR-OBJ-03]

1. Step ID / 2. Title

10.1.4 — Label Tag (Map-native Bitmaps) (type: AI-assisted coding)

3. Objective

Label Tag drawn with StaticLayout into bitmaps cached in an LRU (48 MB), registered as style images.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 10.1.3 done and verified (tracker ticked). Phase Gate: AC-GEO-1, AC-DRW-1, AC-LYR-1 pass on devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. White card, bold navy #14264F, amber #F6B800 square at inline-start, pointer to exact coordinate; sizes S/M/L; bidi-safe.
2. Long-press on empty map creates a label.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 10.1.4 - Label Tag (Map-native Bitmaps)
TASK: Write LabelTagDefinition and bitmap cache.
CONTEXT TO ATTACH: FR-OBJ-03, Spec 7.4 item 2, 7.5 Path B.
FILES TO CREATE OR MODIFY: feature/editor/LabelTag.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-10.1.4. If the result fails verification: git restore . (or git reset --hard before-10.1.4), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure
300 labels at ~60 fps; Arabic label correct.

13. Acceptance criteria
[ ] Amber square on inline-start in RTL.

14. Common errors
- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting
1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies
Depends on step 10.1.3; leads to the next step in the phase order.

17. Estimated download / storage
No download.

STEP 10.1.5 — Marker Object
Provenance: [SPEC FR-OBJ-04]

1. Step ID / 2. Title
10.1.5 — Marker Object (type: AI-assisted coding)

3. Objective
Marker with position, name, icon, colour, rotation, show-name.

4. Why it is necessary
Required by the Spec area named in the tag.

5. Prerequisites
Step 10.1.4 done and verified (tracker ticked). Phase Gate: AC-GEO-1, AC-DRW-1, AC-LYR-1 pass on devices.

6. Required software
Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions
1. Register in MapObjectRegistry with renderer and editor.

8. Terminal commands
Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links
None required for this step.

10. Complete AI implementation prompt
Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 10.1.5 - Marker Object
TASK: Write Marker definition.
CONTEXT TO ATTACH: FR-OBJ-04.
FILES TO CREATE OR MODIFY: feature/editor/Marker.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-10.1.5. If the result fails verification: git restore . (or git reset --hard before-10.1.5), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Place and edit marker.

13. Acceptance criteria

[ ] Save/reload keeps marker.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 10.1.4; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STAGE 10.2 — Drawing Tools

STEP 10.2.1 — Draw Popup and Context Bar State Machine

Provenance: [SPEC FR-DRW-01]

1. Step ID / 2. Title

10.2.1 — Draw Popup and Context Bar State Machine (type: AI-assisted coding)

3. Objective

Draw button opens one ModalLayer popup with exactly the 11 tools; context bar shows tool, Finish, Cancel, Undo point and style chips.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 10.1.5 done and verified (tracker ticked). Phase Gate: AC-GEO-1, AC-DRW-1, AC-LYR-1 pass on devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Choosing a tool closes the popup; active tool in blue; Pan is separate.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 10.2.1 - Draw Popup and Context Bar State Machine
TASK: Write the draw state machine.
CONTEXT TO ATTACH: Guide section 10.
FILES TO CREATE OR MODIFY: feature/drawing/DrawStateMachine.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-10.2.1. If the result fails verification: git restore . (or git reset --hard before-10.2.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Popup lists 11 tools.

13. Acceptance criteria

[ ] Context bar never wraps.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 10.1.5; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 10.2.2 — Freehand, Line, Polyline/Route

Provenance: [SPEC 5.6]

1. Step ID / 2. Title

10.2.2 — Freehand, Line, Polyline/Route (type: AI-assisted coding)

3. Objective

Implement three tools with Douglas-Peucker and vertex cap.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 10.2.1 done and verified (tracker ticked). Phase Gate: AC-GEO-1, AC-DRW-1, AC-LYR-1 pass on devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Freehand stroke = one command.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).

Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 10.2.2 - Freehand, Line, Polyline/Route
TASK: Write the three tool definitions.
CONTEXT TO ATTACH: FR-DRW-01/02/05.
FILES TO CREATE OR MODIFY: feature/drawing/LineTools.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-10.2.2. If the result fails verification: git restore . (or git reset --hard before-10.2.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Draw each; undo once removes whole stroke.

13. Acceptance criteria

[ ] Style persists on reload.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 10.2.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 10.2.3 — Straight and Curved Arrows

Provenance: [SPEC 5.6]

1. Step ID / 2. Title

10.2.3 — Straight and Curved Arrows (type: AI-assisted coding)

3. Objective

Arrows with render-time arrowheads sized in dp and quadratic curve.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 10.2.2 done and verified (tracker ticked). Phase Gate: AC-GEO-1, AC-DRW-1, AC-LYR-1 pass on devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Curve control point is geographic.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each

prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 10.2.3 - Straight and Curved Arrows
TASK: Write arrow tools.
CONTEXT TO ATTACH: Spec 7.3.
FILES TO CREATE OR MODIFY: feature/drawing/ArrowTools.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-10.2.3. If the result fails verification: git restore . (or git reset --hard before-10.2.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Arrowheads stay constant size.

13. Acceptance criteria

[ ] Handles edit curve.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 10.2.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 10.2.4 — Circle and Rectangle

Provenance: [SPEC FR-DRW-03]

1. Step ID / 2. Title

10.2.4 — Circle and Rectangle (type: AI-assisted coding)

3. Objective

Circle by centre+radius metres; rectangle by centre, size, bearing.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 10.2.3 done and verified (tracker ticked). Phase Gate: AC-GEO-1, AC-DRW-1, AC-LYR-1 pass on devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Circle stays around its city when map pans/zooms/rotates.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 10.2.4 - Circle and Rectangle
TASK: Write circle and rectangle tools.
CONTEXT TO ATTACH: FR-DRW-03.
FILES TO CREATE OR MODIFY: feature/drawing/ShapeTools.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-10.2.4. If the result fails verification: git restore . (or git reset --hard before-10.2.4), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Circle radius shown in metric units.

13. Acceptance criteria

[ ] AC-GEO-1 circle test.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 10.2.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 10.2.5 — Polygon, Boundary, Highlighted Area

Provenance: [SPEC 5.6]

1. Step ID / 2. Title

10.2.5 — Polygon, Boundary, Highlighted Area (type: AI-assisted coding)

3. Objective

Polygon-family tools with area/perimeter/vertex count.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 10.2.4 done and verified (tracker ticked). Phase Gate: AC-GEO-1, AC-DRW-1, AC-LYR-1 pass on devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Finish closes ring; highlighted area emphasis fill.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 10.2.5 - Polygon, Boundary, Highlighted Area
TASK: Write polygon tools.
CONTEXT TO ATTACH: FR-DRW-06.
FILES TO CREATE OR MODIFY: feature/drawing/PolygonTools.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-10.2.5. If the result fails verification: git restore . (or git reset --hard before-10.2.5), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Measurements shown.

13. Acceptance criteria

[ ] Metric units only.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 10.2.4; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 10.2.6 — Route with Pins

Provenance: [SPEC 5.6]

1. Step ID / 2. Title

10.2.6 — Route with Pins (type: AI-assisted coding)

3. Objective

Ordered path plus pin objects with route length.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 10.2.5 done and verified (tracker ticked). Phase Gate: AC-GEO-1, AC-DRW-1, AC-LYR-1 pass on devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Pins reference vertices or own points.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 10.2.6 - Route with Pins
TASK: Write route-with-pins tool.
CONTEXT TO ATTACH: Spec 5.6.
FILES TO CREATE OR MODIFY: feature/drawing/RoutePins.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-10.2.6. If the result fails verification: git restore . (or git reset --hard before-10.2.6), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Works.

13. Acceptance criteria

[ ] Route length metric.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 10.2.5; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STAGE 10.3 — Properties, Layers and Undo

STEP 10.3.1 — Properties Panel

Provenance: [SPEC FR-OBJ-09]

1. Step ID / 2. Title

10.3.1 — Properties Panel (type: AI-assisted coding)

3. Objective

Reusable Properties editor in side panel and portrait sheet.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 10.2.6 done and verified (tracker ticked). Phase Gate: AC-GEO-1, AC-DRW-1, AC-LYR-1 pass on devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Name, layer, stroke/fill/opacity/width/dash, metrics, visible/locked; footer Delete/Duplicate.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 10.3.1 - Properties Panel
TASK: Write Properties panel.
CONTEXT TO ATTACH: FR-OBJ-09.
FILES TO CREATE OR MODIFY: feature/editor/PropertiesPanel.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-10.3.1. If the result fails verification: git restore . (or git reset --hard before-10.3.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Edits are commands.

13. Acceptance criteria

[ ] Same composable in both containers.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 10.2.6; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 10.3.2 — Layers Panel

Provenance: [SPEC 5.7]

1. Step ID / 2. Title

10.3.2 — Layers Panel (type: AI-assisted coding)

3. Objective

Rows with eye, lock, name, opacity, count, drag handle; fixed footer New/Delete; row and header overflow.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 10.3.1 done and verified (tracker ticked). Phase Gate: AC-GEO-1, AC-DRW-1, AC-LYR-1 pass on devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Delete populated layer asks move or delete, undoable.
2. Imported layers read-only.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 10.3.2 - Layers Panel
TASK: Write Layers panel.
CONTEXT TO ATTACH: Spec 5.7, BA-07.
FILES TO CREATE OR MODIFY: feature/layers/LayersPanel.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-10.3.2. If the result fails verification: git restore . (or git reset --hard before-10.3.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

AC-LYR-1.

13. Acceptance criteria

[ ] Default layer always exists.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 10.3.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 10.3.3 — Undo/Redo UI, Selection Actions and Multi-select

Provenance: [SPEC FR-OBJ-06]

1. Step ID / 2. Title

10.3.3 — Undo/Redo UI, Selection Actions and Multi-select (type: AI-assisted coding)

3. Objective

Top-bar undo/redo, selection context bar actions in order [n][Move to layer][Duplicate][Delete][Properties].

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 10.3.2 done and verified (tracker ticked). Phase Gate: AC-GEO-1, AC-DRW-1, AC-LYR-1 pass on devices.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Delete red with confirmation where required.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C). Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 10.3.3 - Undo/Redo UI, Selection Actions and Multi-select
TASK: Wire UI to CommandManager.
CONTEXT TO ATTACH: Guide section 10.
FILES TO CREATE OR MODIFY: feature/editor/SelectionActions.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-10.3.3. If the result fails verification: git restore . (or git reset --hard before-10.3.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Undo/redo across all tools.

13. Acceptance criteria

[ ] Direction flips in RTL.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 10.3.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 10.3.4 — Editor Process-death Restore and Anchoring Test Suite

Provenance: [SPEC AC-GEO-1]

1. Step ID / 2. Title

10.3.4 — Editor Process-death Restore and Anchoring Test Suite (type: device test)

3. Objective

Restore project, camera, active tool after kill; test anchoring for every object type.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 10.3.3 done and verified (tracker ticked). Phase Gate: AC-GEO-1, AC-DRW-1, AC-LYR-1 pass on devices.

6. Required software

Android Studio, adb, the Lenovo Tab P12 and/or Samsung Galaxy A16 (USB debugging on), Git.

7. Exact instructions

1. Instrumented tests: pan, zoom out/in, rotate, move, save/reload for each type.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat installDebug --offline
# Then run the app on the device named in the verification procedure and note the result.
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 10.3.4 - Editor Process-death Restore and Anchoring Test Suite
TASK: Write the anchoring suite.
CONTEXT TO ATTACH: AC-GEO-1, FR-PRJ-09.
FILES TO CREATE OR MODIFY: androidTest/AnchoringTest.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-10.3.4. If the result fails verification: git restore . (or git reset --hard before-10.3.4), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Objects never drift.

13. Acceptance criteria

[ ] AC-GEO-1 evidence recorded.

14. Common errors

- App crashes on start -> read logcat FATAL lines.
- Works on one device only -> compare screen class (tablet vs phone) and Android version.
- Install blocked -> see 02.2.4 and 02.2.5 notes.

15. Troubleshooting

1) Run: adb devices -l and confirm 'device'. 2) Uninstall and reinstall: adb uninstall app.mapx then installDebug. 3) Capture the crash: adb logcat -d | Select-String 'FATAL|AndroidRuntime' and give the lines to the AI. 4) Retest on the other device to separate device-specific from general faults.

16. Dependencies

Depends on step 10.3.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 10.3.5 — Stylus Support Pass

Provenance: [SPEC FR-OBJ-11]

1. Step ID / 2. Title

10.3.5 — Stylus Support Pass (type: device test)

3. Objective

Precise drawing and handles with stylus; palm rejection via platform events.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 10.3.4 done and verified (tracker ticked). Phase Gate: AC-GEO-1, AC-DRW-1, AC-LYR-1 pass on devices.

6. Required software

Android Studio, adb, the Lenovo Tab P12 and/or Samsung Galaxy A16 (USB debugging on), Git.

7. Exact instructions

1. Test with the Lenovo Tab Pen if you own one; otherwise mark Not tested.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat installDebug --offline
# Then run the app on the device named in the verification procedure and note the result.
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 10.3.5 - Stylus Support Pass
TASK: Review input handling for stylus.
CONTEXT TO ATTACH: FR-OBJ-11.
FILES TO CREATE OR MODIFY: feature/editor/Input.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-10.3.5. If the result fails verification: git restore . (or git reset --hard before-10.3.5), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Stylus draws without stray touches.

13. Acceptance criteria

[ ] Not claimed tested without a stylus.

14. Common errors

- App crashes on start -> read logcat FATAL lines.
- Works on one device only -> compare screen class (tablet vs phone) and Android version.
- Install blocked -> see 02.2.4 and 02.2.5 notes.

15. Troubleshooting

1) Run: adb devices -l and confirm 'device'. 2) Uninstall and reinstall: adb uninstall app.mapx then installDebug. 3) Capture the crash: adb logcat -d | Select-String 'FATAL|AndroidRuntime' and give the lines to the AI. 4) Retest on the other device to separate device-specific from general faults.

16. Dependencies

Depends on step 10.3.4; leads to the next step in the phase order.

17. Estimated download / storage

No download.

PHASE 11 — MEDIA, SYMBOLS AND ANIMATION

Images, GIF/animated WebP, the central AnimationScheduler and the 90+ symbol set.
Estimated relative effort: L. Gate: AC-IMG-1 passes; symbols searchable in both languages.

STAGE 11.1 — Images

STEP 11.1.1 — Media Store, Import and Decode Safeguards

Provenance: [SPEC 5.8]

1. Step ID / 2. Title

11.1.1 — Media Store, Import and Decode Safeguards (type: AI-assisted coding)

3. Objective

Import PNG/JPG/WebP into files/media with hash dedupe, EXIF, downsampling and caches.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 10.3.5 done and verified (tracker ticked). Phase Gate: AC-IMG-1 passes; symbols searchable in both languages.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Decoded cache <=128 MB; longest side <=2048 px; missing media placeholder.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 11.1.1 - Media Store, Import and Decode Safeguards
TASK: Write MediaStore and decoder.
CONTEXT TO ATTACH: FR-IMG-01/02/05.
FILES TO CREATE OR MODIFY: feature/media/MediaStore.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-11.1.1. If the result fails verification: git restore . (or git reset --hard before-11.1.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

12 MP photo imports without OOM.

13. Acceptance criteria

- [ ] Missing file shows placeholder.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.

- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 10.3.5; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 11.1.2 — Image Objects (Anchored)

Provenance: [SPEC FR-IMG-03]

1. Step ID / 2. Title

11.1.2 — Image Objects (Anchored) (type: AI-assisted coding)

3. Objective

Image object by centre, width metres, rotation, rendered map-natively.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 11.1.1 done and verified (tracker ticked). Phase Gate: AC-IMG-1 passes; symbols searchable in both languages.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. IMAGE_LOCAL images never get coordinates.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 11.1.2 - Image Objects (Anchored)
TASK: Write Image object definition.
CONTEXT TO ATTACH: FR-IMG-03.
FILES TO CREATE OR MODIFY: feature/media/ImageObject.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-11.1.2. If the result fails verification: git restore . (or git reset --hard before-11.1.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Image scales with map.

13. Acceptance criteria

[ ] Anchoring test passes.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.

- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 11.1.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 11.1.3 — Image Transform Gestures and Context Actions

Provenance: [SPEC FR-IMG-04]

1. Step ID / 2. Title

11.1.3 — Image Transform Gestures and Context Actions (type: AI-assisted coding)

3. Objective

Move, scale, rotate, flip, opacity, lock; one gesture one undo.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 11.1.2 done and verified (tracker ticked). Phase Gate: AC-IMG-1 passes; symbols searchable in both languages.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Large handles tablet, contextual sheet phone.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 11.1.3 - Image Transform Gestures and Context Actions
TASK: Write transform handling.
CONTEXT TO ATTACH: FR-IMG-04.
FILES TO CREATE OR MODIFY: feature/media/ImageTransform.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-11.1.3. If the result fails verification: git restore . (or git reset --hard before-11.1.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Works.

13. Acceptance criteria

[ ] One undo per gesture.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 11.1.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STAGE 11.2 — Animation

STEP 11.2.1 — AnimationScheduler

Provenance: [SPEC 5.9]

1. Step ID / 2. Title

11.2.1 — AnimationScheduler (type: AI-assisted coding)

3. Objective

Single clock; play, pause, restart, speed, loop, progress; cap of 6 concurrent; visible-only.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 11.1.3 done and verified (tracker ticked). Phase Gate: AC-IMG-1 passes; symbols searchable in both languages.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Fake-clock tests; pause on background.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 11.2.1 - AnimationScheduler
TASK: Write AnimationScheduler.
CONTEXT TO ATTACH: FR-ANI-02..05.
FILES TO CREATE OR MODIFY: feature/media/AnimationScheduler.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-11.2.1. If the result fails verification: git restore . (or git reset --hard before-11.2.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Timing tests pass.

13. Acceptance criteria

[ ] No loops inside composables.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 11.1.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 11.2.2 — GIF and Animated WebP Playback

Provenance: [SPEC 5.9]

1. Step ID / 2. Title

11.2.2 — GIF and Animated WebP Playback (type: AI-assisted coding)

3. Objective

Platform ImageDecoder/AnimatedImageDrawable frames pushed to the map image source at capped rate.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 11.2.1 done and verified (tracker ticked). Phase Gate: AC-IMG-1 passes; symbols searchable in both languages.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Release decoders when off-screen; honour reduced motion.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 11.2.2 - GIF and Animated WebP Playback
TASK: Write animated image playback.
CONTEXT TO ATTACH: FR-ANI-01/03/04.
FILES TO CREATE OR MODIFY: feature/media/AnimatedImage.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-11.2.2. If the result fails verification: git restore . (or git reset --hard before-11.2.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

GIF plays; off-screen stops.

13. Acceptance criteria

[ ] Cap respected.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 11.2.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STAGE 11.3 — Symbols

STEP 11.3.1 — Symbol Set and SYMBOLS_LICENSE.md

Provenance: [SPEC FR-SYM-02]

1. Step ID / 2. Title

11.3.1 — Symbol Set and SYMBOLS_LICENSE.md (type: AI-assisted coding)

3. Objective

Create 15 categories x 6+ symbols of original vector art.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 11.2.2 done and verified (tracker ticked). Phase Gate: AC-IMG-1 passes; symbols searchable in both languages.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Use only original or clearly licensed art; no copied military-standard artwork.
2. Fill SYMBOLS_LICENSE.md for every symbol.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 11.3.1 - Symbol Set and SYMBOLS_LICENSE.md
TASK: Generate original SVG-like vector path definitions for the 15 categories.
CONTEXT TO ATTACH: FR-SYM-01/02.
FILES TO CREATE OR MODIFY: assets/symbols/*, docs/SYMBOLS_LICENSE.md
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-11.3.1. If the result fails verification: git restore . (or git reset --hard before-11.3.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the

backup.ps1 copy only if the repository itself is damaged.

11. Expected result
Feature works and tests pass.

12. Verification procedure
90+ symbols listed with licence.

13. Acceptance criteria
[ ] Every symbol has a licence row.

14. Common errors
- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting
1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies
Depends on step 11.2.2; leads to the next step in the phase order.

17. Estimated download / storage
No download.

STEP 11.3.2 — Symbol Registry, SDF Rendering and Search

Provenance: [SPEC 5.10]

1. Step ID / 2. Title
11.3.2 — Symbol Registry, SDF Rendering and Search (type: AI-assisted coding)

3. Objective
Register as SDF-capable images; recolour without re-rasterising; English/Arabic search; favourites/recents.

4. Why it is necessary
Required by the Spec area named in the tag.

5. Prerequisites
Step 11.3.1 done and verified (tracker ticked). Phase Gate: AC-IMG-1 passes; symbols searchable in both languages.

6. Required software
Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions
1. Re-register on density change.

8. Terminal commands
Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links
None required for this step.

10. Complete AI implementation prompt
Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 11.3.2 - Symbol Registry, SDF Rendering and Search
TASK: Write SymbolRegistry and search.
CONTEXT TO ATTACH: FR-SYM-03/04.
FILES TO CREATE OR MODIFY: feature/symbols/SymbolRegistry.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-11.3.2. If the result fails verification: git restore . (or git reset --hard before-11.3.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Search works in both languages.

13. Acceptance criteria

[ ] Search tests pass.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 11.3.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 11.3.3 — Symbols Panel and Placement

Provenance: [SPEC 5.10]

1. Step ID / 2. Title

11.3.3 — Symbols Panel and Placement (type: AI-assisted coding)

3. Objective

Panel with categories, grid, tap symbol then map; edit move/rotate/resize/recolour/delete.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 11.3.2 done and verified (tracker ticked). Phase Gate: AC-IMG-1 passes; symbols searchable in both languages.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. 30 symbols tested world to street zoom.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 11.3.3 - Symbols Panel and Placement
TASK: Build Symbols panel.
CONTEXT TO ATTACH: FR-SYM-05.
FILES TO CREATE OR MODIFY: feature/symbols/SymbolsPanel.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-11.3.3. If the result fails verification: git restore . (or git reset --hard before-11.3.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Works.

13. Acceptance criteria

[ ] Undoable.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 11.3.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

PHASE 12 — IMPORT, EXPORT AND PORTABILITY

GIS importers, exchange exporters with fidelity reports, MapX packages, encryption, backup, Transfer Bundle and round-trip tests.

Estimated relative effort: XL. Gate: AC-EXP-1..5 pass; two-device transfer verified.

STAGE 12.1 — GIS Import

STEP 12.1.1 — GeoJSON Streaming Importer

Provenance: [SPEC 9.5]

1. Step ID / 2. Title

12.1.1 — GeoJSON Streaming Importer (type: AI-assisted coding)

3. Objective

Streaming JsonReader importer, RFC 7946, caps (100 MB, 250,000 features).

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 11.3.3 done and verified (tracker ticked). Phase Gate: AC-EXP-1..5 pass; two-device transfer verified.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. lon,lat order converted only at the boundary; other CRS reported.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 12.1.1 - GeoJSON Streaming Importer
TASK: Write GeoJsonImporter with tests.
CONTEXT TO ATTACH: Spec 9.5.
FILES TO CREATE OR MODIFY: feature/gis/GeoJsonImporter.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-12.1.1. If the result fails verification: git restore . (or git reset --hard before-12.1.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Coordinates within 1 m.

13. Acceptance criteria

[ ] Malformed file rejected safely.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.

- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 11.3.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 12.1.2 — KML/KMZ Importer (incl. GroundOverlay)

Provenance: [SPEC 9.5]

1. Step ID / 2. Title

12.1.2 — KML/KMZ Importer (incl. GroundOverlay) (type: AI-assisted coding)

3. Objective

XmlPullParser KML, ZipInputStream KMZ, GroundOverlay to image object.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 12.1.1 done and verified (tracker ticked). Phase Gate: AC-EXP-1..5 pass; two-device transfer verified.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Warn on NetworkLink, ScreenOverlay, PhotoOverlay, 3D, time spans.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 12.1.2 - KML/KMZ Importer (incl. GroundOverlay)
TASK: Write KmlImporter.
CONTEXT TO ATTACH: Spec 9.5, FR-IMG-06.
FILES TO CREATE OR MODIFY: feature/gis/KmlImporter.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-12.1.2. If the result fails verification: git restore . (or git reset --hard before-12.1.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Google Earth sample imports.

13. Acceptance criteria

[ ] Report lists ignored items.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.

- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 12.1.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 12.1.3 — GPX and CSV/TSV Importers

Provenance: [SPEC 9.5]

1. Step ID / 2. Title

12.1.3 — GPX and CSV/TSV Importers (type: AI-assisted coding)

3. Objective

GPX waypoints/routes/tracks; CSV with column mapping.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 12.1.2 done and verified (tracker ticked). Phase Gate: AC-EXP-1..5 pass; two-device transfer verified.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Auto-detect obvious lat/lon names.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 12.1.3 - GPX and CSV/TSV Importers
TASK: Write GpxImporter and CsvImporter.
CONTEXT TO ATTACH: Spec 9.5.
FILES TO CREATE OR MODIFY: feature/gis/*.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-12.1.3. If the result fails verification: git restore . (or git reset --hard before-12.1.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Samples import.

13. Acceptance criteria

[ ] No lat/lon swap.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 12.1.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 12.1.4 — GIS Import Panel and Convert to MapX Objects

Provenance: [SPEC 9.5]

1. Step ID / 2. Title

12.1.4 — GIS Import Panel and Convert to MapX Objects (type: AI-assisted coding)

3. Objective

Panel with format, CRS, counts, mini-map, warnings; imported layer read-only; Convert is undoable.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 12.1.3 done and verified (tracker ticked). Phase Gate: AC-EXP-1..5 pass; two-device transfer verified.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Imported read-only features show Read-only layer context actions.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 12.1.4 - GIS Import Panel and Convert to MapX Objects
TASK: Build GIS Import panel.
CONTEXT TO ATTACH: BA-08.
FILES TO CREATE OR MODIFY: feature/gis/GisImportPanel.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-12.1.4. If the result fails verification: git restore . (or git reset --hard before-12.1.4), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Works.

13. Acceptance criteria

[ ] Convert undoable.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 12.1.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 12.1.5 — Security Fixtures (XXE, Zip-slip, Zip-bomb)

Provenance: [SPEC 9.5]

1. Step ID / 2. Title

12.1.5 — Security Fixtures (XXE, Zip-slip, Zip-bomb) (type: AI-assisted coding)

3. Objective

Fixtures and tests for hostile files.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 12.1.4 done and verified (tracker ticked). Phase Gate: AC-EXP-1..5 pass; two-device transfer verified.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. XXE disabled; entry count, size, ratio limits.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 12.1.5 - Security Fixtures (XXE, Zip-slip, Zip-bomb)
TASK: Write hostile-file tests.
CONTEXT TO ATTACH: Spec 9.5.
FILES TO CREATE OR MODIFY: test fixtures
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-12.1.5. If the result fails verification: git restore . (or git reset --hard before-12.1.5), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

All rejected safely.

13. Acceptance criteria

[ ] No crash, no escape.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 12.1.4; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STAGE 12.2 — Exchange Export

STEP 12.2.1 — Fidelity Report Engine

Provenance: [SPEC 9.7]

1. Step ID / 2. Title

12.2.1 — Fidelity Report Engine (type: AI-assisted coding)

3. Objective

Report of exported/imported/converted/skipped items with reasons.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 12.1.5 done and verified (tracker ticked). Phase Gate: AC-EXP-1..5 pass; two-device transfer verified.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Every lossy export ends with a report.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 12.2.1 - Fidelity Report Engine
TASK: Write FidelityReport.
CONTEXT TO ATTACH: Spec 9.7.
FILES TO CREATE OR MODIFY: feature/exchange/FidelityReport.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-12.2.1. If the result fails verification: git restore . (or git reset --hard before-12.2.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Report accurate.

13. Acceptance criteria

[ ] Nothing silently dropped.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 12.1.5; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 12.2.2 — KML/KMZ, GeoJSON, GPX and CSV Exporters

Provenance: [SPEC AC-EXP-3]

1. Step ID / 2. Title

12.2.2 — KML/KMZ, GeoJSON, GPX and CSV Exporters (type: AI-assisted coding)

3. Objective

Exporters following the mapping table; options all/selected/visible layers.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 12.2.1 done and verified (tracker ticked). Phase Gate: AC-EXP-1..5 pass; two-device transfer verified.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Circle -> 72-point polygon with note; antimeridian split for GeoJSON.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 12.2.2 - KML/KMZ, GeoJSON, GPX and CSV Exporters
TASK: Write exporters.
CONTEXT TO ATTACH: Spec 9.5 mapping table.
FILES TO CREATE OR MODIFY: feature/exchange/Exporters.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-12.2.2. If the result fails verification: git restore . (or git reset --hard before-12.2.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Open in Google Earth/QGIS/spreadsheet.

13. Acceptance criteria

[ ] Re-import within 1 m.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 12.2.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STAGE 12.3 — MapX Packages

STEP 12.3.1 — PackageWriter/Reader (.mapx) with Manifest and Checksums

Provenance: [SPEC 9.2]

1. Step ID / 2. Title

12.3.1 — PackageWriter/Reader (.mapx) with Manifest and Checksums (type: AI-assisted coding)

3. Objective

ZIP package with manifest SHA-256 for every entry, staging and atomic commit, never overwrite.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 12.2.2 done and verified (tracker ticked). Phase Gate: AC-EXP-1..5 pass; two-device transfer verified.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Newer format version refused with Update MapX.
2. Map source matching by SHA-256, relink flow.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 12.3.1 - PackageWriter/Reader (.mapx) with Manifest and Checksums
TASK: Write PackageWriter and PackageReader.
CONTEXT TO ATTACH: Spec 9.2.
FILES TO CREATE OR MODIFY: feature/exchange/Package*.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-12.3.1. If the result fails verification: git restore . (or git reset --hard before-12.3.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Tamper test fails verification.

13. Acceptance criteria

[ ] AC-EXP-1.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 12.2.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 12.3.2 — Layer Package (.mapxlayer)

Provenance: [SPEC 9.3]

1. Step ID / 2. Title

12.3.2 — Layer Package (.mapxlayer) (type: AI-assisted coding)

3. Objective

Layer export/import as one undoable command with regenerated ids.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 12.3.1 done and verified (tracker ticked). Phase Gate: AC-EXP-1..5 pass; two-device transfer verified.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. IMAGE_LOCAL layers only into same image source.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).

Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 12.3.2 - Layer Package (.mapxlayer)
TASK: Write layer package support.
CONTEXT TO ATTACH: Spec 9.3.
FILES TO CREATE OR MODIFY: feature/exchange/LayerPackage.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-12.3.2. If the result fails verification: git restore . (or git reset --hard before-12.3.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

AC-EXP-2.

13. Acceptance criteria

[ ] Blocked case explained.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 12.3.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 12.3.3 — Passphrase Encryption

Provenance: [SPEC 9.2]

1. Step ID / 2. Title

12.3.3 — Passphrase Encryption (type: AI-assisted coding)

3. Objective

AES-256-GCM, PBKDF2 >=600,000 iterations, chunked nonces.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 12.3.2 done and verified (tracker ticked). Phase Gate: AC-EXP-1..5 pass; two-device transfer verified.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Wrong passphrase detected before extraction; lost passphrase unrecoverable warning.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each

prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 12.3.3 - Passphrase Encryption
TASK: Write encryption stream wrapper.
CONTEXT TO ATTACH: Spec 9.2.
FILES TO CREATE OR MODIFY: feature/exchange/Crypto.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-12.3.3. If the result fails verification: git restore . (or git reset --hard before-12.3.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

AC-EXP-5.

13. Acceptance criteria

[ ] Round trip and wrong-passphrase tests.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 12.3.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 12.3.4 — Backup and Restore (.mapxbackup)

Provenance: [SPEC 9.4]

1. Step ID / 2. Title

12.3.4 — Backup and Restore (.mapxbackup) (type: AI-assisted coding)

3. Objective

Backup all account projects, favourites, recents, settings (never password/keys); restore add or replace.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 12.3.3 done and verified (tracker ticked). Phase Gate: AC-EXP-1..5 pass; two-device transfer verified.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Replace is confirmed and atomic.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 12.3.4 - Backup and Restore (.mapxbackup)
TASK: Write backup/restore.
CONTEXT TO ATTACH: FR-EXP-01/02.
FILES TO CREATE OR MODIFY: feature/exchange/Backup.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-12.3.4. If the result fails verification: git restore . (or git reset --hard before-12.3.4), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Works.

13. Acceptance criteria

[ ] Restore verified.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 12.3.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 12.3.5 — Transfer Bundle and Map Source Export

Provenance: [SPEC DN-5]

1. Step ID / 2. Title

12.3.5 — Transfer Bundle and Map Source Export (type: AI-assisted coding)

3. Objective

Folder bundle with .mapx plus sources/ for large sources; export a source as-is.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 12.3.4 done and verified (tracker ticked). Phase Gate: AC-EXP-1..5 pass; two-device transfer verified.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Threshold about 2 GB (DN-5).

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 12.3.5 - Transfer Bundle and Map Source Export
TASK: Write Transfer Bundle.
CONTEXT TO ATTACH: DN-5.
FILES TO CREATE OR MODIFY: feature/exchange/TransferBundle.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-12.3.5. If the result fails verification: git restore . (or git reset --hard before-12.3.5), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Works.

13. Acceptance criteria

[ ] Peak heap does not grow with size.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 12.3.4; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 12.3.6 — Export/Import Wizard UI, Relink, Share Sheet, Open-with

Provenance: [SPEC BA-10]

1. Step ID / 2. Title

12.3.6 — Export/Import Wizard UI, Relink, Share Sheet, Open-with (type: AI-assisted coding)

3. Objective

Single ModalLayer wizard with stepper, progress, result report; FileProvider share; open-with filters.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 12.3.5 done and verified (tracker ticked). Phase Gate: AC-EXP-1..5 pass; two-device transfer verified.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Completion snackbar Export finished - Share.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 12.3.6 - Export/Import Wizard UI, Relink, Share Sheet, Open-with
TASK: Build wizard and intent filters.
CONTEXT TO ATTACH: BA-10, FR-EXP-03/04.
FILES TO CREATE OR MODIFY: feature/exchange/Wizard.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-12.3.6. If the result fails verification: git restore . (or git reset --hard before-12.3.6), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Share and open-with work.

13. Acceptance criteria

[ ] One dialog at a time.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 12.3.5; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 12.3.7 — Round-trip Tests and Two-device Transfer

Provenance: [SPEC AC-EXP-1]

1. Step ID / 2. Title

12.3.7 — Round-trip Tests and Two-device Transfer (type: device test)

3. Objective

Automated round-trip and manual Lenovo to Samsung transfer.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 12.3.6 done and verified (tracker ticked). Phase Gate: AC-EXP-1..5 pass; two-device transfer verified.

6. Required software

Android Studio, adb, the Lenovo Tab P12 and/or Samsung Galaxy A16 (USB debugging on), Git.

7. Exact instructions

1. Canonical JSON equal apart from regenerated ids.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat installDebug --offline
# Then run the app on the device named in the verification procedure and note the result.
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 12.3.7 - Round-trip Tests and Two-device Transfer
TASK: Write round-trip test.
CONTEXT TO ATTACH: FR-EXP-06.
FILES TO CREATE OR MODIFY: androidTest/RoundTripTest.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-12.3.7. If the result fails verification: git restore . (or git reset --hard before-12.3.7), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

AC-EXP-1 verified.

13. Acceptance criteria

[ ] Two-device evidence recorded.

14. Common errors

- App crashes on start -> read logcat FATAL lines.
- Works on one device only -> compare screen class (tablet vs phone) and Android version.
- Install blocked -> see 02.2.4 and 02.2.5 notes.

15. Troubleshooting

1) Run: adb devices -l and confirm 'device'. 2) Uninstall and reinstall: adb uninstall app.mapx then installDebug. 3) Capture the crash: adb logcat -d | Select-String 'FATAL|AndroidRuntime' and give the lines to the AI. 4) Retest on the other device to separate device-specific from general faults.

16. Dependencies

Depends on step 12.3.6; leads to the next step in the phase order.

17. Estimated download / storage

No download.

PHASE 13 — GEOREFERENCING

Manual control-point georeferencing, S8 screen, IMAGE_LOCAL conversion and large-image tile pyramids.
Estimated relative effort: M. Gate: AC-SRC-2 passes.

STAGE 13.1 — Georeferencing

STEP 13.1.1 — Affine Transform, Residuals and RMSE

Provenance: [SPEC 6.5]

1. Step ID / 2. Title

13.1.1 — Affine Transform, Residuals and RMSE (type: AI-assisted coding)

3. Objective

Least-squares affine from >=3 non-collinear points; residuals and RMSE in metres with >=4 points.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 12.3.7 done and verified (tracker ticked). Phase Gate: AC-SRC-2 passes.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Warn on poor distribution.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 13.1.1 - Affine Transform, Residuals and RMSE
TASK: Write GeoreferenceModel with tests.
CONTEXT TO ATTACH: FR-GRF-01/02.
FILES TO CREATE OR MODIFY: core/geo/Georeference.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-13.1.1. If the result fails verification: git restore . (or git reset --hard before-13.1.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Tests pass.

13. Acceptance criteria

[ ] RMSE verified on synthetic data.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 12.3.7; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 13.1.2 — S8 Georeference Screen

Provenance: [SPEC BA-09]

1. Step ID / 2. Title

13.1.2 — S8 Georeference Screen (type: AI-assisted coding)

3. Objective

Synchronised image and map views with control-point table; Accept/Adjust/Reject; context bar Add/Delete/Accept.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 13.1.1 done and verified (tracker ticked). Phase Gate: AC-SRC-2 passes.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Phone: tabs; tablet: side-by-side.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 13.1.2 - S8 Georeference Screen
TASK: Build S8.
CONTEXT TO ATTACH: BA-09, BA-16.
FILES TO CREATE OR MODIFY: feature/georef/GeoreferenceScreen.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-13.1.2. If the result fails verification: git restore . (or git reset --hard before-13.1.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Works.

13. Acceptance criteria

[ ] Layout laws respected.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 13.1.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 13.1.3 — IMAGE_LOCAL to GEOGRAPHIC Conversion Command

Provenance: [SPEC FR-GRF-03]

1. Step ID / 2. Title

13.1.3 — IMAGE_LOCAL to GEOGRAPHIC Conversion Command (type: AI-assisted coding)

3. Objective

Explicit undoable conversion through the stored transform.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 13.1.2 done and verified (tracker ticked). Phase Gate: AC-SRC-2 passes.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Store transform, points, RMSE, CRS with source.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 13.1.3 - IMAGE_LOCAL to GEOGRAPHIC Conversion Command
TASK: Write conversion command.
CONTEXT TO ATTACH: FR-GRF-03.
FILES TO CREATE OR MODIFY: core/command/ConvertCoordinateSpace.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-13.1.3. If the result fails verification: git restore . (or git reset --hard before-13.1.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Undo restores.

13. Acceptance criteria

[ ] AC-SRC-2.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.

- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 13.1.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 13.1.4 — Tile Pyramid for Large Georeferenced Images (P2)

Provenance: [SPEC FR-GRF-05]

1. Step ID / 2. Title

13.1.4 — Tile Pyramid for Large Georeferenced Images (P2) (type: AI-assisted coding)

3. Objective

Streaming region decode to MBTiles for extents over ~1 degree or images over 4096 px.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 13.1.3 done and verified (tracker ticked). Phase Gate: AC-SRC-2 passes.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Document limits.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 13.1.4 - Tile Pyramid for Large Georeferenced Images (P2)
TASK: Write pyramid generator.
CONTEXT TO ATTACH: FR-GRF-05.
FILES TO CREATE OR MODIFY: feature/georef/TilePyramid.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-13.1.4. If the result fails verification: git restore . (or git reset --hard before-13.1.4), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Large sample stays within memory.

13. Acceptance criteria

[ ] Distortion bounded.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.

- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 13.1.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

PHASE 14 — SCENES, PRESENTER MODE AND BROADCAST GRAPHICS

Scenes, PresentationController with session overrides, clean presenter mode, graphics zones Z1-Z5, transitions, preview, sample briefing.

Estimated relative effort: XL. Gate: AC-PRS-1 passes including byte-identical document test.

STAGE 14.1 — Scenes

STEP 14.1.1 — Scene Commands and Scenes Panel

Provenance: [SPEC 5.11]

1. Step ID / 2. Title

14.1.1 — Scene Commands and Scenes Panel (type: AI-assisted coding)

3. Objective

Create from view, update, rename, reorder, delete, go to; Scenes panel with thumbnails.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 13.1.4 done and verified (tracker ticked). Phase Gate: AC-PRS-1 passes including byte-identical document test.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Scene is a view state referencing data.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 14.1.1 - Scene Commands and Scenes Panel
TASK: Build scenes.
CONTEXT TO ATTACH: FR-SCN-01..03.
FILES TO CREATE OR MODIFY: feature/scenes/*.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-14.1.1. If the result fails verification: git restore . (or git reset --hard before-14.1.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Works.

13. Acceptance criteria

[ ] All undoable.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.

- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 13.1.4; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 14.1.2 — PresentationController and Session Overrides

Provenance: [SPEC 5.12]

1. Step ID / 2. Title

14.1.2 — PresentationController and Session Overrides (type: AI-assisted coding)

3. Objective

Own presentation state; remote and presenter overrides held in memory only.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 14.1.1 done and verified (tracker ticked). Phase Gate: AC-PRS-1 passes including byte-identical document test.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Auto-lock exemption wired.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 14.1.2 - PresentationController and Session Overrides
TASK: Write PresentationController with tests.
CONTEXT TO ATTACH: FR-PRS-01, 09, 10.
FILES TO CREATE OR MODIFY: feature/presentation/PresentationController.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-14.1.2. If the result fails verification: git restore . (or git reset --hard before-14.1.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Tests pass.

13. Acceptance criteria

[ ] Overrides never saved.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.

- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 14.1.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STAGE 14.2 — Presenter Mode and Graphics

STEP 14.2.1 — Presenter Mode Screen

Provenance: [SPEC 5.12]

1. Step ID / 2. Title

14.2.1 — Presenter Mode Screen (type: AI-assisted coding)

3. Objective

Clean full-screen, immersive bars, keep screen on, transient controls (4 s), accidental-exit protection.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 14.1.2 done and verified (tracker ticked). Phase Gate: AC-PRS-1 passes including byte-identical document test.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Restore exact editor state on exit.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 14.2.1 - Presenter Mode Screen
TASK: Build presenter mode.
CONTEXT TO ATTACH: FR-PRS-01..04.
FILES TO CREATE OR MODIFY: feature/presentation/PresenterScreen.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-14.2.1. If the result fails verification: git restore . (or git reset --hard before-14.2.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Works.

13. Acceptance criteria

[ ] No editor UI.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 14.1.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 14.2.2 — Graphics Zones Z1-Z5 Renderer

Provenance: [SPEC 5.12]

1. Step ID / 2. Title

14.2.2 — Graphics Zones Z1-Z5 Renderer (type: AI-assisted coding)

3. Objective

Headline, legend, info card, lower third, scale, north, source within title-safe 5% and action-safe 3.5%.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 14.2.1 done and verified (tracker ticked). Phase Gate: AC-PRS-1 passes including byte-identical document test.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Percent-based zone sizes from Blueprint; RTL swaps Z2/Z3.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 14.2.2 - Graphics Zones Z1-Z5 Renderer
TASK: Write zone renderer.
CONTEXT TO ATTACH: Guide section 16.
FILES TO CREATE OR MODIFY: feature/presentation/Zones.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-14.2.2. If the result fails verification: git restore . (or git reset --hard before-14.2.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Zones never collide.

13. Acceptance criteria

[ ] FR-PRS-05/06/11.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 14.2.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 14.2.3 — Scene Graphics Panel and Transitions

Provenance: [SPEC 5.12]

1. Step ID / 2. Title

14.2.3 — Scene Graphics Panel and Transitions (type: AI-assisted coding)

3. Objective

Per-scene toggles/editors; transitions instant, fly-to, fade, object appearance; Show my location toggle default off.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 14.2.2 done and verified (tracker ticked). Phase Gate: AC-PRS-1 passes including byte-identical document test.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Reduced motion honoured.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 14.2.3 - Scene Graphics Panel and Transitions
TASK: Build Scene Graphics panel and transitions.
CONTEXT TO ATTACH: FR-PRS-07, BA-13.
FILES TO CREATE OR MODIFY: feature/scenes/SceneGraphics.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-14.2.3. If the result fails verification: git restore . (or git reset --hard before-14.2.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Works.

13. Acceptance criteria

[ ] All four transitions.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 14.2.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 14.2.4 — In-editor Preview and Audience Preview Window

Provenance: [SPEC FR-PRS-08]

1. Step ID / 2. Title

14.2.4 — In-editor Preview and Audience Preview Window (type: AI-assisted coding)

3. Objective

Preview exactly as audience sees it; draggable Audience Preview.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 14.2.3 done and verified (tracker ticked). Phase Gate: AC-PRS-1 passes including byte-identical document test.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Preview matches output.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 14.2.4 - In-editor Preview and Audience Preview Window
TASK: Build preview.
CONTEXT TO ATTACH: FR-PRS-08.
FILES TO CREATE OR MODIFY: feature/presentation/Preview.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-14.2.4. If the result fails verification: git restore . (or git reset --hard before-14.2.4), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure
Works.

13. Acceptance criteria
[ ] Matches audience render.

14. Common errors
- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting
1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies
Depends on step 14.2.3; leads to the next step in the phase order.

17. Estimated download / storage
No download.

STEP 14.2.5 — Built-in Sample Briefing
Provenance: [SPEC FR-SCN-04]

1. Step ID / 2. Title
14.2.5 — Built-in Sample Briefing (type: AI-assisted coding)

3. Objective
Sample project with >=6 scenes, layers, symbols, images, drawings, an animation, all graphics and transitions.

4. Why it is necessary
Required by the Spec area named in the tag.

5. Prerequisites
Step 14.2.4 done and verified (tracker ticked). Phase Gate: AC-PRS-1 passes including byte-identical document test.

6. Required software
Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions
1. Used for validation and offline guard runs.

8. Terminal commands
Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links
None required for this step.

10. Complete AI implementation prompt
Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 14.2.5 - Built-in Sample Briefing
TASK: Generate sample briefing JSON.
CONTEXT TO ATTACH: FR-SCN-04.
FILES TO CREATE OR MODIFY: assets/sample/briefing.mapx
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-14.2.5. If the result fails verification: git restore . (or git reset --hard before-14.2.5), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Opens and presents.

13. Acceptance criteria

[ ] All graphic types present.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 14.2.4; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 14.2.6 — Presenting-Never-Mutates Test

Provenance: [SPEC AC-PRS-1]

1. Step ID / 2. Title

14.2.6 — Presenting-Never-Mutates Test (type: device test)

3. Objective

Compare project document bytes before and after a presentation.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 14.2.5 done and verified (tracker ticked). Phase Gate: AC-PRS-1 passes including byte-identical document test.

6. Required software

Android Studio, adb, the Lenovo Tab P12 and/or Samsung Galaxy A16 (USB debugging on), Git.

7. Exact instructions

1. Include remote overrides later.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat installDebug --offline
# Then run the app on the device named in the verification procedure and note the result.
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 14.2.6 - Presenting-Never-Mutates Test
TASK: Write byte-identical test.
CONTEXT TO ATTACH: AC-PRS-1.
FILES TO CREATE OR MODIFY: androidTest/PresentationImmutabilityTest.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-14.2.6. If the result fails verification: git restore . (or git reset --hard before-14.2.6), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Identical.

13. Acceptance criteria

[ ] BLOCKER if differs.

14. Common errors

- App crashes on start -> read logcat FATAL lines.
- Works on one device only -> compare screen class (tablet vs phone) and Android version.
- Install blocked -> see 02.2.4 and 02.2.5 notes.

15. Troubleshooting

1) Run: adb devices -l and confirm 'device'. 2) Uninstall and reinstall: adb uninstall app.mapx then installDebug. 3) Capture the crash: adb logcat -d | Select-String 'FATAL|AndroidRuntime' and give the lines to the AI. 4) Retest on the other device to separate device-specific from general faults.

16. Dependencies

Depends on step 14.2.5; leads to the next step in the phase order.

17. Estimated download / storage

No download.

PHASE 15 — EXTERNAL DISPLAY

DisplayController, second MapView in a Presentation window, failure handling and honest hardware evidence.
Estimated relative effort: M. Gate: AC-DSP-1 with simulated or real evidence labelled honestly.

STAGE 15.1 — External Display

STEP 15.1.1 — DisplayController with Fake DisplayManager Tests

Provenance: [SPEC 5.13]

1. Step ID / 2. Title

15.1.1 — DisplayController with Fake DisplayManager Tests (type: AI-assisted coding)

3. Objective

Abstract DisplayManager; handle added/removed/changed.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 14.2.6 done and verified (tracker ticked). Phase Gate: AC-DSP-1 with simulated or real evidence labelled honestly.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Tests with a fake.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 15.1.1 - DisplayController with Fake DisplayManager Tests
TASK: Write DisplayController.
CONTEXT TO ATTACH: FR-DSP-02.
FILES TO CREATE OR MODIFY: feature/display/DisplayController.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-15.1.1. If the result fails verification: git restore . (or git reset --hard before-15.1.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Tests pass.

13. Acceptance criteria

[ ] Lifecycle covered.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 14.2.6; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 15.1.2 — Presentation Window with Second MapView

Provenance: [SPEC 5.13]

1. Step ID / 2. Title

15.1.2 — Presentation Window with Second MapView (type: AI-assisted coding)

3. Objective

android.app.Presentation with its own MapView driven by shared state; no editor UI, no FLAG_SECURE.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 15.1.1 done and verified (tracker ticked). Phase Gate: AC-DSP-1 with simulated or real evidence labelled honestly.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Memory cost documented.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 15.1.2 - Presentation Window with Second MapView
TASK: Write audience presentation.
CONTEXT TO ATTACH: FR-DSP-03/04.
FILES TO CREATE OR MODIFY: feature/display/AudiencePresentation.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-15.1.2. If the result fails verification: git restore . (or git reset --hard before-15.1.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Works on simulated display.

13. Acceptance criteria

[ ] Hidden layers never leak.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 15.1.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 15.1.3 — LIVE Chip, Stop and Failure Handling

Provenance: [SPEC 5.13]

1. Step ID / 2. Title

15.1.3 — LIVE Chip, Stop and Failure Handling (type: AI-assisted coding)

3. Objective

Red LIVE chip with Stop; cable-removed snackbar Reconnect/Use in-app preview; mirroring-only disclosure.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 15.1.2 done and verified (tracker ticked). Phase Gate: AC-DSP-1 with simulated or real evidence labelled honestly.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Resolution change relayouts by percentage.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 15.1.3 - LIVE Chip, Stop and Failure Handling
TASK: Build live UI and failures.
CONTEXT TO ATTACH: FR-DSP-05/06.
FILES TO CREATE OR MODIFY: feature/display/LiveUi.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-15.1.3. If the result fails verification: git restore . (or git reset --hard before-15.1.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Works.

13. Acceptance criteria

[ ] Project never corrupted.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.

- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 15.1.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 15.1.4 — Hardware Evidence Run

Provenance: [SPEC AC-DSP-1]

1. Step ID / 2. Title

15.1.4 — Hardware Evidence Run (type: device test)

3. Objective

Test simulated display, and wireless display if available; run 30 minutes.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 15.1.3 done and verified (tracker ticked). Phase Gate: AC-DSP-1 with simulated or real evidence labelled honestly.

6. Required software

Android Studio, adb, the Lenovo Tab P12 and/or Samsung Galaxy A16 (USB debugging on), Git.

7. Exact instructions

1. Label result Hardware verified, Simulated only or Not tested.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat installDebug --offline
# Then run the app on the device named in the verification procedure and note the result.
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 15.1.4 - Hardware Evidence Run
TASK: Create evidence table.
CONTEXT TO ATTACH: AC-DSP-1.
FILES TO CREATE OR MODIFY: docs/MAPX_STATUS.md
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-15.1.4. If the result fails verification: git restore . (or git reset --hard before-15.1.4), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Recorded.

13. Acceptance criteria

[ ] No false hardware claim.

14. Common errors

- App crashes on start -> read logcat FATAL lines.
- Works on one device only -> compare screen class (tablet vs phone) and Android version.

- Install blocked -> see 02.2.4 and 02.2.5 notes.

15. Troubleshooting

1) Run: adb devices -l and confirm 'device'. 2) Uninstall and reinstall: adb uninstall app.mapx then installDebug. 3) Capture the crash: adb logcat -d | Select-String 'FATAL|AndroidRuntime' and give the lines to the AI. 4) Retest on the other device to separate device-specific from general faults.

16. Dependencies

Depends on step 15.1.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

PHASE 16 — PHONE REMOTE

RemoteTransport, allow-list policy, TLS companion host, pairing, phone controller, scanning and two-device validation.
Estimated relative effort: L. Gate: AC-REM-1 passes between Lenovo and Samsung.

STAGE 16.1 — Protocol and Host

STEP 16.1.1 — RemoteTransport, RemoteCommandPolicy and Protocol Tests

Provenance: [SPEC 10.5]

1. Step ID / 2. Title

16.1.1 — RemoteTransport, RemoteCommandPolicy and Protocol Tests (type: AI-assisted coding)

3. Objective

Message format, allow-list policy, nonce/counter, expiry, rate limit, size limit.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 15.1.4 done and verified (tracker ticked). Phase Gate: AC-REM-1 passes between Lenovo and Samsung.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Allow-list exactly per FR-REM-02; every other command rejected and logged.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 16.1.1 - RemoteTransport, RemoteCommandPolicy and Protocol Tests
TASK: Write protocol and policy with tests.
CONTEXT TO ATTACH: FR-REM-02/03, 10.5.
FILES TO CREATE OR MODIFY: feature/companion/Protocol.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-16.1.1. If the result fails verification: git restore . (or git reset --hard before-16.1.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Tests pass.

13. Acceptance criteria

[ ] Every forbidden command rejected.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 15.1.4; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 16.1.2 — CompanionHost Server (TLS)

Provenance: [SPEC 10.5]

1. Step ID / 2. Title

16.1.2 — CompanionHost Server (TLS) (type: AI-assisted coding)

3. Objective

Server on random port, only while Remote control is on and app foreground; no file serving.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 16.1.1 done and verified (tracker ticked). Phase Gate: AC-REM-1 passes between Lenovo and Samsung.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Locking ends sessions.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 16.1.2 - CompanionHost Server (TLS)
TASK: Write CompanionHost per spike.
CONTEXT TO ATTACH: 10.5.
FILES TO CREATE OR MODIFY: feature/companion/CompanionHost.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-16.1.2. If the result fails verification: git restore . (or git reset --hard before-16.1.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Works.

13. Acceptance criteria

[ ] Server off by default.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 16.1.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 16.1.3 — Pairing: QR, Manual Code, Approval, Session

Provenance: [SPEC 10.5]

1. Step ID / 2. Title

16.1.3 — Pairing: QR, Manual Code, Approval, Session (type: AI-assisted coding)

3. Objective

QR with host, port, fingerprint, one-time secret (2 min); approval dialog with device name; 8 h session; revoke; takeover.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 16.1.2 done and verified (tracker ticked). Phase Gate: AC-REM-1 passes between Lenovo and Samsung.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. QR encoding with ZXing-core.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 16.1.3 - Pairing: QR, Manual Code, Approval, Session
TASK: Build pairing.
CONTEXT TO ATTACH: FR-REM-07/08, BA-14, BA-17.
FILES TO CREATE OR MODIFY: feature/companion/Pairing.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-16.1.3. If the result fails verification: git restore . (or git reset --hard before-16.1.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Works.

13. Acceptance criteria

[ ] Secret single-use.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.

- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 16.1.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STAGE 16.2 — Phone Side

STEP 16.2.1 — Remote Client and Controller Screen

Provenance: [SPEC 5.14]

1. Step ID / 2. Title

16.2.1 — Remote Client and Controller Screen (type: AI-assisted coding)

3. Objective

Single scrolling column; landscape two columns; disabled-not-hidden states; no undo/redo/tool.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 16.1.3 done and verified (tracker ticked). Phase Gate: AC-REM-1 passes between Lenovo and Samsung.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. States Connected green, Reconnecting amber, Disconnected grey, Expired red.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 16.2.1 - Remote Client and Controller Screen
TASK: Build S6.
CONTEXT TO ATTACH: FR-REM-05/06, BA-03.
FILES TO CREATE OR MODIFY: feature/companion/RemoteScreen.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-16.2.1. If the result fails verification: git restore . (or git reset --hard before-16.2.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Works.

13. Acceptance criteria

[ ] Controls disabled when disconnected.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 16.1.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 16.2.2 — CameraX and ZXing QR Scanning

Provenance: [SPEC 10.5]

1. Step ID / 2. Title

16.2.2 — CameraX and ZXing QR Scanning (type: AI-assisted coding)

3. Objective

Camera permission requested only on scan screen after explanation; manual code fallback.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 16.2.1 done and verified (tracker ticked). Phase Gate: AC-REM-1 passes between Lenovo and Samsung.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. No ML Kit.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 16.2.2 - CameraX and ZXing QR Scanning
TASK: Build scanner.
CONTEXT TO ATTACH: 10.5.
FILES TO CREATE OR MODIFY: feature/companion/QrScanner.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-16.2.2. If the result fails verification: git restore . (or git reset --hard before-16.2.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Scans tablet QR.

13. Acceptance criteria

[ ] Manual entry works.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 16.2.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 16.2.3 — Session Overrides Wiring and Reset View

Provenance: [SPEC FR-REM-04]

1. Step ID / 2. Title

16.2.3 — Session Overrides Wiring and Reset View (type: AI-assisted coding)

3. Objective

Remote pan/zoom, layer toggles, reset view as session overrides in PresentationController.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 16.2.2 done and verified (tracker ticked). Phase Gate: AC-REM-1 passes between Lenovo and Samsung.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Discarded on scene change or exit.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 16.2.3 - Session Overrides Wiring and Reset View
TASK: Wire remote commands.
CONTEXT TO ATTACH: FR-REM-04.
FILES TO CREATE OR MODIFY: feature/companion/RemoteExecutor.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-16.2.3. If the result fails verification: git restore . (or git reset --hard before-16.2.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Works.

13. Acceptance criteria

[ ] Document unchanged.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 16.2.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 16.2.4 — Two-device Validation

Provenance: [SPEC AC-REM-1]

1. Step ID / 2. Title

16.2.4 — Two-device Validation (type: device test)

3. Objective

Run AC-REM-1: pairing approval, allow-list, forbidden rejection, replay, expiry, takeover, latency <300 ms.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 16.2.3 done and verified (tracker ticked). Phase Gate: AC-REM-1 passes between Lenovo and Samsung.

6. Required software

Android Studio, adb, the Lenovo Tab P12 and/or Samsung Galaxy A16 (USB debugging on), Git.

7. Exact instructions

1. Test on same Wi-Fi and phone hotspot; guest Wi-Fi isolation help message.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat installDebug --offline
# Then run the app on the device named in the verification procedure and note the result.
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 16.2.4 - Two-device Validation
TASK: Create AC-REM-1 test script.
CONTEXT TO ATTACH: AC-REM-1.
FILES TO CREATE OR MODIFY: docs/test/remote.md
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-16.2.4. If the result fails verification: git restore . (or git reset --hard before-16.2.4), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Evidence recorded.

13. Acceptance criteria

[ ] Latency measured.

14. Common errors

- App crashes on start -> read logcat FATAL lines.
- Works on one device only -> compare screen class (tablet vs phone) and Android version.
- Install blocked -> see 02.2.4 and 02.2.5 notes.

15. Troubleshooting

1) Run: adb devices -l and confirm 'device'. 2) Uninstall and reinstall: adb uninstall app.mapx then installDebug. 3) Capture the crash: adb logcat -d | Select-String 'FATAL|AndroidRuntime' and give the lines to the AI. 4) Retest on the other device to separate device-specific from general faults.

16. Dependencies

Depends on step 16.2.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

PHASE 17 — GPS LOCATION

LocationProvider, Locate control states, follow mode, drop marker and device test.
Estimated relative effort: S. Gate: AC-GPS-1 verified on the GNSS device.

STAGE 17.1 — Location

STEP 17.1.1 — LocationProvider and Permission Flow

Provenance: [SPEC 5.3]

1. Step ID / 2. Title

17.1.1 — LocationProvider and Permission Flow (type: AI-assisted coding)

3. Objective

LocationManager only; request on tap after one-line explanation; foreground only.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 16.2.4 done and verified (tracker ticked). Phase Gate: AC-GPS-1 verified on the GNSS device.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Add location permissions and uses-feature required=false now.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 17.1.1 - LocationProvider and Permission Flow
TASK: Write LocationProvider.
CONTEXT TO ATTACH: FR-GPS-01..03.
FILES TO CREATE OR MODIFY: feature/location/LocationProvider.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-17.1.1. If the result fails verification: git restore . (or git reset --hard before-17.1.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Works.

13. Acceptance criteria

[ ] No Play Services.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 16.2.4; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 17.1.2 — Locate Control States, Follow and Drop Marker

Provenance: [SPEC 5.3]

1. Step ID / 2. Title

17.1.2 — Locate Control States, Follow and Drop Marker (type: AI-assisted coding)

3. Objective

Idle, searching, located, follow, denied/off, no GNSS; dot and accuracy circle; Drop marker command.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 17.1.1 done and verified (tracker ticked). Phase Gate: AC-GPS-1 verified on the GNSS device.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Presenter hides dot unless enabled.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 17.1.2 - Locate Control States, Follow and Drop Marker
TASK: Build Locate UI.
CONTEXT TO ATTACH: FR-GPS-04..08, BA-06.
FILES TO CREATE OR MODIFY: feature/location/LocateControl.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-17.1.2. If the result fails verification: git restore . (or git reset --hard before-17.1.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Works.

13. Acceptance criteria

[ ] Location never stored.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 17.1.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 17.1.3 — GNSS Device Test

Provenance: [SPEC AC-GPS-1]

1. Step ID / 2. Title

17.1.3 — GNSS Device Test (type: device test)

3. Objective

Test outdoors on the designated device including airplane mode.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 17.1.2 done and verified (tracker ticked). Phase Gate: AC-GPS-1 verified on the GNSS device.

6. Required software

Android Studio, adb, the Lenovo Tab P12 and/or Samsung Galaxy A16 (USB debugging on), Git.

7. Exact instructions

1. Mark Not tested if no fix.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat installDebug --offline
# Then run the app on the device named in the verification procedure and note the result.
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 17.1.3 - GNSS Device Test
TASK: Create test script.
CONTEXT TO ATTACH: AC-GPS-1.
FILES TO CREATE OR MODIFY: docs/test/gps.md
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-17.1.3. If the result fails verification: git restore . (or git reset --hard before-17.1.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Recorded.

13. Acceptance criteria

[ ] Honest evidence.

14. Common errors

- App crashes on start -> read logcat FATAL lines.
- Works on one device only -> compare screen class (tablet vs phone) and Android version.
- Install blocked -> see 02.2.4 and 02.2.5 notes.

15. Troubleshooting

1) Run: adb devices -l and confirm 'device'. 2) Uninstall and reinstall: adb uninstall app.mapx then installDebug. 3) Capture the crash: adb logcat -d | Select-String 'FATAL|AndroidRuntime' and give the lines to the AI. 4) Retest on the other device to separate device-specific from general faults.

16. Dependencies

Depends on step 17.1.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

PHASE 18 — IMAGE AND PDF EXPORT

PNG, PDF and optional JPEG/WebP export using the engine snapshot and platform PDF.
Estimated relative effort: M. Gate: AC-EXP-4 passes.

STAGE 18.1 — Image and PDF Export

STEP 18.1.1 — PNG Export Engine

Provenance: [SPEC 9.6]

1. Step ID / 2. Title

18.1.1 — PNG Export Engine (type: AI-assisted coding)

3. Objective

Current view, scene, all scenes ZIP; <=8192 px with memory check; attribution always included; no hidden layers or editor UI.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 17.1.3 done and verified (tracker ticked). Phase Gate: AC-EXP-4 passes.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Use offscreen snapshot.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 18.1.1 - PNG Export Engine
TASK: Write PNG exporter.
CONTEXT TO ATTACH: FR-IMX-01/02.
FILES TO CREATE OR MODIFY: feature/exchange/PngExport.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-18.1.1. If the result fails verification: git restore . (or git reset --hard before-18.1.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

4096 px in <=10 s.

13. Acceptance criteria

[ ] Matches screen.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 17.1.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 18.1.2 — PDF Export and JPEG/WebP

Provenance: [SPEC 9.6]

1. Step ID / 2. Title

18.1.2 — PDF Export and JPEG/WebP (type: AI-assisted coding)

3. Objective

PdfDocument pages per scene, A4/A3/Letter/custom, 150/300 dpi, real text with correct Arabic shaping.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 18.1.1 done and verified (tracker ticked). Phase Gate: AC-EXP-4 passes.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Cancelable background export.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 18.1.2 - PDF Export and JPEG/WebP
TASK: Write PDF exporter.
CONTEXT TO ATTACH: FR-IMX-03/04/05.
FILES TO CREATE OR MODIFY: feature/exchange/PdfExport.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-18.1.2. If the result fails verification: git restore . (or git reset --hard before-18.1.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

10 scenes in <=60 s.

13. Acceptance criteria

[ ] Arabic shaped.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.

- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 18.1.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

PHASE 19 — LOCALIZATION, ACCESSIBILITY AND POLISH

Full Arabic/RTL audit, pseudo-locales, accessibility, Settings S7 and empty/error states.
Estimated relative effort: M. Gate: AC-I18N-1 and accessibility checks pass.

STAGE 19.1 — Localization and Accessibility

STEP 19.1.1 — Settings S7 Completion

Provenance: [SPEC 5.15]

1. Step ID / 2. Title
19.1.1 — Settings S7 Completion (type: AI-assisted coding)

3. Objective
All FR-SET-02 sections; phone single column; tablet list+detail.

4. Why it is necessary
Required by the Spec area named in the tag.

5. Prerequisites
Step 18.1.2 done and verified (tracker ticked). Phase Gate: AC-I18N-1 and accessibility checks pass.

6. Required software
Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions
1. Language, coordinates, digits, security, storage, backup, offline guard, about and licences.

8. Terminal commands
Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links
None required for this step.

10. Complete AI implementation prompt
Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 19.1.1 - Settings S7 Completion
TASK: Build Settings.
CONTEXT TO ATTACH: FR-SET-02, BA-12.
FILES TO CREATE OR MODIFY: feature/settings/*.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-19.1.1. If the result fails verification: git restore . (or git reset --hard before-19.1.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result
Feature works and tests pass.

12. Verification procedure
Works.

13. Acceptance criteria
[ ] Metric units fixed.

14. Common errors
- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.

- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 18.1.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 19.1.2 — Full Arabic Translation and RTL Audit

Provenance: [SPEC AC-I18N-1]

1. Step ID / 2. Title

19.1.2 — Full Arabic Translation and RTL Audit (type: device test)

3. Objective

Translate all strings; native reviewer; test all screens at ten stress sizes.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 19.1.1 done and verified (tracker ticked). Phase Gate: AC-I18N-1 and accessibility checks pass.

6. Required software

Android Studio, adb, the Lenovo Tab P12 and/or Samsung Galaxy A16 (USB debugging on), Git.

7. Exact instructions

1. Pseudo-locales en-XA, ar-XB; 40% expansion; mixed bidi coordinates.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat installDebug --offline
# Then run the app on the device named in the verification procedure and note the result.
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 19.1.2 - Full Arabic Translation and RTL Audit
TASK: Translate strings.
CONTEXT TO ATTACH: AC-I18N-1.
FILES TO CREATE OR MODIFY: res/values-ar/strings.xml
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-19.1.2. If the result fails verification: git restore . (or git reset --hard before-19.1.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

All screens correct.

13. Acceptance criteria

[ ] Hard-coded string scan passes.

14. Common errors

- App crashes on start -> read logcat FATAL lines.
- Works on one device only -> compare screen class (tablet vs phone) and Android version.

- Install blocked -> see 02.2.4 and 02.2.5 notes.

15. Troubleshooting

1) Run: adb devices -l and confirm 'device'. 2) Uninstall and reinstall: adb uninstall app.mapx then installDebug. 3) Capture the crash: adb logcat -d | Select-String 'FATAL|AndroidRuntime' and give the lines to the AI. 4) Retest on the other device to separate device-specific from general faults.

16. Dependencies

Depends on step 19.1.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 19.1.3 — Accessibility Pass

Provenance: [SPEC 5.16]

1. Step ID / 2. Title

19.1.3 — Accessibility Pass (type: device test)

3. Objective

TalkBack descriptions and focus order, 48 dp, contrast, 200% font, reduced motion.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 19.1.2 done and verified (tracker ticked). Phase Gate: AC-I18N-1 and accessibility checks pass.

6. Required software

Android Studio, adb, the Lenovo Tab P12 and/or Samsung Galaxy A16 (USB debugging on), Git.

7. Exact instructions

1. List-view alternative is P2 (Phase 20).

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat installDebug --offline
# Then run the app on the device named in the verification procedure and note the result.
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 19.1.3 - Accessibility Pass
TASK: Review accessibility issues.
CONTEXT TO ATTACH: FR-ACC-01/03.
FILES TO CREATE OR MODIFY: ui/**
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-19.1.3. If the result fails verification: git restore . (or git reset --hard before-19.1.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Passes.

13. Acceptance criteria

[ ] No colour-only state.

14. Common errors

- App crashes on start -> read logcat FATAL lines.
- Works on one device only -> compare screen class (tablet vs phone) and Android version.
- Install blocked -> see 02.2.4 and 02.2.5 notes.

15. Troubleshooting

1) Run: adb devices -l and confirm 'device'. 2) Uninstall and reinstall: adb uninstall app.mapx then installDebug. 3) Capture the crash: adb logcat -d | Select-String 'FATAL|AndroidRuntime' and give the lines to the AI. 4) Retest on the other device to separate device-specific from general faults.

16. Dependencies

Depends on step 19.1.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 19.1.4 — Empty, Loading and Error State Audit

Provenance: [SPEC 13.3]

1. Step ID / 2. Title

19.1.4 — Empty, Loading and Error State Audit (type: device test)

3. Objective

Verify all states in Guide section 18.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 19.1.3 done and verified (tracker ticked). Phase Gate: AC-I18N-1 and accessibility checks pass.

6. Required software

Android Studio, adb, the Lenovo Tab P12 and/or Samsung Galaxy A16 (USB debugging on), Git.

7. Exact instructions

1. Never a white screen.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat installDebug --offline
# Then run the app on the device named in the verification procedure and note the result.
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 19.1.4 - Empty, Loading and Error State Audit
TASK: Audit states.
CONTEXT TO ATTACH: Guide section 18.
FILES TO CREATE OR MODIFY: ui/**
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-19.1.4. If the result fails verification: git restore . (or git reset --hard before-19.1.4), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Complete.

13. Acceptance criteria

[ ] All table rows implemented.

14. Common errors

- App crashes on start -> read logcat FATAL lines.
- Works on one device only -> compare screen class (tablet vs phone) and Android version.
- Install blocked -> see 02.2.4 and 02.2.5 notes.

15. Troubleshooting

1) Run: adb devices -l and confirm 'device'. 2) Uninstall and reinstall: adb uninstall app.mapx then installDebug. 3) Capture the crash: adb logcat -d | Select-String 'FATAL|AndroidRuntime' and give the lines to the AI. 4) Retest on the other device to separate device-specific from general faults.

16. Dependencies

Depends on step 19.1.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

PHASE 20 — P2 ADVANCED FEATURES

Features required for the finished product but after P0/P1; Deferred items remain documented only.
Estimated relative effort: L. Gate: each P2 item either verified or explicitly marked Not built/Deferred.

STAGE 20.1 — Advanced Formats

STEP 20.1.1 — GeoTIFF Simple Subset (Partial)

Provenance: [SPEC DN-4]

1. Step ID / 2. Title

20.1.1 — GeoTIFF Simple Subset (Partial) (type: AI-assisted coding)

3. Objective

In-app reader for uncompressed/Deflate/LZW 8-bit RGB(A)/gray, EPSG:4326/3857/UTM; clear refusal otherwise.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 19.1.4 done and verified (tracker ticked). Phase Gate: each P2 item either verified or explicitly marked Not built/Deferred.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Alternative: QGIS/GDAL conversion.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 20.1.1 - GeoTIFF Simple Subset (Partial)
TASK: Write own TIFF reader (no library without approval).
CONTEXT TO ATTACH: DN-4.
FILES TO CREATE OR MODIFY: feature/mapsources/GeoTiff.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-20.1.1. If the result fails verification: git restore . (or git reset --hard before-20.1.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Simple sample works; others refused.

13. Acceptance criteria

[ ] Honest Partial status.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 19.1.4; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 20.1.2 — GeoPackage Raster, SQLiteDB and Calibrated .map

Provenance: [SPEC 6.3]

1. Step ID / 2. Title

20.1.2 — GeoPackage Raster, SQLiteDB and Calibrated .map (type: AI-assisted coding)

3. Objective

Convert tile tables to MBTiles; parse OziExplorer .map calibration.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 20.1.1 done and verified (tracker ticked). Phase Gate: each P2 item either verified or explicitly marked Not built/Deferred.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. OZF2/OZF3/QCT stay Deferred.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 20.1.2 - GeoPackage Raster, SQLiteDB and Calibrated .map
TASK: Write converters.
CONTEXT TO ATTACH: Spec 6.3.
FILES TO CREATE OR MODIFY: feature/mapsources/*.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-20.1.2. If the result fails verification: git restore . (or git reset --hard before-20.1.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Samples work.

13. Acceptance criteria

[ ] Matrix updated.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.

- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 20.1.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 20.1.3 — Shapefile Import/Export and GeoPackage Vector Import

Provenance: [SPEC 9.1]

1. Step ID / 2. Title

20.1.3 — Shapefile Import/Export and GeoPackage Vector Import (type: AI-assisted coding)

3. Objective

ZIP shp/shx/dbf/prj; GeoPackage vector import only.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 20.1.2 done and verified (tracker ticked). Phase Gate: each P2 item either verified or explicitly marked Not built/Deferred.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. One geometry type per file.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 20.1.3 - Shapefile Import/Export and GeoPackage Vector Import
TASK: Write shapefile IO.
CONTEXT TO ATTACH: Spec 9.1.
FILES TO CREATE OR MODIFY: feature/gis/Shapefile.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-20.1.3. If the result fails verification: git restore . (or git reset --hard before-20.1.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Samples work.

13. Acceptance criteria

[ ] Reports produced.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 20.1.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STAGE 20.2 — Other P2 Features

STEP 20.2.1 — Stacked Map Sources and Cropped MBTiles Export

Provenance: [SPEC 6.1]

1. Step ID / 2. Title

20.2.1 — Stacked Map Sources and Cropped MBTiles Export (type: AI-assisted coding)

3. Objective

Multiple background sources with opacity and order; crop MBTiles to project extent.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 20.1.3 done and verified (tracker ticked). Phase Gate: each P2 item either verified or explicitly marked Not built/Deferred.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. PMTiles cropping stays Deferred.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C). Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 20.2.1 - Stacked Map Sources and Cropped MBTiles Export
TASK: Write stacking and crop.
CONTEXT TO ATTACH: FR-SRC-06.
FILES TO CREATE OR MODIFY: feature/mapsources/Stack.kt
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-20.2.1. If the result fails verification: git restore . (or git reset --hard before-20.2.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure
Works.

13. Acceptance criteria
[ ] First source defines scale/attribution.

14. Common errors
- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting
1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies
Depends on step 20.1.3; leads to the next step in the phase order.

17. Estimated download / storage
No download.

STEP 20.2.2 — Biometric Unlock, Arabic-Indic Digits and TalkBack List View
Provenance: [SPEC P2]

1. Step ID / 2. Title
20.2.2 — Biometric Unlock, Arabic-Indic Digits and TalkBack List View (type: AI-assisted coding)

3. Objective
androidx.biometric strong-only; digits setting; object list alternative.

4. Why it is necessary
Required by the Spec area named in the tag.

5. Prerequisites
Step 20.2.1 done and verified (tracker ticked). Phase Gate: each P2 item either verified or explicitly marked Not built/Deferred.

6. Required software
Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions
1. Password always remains available.

8. Terminal commands
Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links
None required for this step.

10. Complete AI implementation prompt
Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 20.2.2 - Biometric Unlock, Arabic-Indic Digits and TalkBack List View
TASK: Write the three features.
CONTEXT TO ATTACH: FR-SEC-09, FR-CRD-09, FR-ACC-02.
FILES TO CREATE OR MODIFY: feature/*
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-20.2.2. If the result fails verification: git restore . (or git reset --hard before-20.2.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result
Feature works and tests pass.

12. Verification procedure
Works.

13. Acceptance criteria
[ ] Documented.

14. Common errors
- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting
1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies
Depends on step 20.2.1; leads to the next step in the phase order.

17. Estimated download / storage
No download.

STEP 20.2.3 — Layer Copy, Backup Reminders, Coordinate Grid and Nearby Transfer
Provenance: [SPEC DN-10]

1. Step ID / 2. Title
20.2.3 — Layer Copy, Backup Reminders, Coordinate Grid and Nearby Transfer (type: AI-assisted coding)

3. Objective
Copy layers between projects; reminder; grid overlay; direct device transfer reusing pairing.

4. Why it is necessary
Required by the Spec area named in the tag.

5. Prerequisites
Step 20.2.2 done and verified (tracker ticked). Phase Gate: each P2 item either verified or explicitly marked Not built/Deferred.

6. Required software
Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions
1. Nearby transfer may be marked Not built if time is short (P2).

8. Terminal commands
Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat testDebugUnitTest assembleDebug --offline
# Success: BUILD SUCCESSFUL. Failure: read the first 'error:' line and use the prompt in Troubleshooting.
.\tools\check.ps1
```

9. Official download links
None required for this step.

10. Complete AI implementation prompt
Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 20.2.3 - Layer Copy, Backup Reminders, Coordinate Grid and Nearby Transfer
TASK: Write features.
CONTEXT TO ATTACH: FR-LYR-06, FR-EXP-05, DN-10.
FILES TO CREATE OR MODIFY: feature/*
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-20.2.3. If the result fails verification: git restore . (or git reset --hard before-20.2.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Works or documented.

13. Acceptance criteria

[ ] No false claims.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 20.2.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

PHASE 21 — HARDENING, PERFORMANCE AND ACCEPTANCE

Budget measurements, leaks, security review, targetSdk decision, full acceptance run and regression pack.
Estimated relative effort: L. Gate: all AC- criteria have evidence or an honest Not tested label.*

STAGE 21.1 — Performance and Stability

STEP 21.1.1 — Performance Budgets on the Reference Device

Provenance: [SPEC 13.2]

1. Step ID / 2. Title

21.1.1 — Performance Budgets on the Reference Device (type: device test)

3. Objective

Measure every Spec 13.2 budget on the Lenovo Tab P12 and record numbers.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 20.2.3 done and verified (tracker ticked). Phase Gate: all AC-* criteria have evidence or an honest Not tested label.

6. Required software

Android Studio, adb, the Lenovo Tab P12 and/or Samsung Galaxy A16 (USB debugging on), Git.

7. Exact instructions

1. Cold start <3 s; 1,000 objects ~60 fps; 5,000-object open <3 s; PNG/PDF; 500 MB package stream.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat installDebug --offline
# Then run the app on the device named in the verification procedure and note the result.
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 21.1.1 - Performance Budgets on the Reference Device
TASK: Create performance test script.
CONTEXT TO ATTACH: AC-PERF-1.
FILES TO CREATE OR MODIFY: docs/test/perf.md
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-21.1.1. If the result fails verification: git restore . (or git reset --hard before-21.1.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Numbers recorded.

13. Acceptance criteria

[ ] Failures logged as issues.

14. Common errors

- App crashes on start -> read logcat FATAL lines.
- Works on one device only -> compare screen class (tablet vs phone) and Android version.
- Install blocked -> see 02.2.4 and 02.2.5 notes.

15. Troubleshooting

1) Run: adb devices -l and confirm 'device'. 2) Uninstall and reinstall: adb uninstall app.mapx then installDebug. 3) Capture the crash: adb logcat -d | Select-String 'FATAL|AndroidRuntime' and give the lines to the AI. 4) Retest on the other device to separate device-specific from general faults.

16. Dependencies

Depends on step 20.2.3; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 21.1.2 — Memory, Leaks and StrictMode

Provenance: [SPEC 13.2]

1. Step ID / 2. Title

21.1.2 — Memory, Leaks and StrictMode (type: device test)

3. Objective

LeakCanary and StrictMode runs; process-death restore; low-memory behaviour.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 21.1.1 done and verified (tracker ticked). Phase Gate: all AC-* criteria have evidence or an honest Not tested label.

6. Required software

Android Studio, adb, the Lenovo Tab P12 and/or Samsung Galaxy A16 (USB debugging on), Git.

7. Exact instructions

1. Presenter 60 min; external display 30 min.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat installDebug --offline
# Then run the app on the device named in the verification procedure and note the result.
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 21.1.2 - Memory, Leaks and StrictMode
TASK: Review leak reports.
CONTEXT TO ATTACH: Spec 13.2.
FILES TO CREATE OR MODIFY: docs/test/stability.md
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-21.1.2. If the result fails verification: git restore . (or git reset --hard before-21.1.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

No leaks.

13. Acceptance criteria

[ ] No StrictMode violations.

14. Common errors

- App crashes on start -> read logcat FATAL lines.
- Works on one device only -> compare screen class (tablet vs phone) and Android version.
- Install blocked -> see 02.2.4 and 02.2.5 notes.

15. Troubleshooting

1) Run: adb devices -l and confirm 'device'. 2) Uninstall and reinstall: adb uninstall app.mapx then installDebug. 3) Capture the crash: adb logcat -d | Select-String 'FATAL|AndroidRuntime' and give the lines to the AI. 4) Retest on the other device to separate device-specific from general faults.

16. Dependencies

Depends on step 21.1.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STAGE 21.2 — Security and Offline Audit

STEP 21.2.1 — Full Offline Guard Run

Provenance: [SPEC AC-OFF-1]

1. Step ID / 2. Title

21.2.1 — Full Offline Guard Run (type: device test)

3. Objective

Airplane mode run of sample briefing and an export/import round trip.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 21.1.2 done and verified (tracker ticked). Phase Gate: all AC-* criteria have evidence or an honest Not tested label.

6. Required software

Android Studio, adb, the Lenovo Tab P12 and/or Samsung Galaxy A16 (USB debugging on), Git.

7. Exact instructions

1. Offline guard reports zero.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat installDebug --offline
# Then run the app on the device named in the verification procedure and note the result.
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 21.2.1 - Full Offline Guard Run
TASK: Create offline script.
CONTEXT TO ATTACH: AC-OFF-1.
FILES TO CREATE OR MODIFY: docs/test/offline.md
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-21.2.1. If the result fails verification: git restore . (or git reset --hard before-21.2.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Zero requests.

13. Acceptance criteria

- [ ] Evidence recorded.

14. Common errors

- App crashes on start -> read logcat FATAL lines.
- Works on one device only -> compare screen class (tablet vs phone) and Android version.

- Install blocked -> see 02.2.4 and 02.2.5 notes.

15. Troubleshooting

1) Run: adb devices -l and confirm 'device'. 2) Uninstall and reinstall: adb uninstall app.mapx then installDebug. 3) Capture the crash: adb logcat -d | Select-String 'FATAL|AndroidRuntime' and give the lines to the AI. 4) Retest on the other device to separate device-specific from general faults.

16. Dependencies

Depends on step 21.1.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 21.2.2 — Security Review and SECURITY.md Completion

Provenance: [SPEC 18.1 rule 15]

1. Step ID / 2. Title

21.2.2 — Security Review and SECURITY.md Completion (type: document)

3. Objective

Review passwords, remote, permissions, exports; complete SECURITY.md.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 21.2.1 done and verified (tracker ticked). Phase Gate: all AC-* criteria have evidence or an honest Not tested label.

6. Required software

VS Code or any text editor, Git, one AI tool from the AI field.

7. Exact instructions

1. Check no plaintext secrets, no cleartext except loopback.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
# No command required. Commit when finished:
git add -A
git commit -m "docs: step done"
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 21.2.2 - Security Review and SECURITY.md Completion
TASK: Review security checklist.
CONTEXT TO ATTACH: AC-SEC-1, AC-REM-1.
FILES TO CREATE OR MODIFY: docs/SECURITY.md
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-21.2.2. If the result fails verification: git restore . (or git reset --hard before-21.2.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Complete.

13. Acceptance criteria

[ ] BLOCKER list empty.

14. Common errors

- AI invents facts -> require it to quote the specification section it relied on.

15. Troubleshooting

1) Compare the document with the listed sources. 2) Ask the AI to list contradictions between the document and the specification. 3) Fix and commit.

16. Dependencies

Depends on step 21.2.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 21.2.3 — targetSdk 37 Decision

Provenance: [VERIFIED 4 Nov 2026]

1. Step ID / 2. Title

21.2.3 — targetSdk 37 Decision (type: decision)

3. Objective

Decide whether to raise targetSdk to 37 (needs ACCESS_LOCAL_NETWORK handling for the remote).

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 21.2.2 done and verified (tracker ticked). Phase Gate: all AC-* criteria have evidence or an honest Not tested label.

6. Required software

A web browser for official pages, VS Code, Git, one AI tool from the AI field.

7. Exact instructions

1. If raised: declare permission, rationale UI, denied state, test on an Android 17 device or keep 36 and record why.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
# No command required. Record the decision in docs\MAPX_DECISIONS.md and commit.
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).

Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 21.2.3 - targetSdk 37 Decision
TASK: Prepare targetSdk migration checklist.
CONTEXT TO ATTACH: Spec 10.4.
FILES TO CREATE OR MODIFY: app/build.gradle.kts, manifest
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-21.2.3. If the result fails verification: git restore . (or git reset --hard before-21.2.3), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Decision recorded.

13. Acceptance criteria

[ ] Remote re-tested if changed.

14. Common errors

- Source page unavailable -> mark the value 'unverified' and continue with the template version.

15. Troubleshooting

1) Reopen the official page and copy the exact text or version. 2) If two sources disagree, prefer the specification, then the official page, and record the conflict in MAPX_DECISIONS.md.

16. Dependencies

Depends on step 21.2.2; leads to the next step in the phase order.

17. Estimated download / storage
No download.

STAGE 21.3 — Full Acceptance

STEP 21.3.1 — Full Acceptance Matrix Run

Provenance: [SPEC 16]

1. Step ID / 2. Title
21.3.1 — Full Acceptance Matrix Run (type: device test)

3. Objective
Run all AC-* criteria and the manual verification list on both devices in English and Arabic at all four modes.

4. Why it is necessary
Required by the Spec area named in the tag.

5. Prerequisites
Step 21.2.3 done and verified (tracker ticked). Phase Gate: all AC-* criteria have evidence or an honest Not tested label.

6. Required software
Android Studio, adb, the Lenovo Tab P12 and/or Samsung Galaxy A16 (USB debugging on), Git.

7. Exact instructions
1. Fill MAPX_STATUS with Implemented/Built/Automated/Device verified/Not tested/Deferred.

8. Terminal commands
Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat installDebug --offline
# Then run the app on the device named in the verification procedure and note the result.
```

9. Official download links
None required for this step.

10. Complete AI implementation prompt
Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 21.3.1 - Full Acceptance Matrix Run
TASK: Create acceptance run sheet.
CONTEXT TO ATTACH: Spec 16.
FILES TO CREATE OR MODIFY: docs/test/acceptance.md
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-21.3.1. If the result fails verification: git restore . (or git reset --hard before-21.3.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result
Feature works and tests pass.

12. Verification procedure
Complete evidence.

13. Acceptance criteria
[ ] Honest statuses.

14. Common errors
- App crashes on start -> read logcat FATAL lines.
- Works on one device only -> compare screen class (tablet vs phone) and Android version.
- Install blocked -> see 02.2.4 and 02.2.5 notes.

15. Troubleshooting
1) Run: adb devices -l and confirm 'device'. 2) Uninstall and reinstall: adb uninstall app.mapx then installDebug. 3) Capture the crash: adb logcat -d | Select-String 'FATAL|AndroidRuntime' and give the lines to the AI. 4) Retest on the other device to separate device-specific from general faults.

16. Dependencies
Depends on step 21.2.3; leads to the next step in the phase order.

17. Estimated download / storage
No download.

STEP 21.3.2 — Regression Pack and Fix Cycle
Provenance: [SPEC 16]

1. Step ID / 2. Title
21.3.2 — Regression Pack and Fix Cycle (type: device test)

3. Objective
Re-run all automated tests after fixes; fix BLOCKERs and MAJORs.

4. Why it is necessary
Required by the Spec area named in the tag.

5. Prerequisites
Step 21.3.1 done and verified (tracker ticked). Phase Gate: all AC-* criteria have evidence or an honest Not tested label.

6. Required software
Android Studio, adb, the Lenovo Tab P12 and/or Samsung Galaxy A16 (USB debugging on), Git.

7. Exact instructions
1. Never weaken a test.

8. Terminal commands
Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
.\gradlew.bat installDebug --offline
# Then run the app on the device named in the verification procedure and note the result.
```

9. Official download links
None required for this step.

10. Complete AI implementation prompt
Which AI: Primary: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) Fallback: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 21.3.2 - Regression Pack and Fix Cycle
TASK: Triage failures.
CONTEXT TO ATTACH: Spec 16.
FILES TO CREATE OR MODIFY: tests
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-21.3.2. If the result fails verification: git restore . (or git reset --hard before-21.3.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result
Feature works and tests pass.

12. Verification procedure
Green.

13. Acceptance criteria
[ ] No open BLOCKER.

14. Common errors
- App crashes on start -> read logcat FATAL lines.
- Works on one device only -> compare screen class (tablet vs phone) and Android version.
- Install blocked -> see 02.2.4 and 02.2.5 notes.

15. Troubleshooting
1) Run: adb devices -l and confirm 'device'. 2) Uninstall and reinstall: adb uninstall app.mapx then installDebug. 3) Capture the crash: adb logcat -d | Select-String 'FATAL|AndroidRuntime' and give the lines to the AI. 4) Retest on the other device to separate device-specific from general faults.

16. Dependencies

Depends on step 21.3.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

PHASE 22 — RELEASE AND HANDOFF

Signed release build, installation on both devices, final documents and final report.
Estimated relative effort: S. Gate: signed APK installed; final report written with honest verdict.

STAGE 22.1 — Release

STEP 22.1.1 — R8 Release Build and Signing

Provenance: [SPEC 14]

1. Step ID / 2. Title

22.1.1 — R8 Release Build and Signing (type: AI-assisted coding)

3. Objective

Enable R8, keep rules for MapLibre/serialization/Room; build signed APK; verify 16 KB alignment again.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 21.3.2 done and verified (tracker ticked). Phase Gate: signed APK installed; final report written with honest verdict.

6. Required software

Android Studio, Gradle wrapper (.\gradlew.bat), Git, one AI tool from the AI field.

7. Exact instructions

1. Install and run release build.

8. Terminal commands

Where to run: PowerShell.

```
.\gradlew.bat assembleRelease
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Google AI Studio free tier (best for large, multi-file generation because of its long context) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).
Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 22.1.1 - R8 Release Build and Signing
TASK: Write proguard rules.
CONTEXT TO ATTACH: Spec 14.
FILES TO CREATE OR MODIFY: app/proguard-rules.pro
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-22.1.1. If the result fails verification: git restore . (or git reset --hard before-22.1.1), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Release works.

13. Acceptance criteria

[ ] Alignment check passes.

14. Common errors

- Build error after pasting AI code -> the AI used a different library version; give it the exact version from the catalog.
- Test fails after AI edit -> the AI changed behaviour; revert with git restore and re-prompt with 'do not change existing tests'.
- 'Unresolved reference' -> missing import or wrong package; check package names against Spec 7.2.

15. Troubleshooting

1) Read the first error line (not the last). 2) Paste that line, the file path and the failing code to the AI with 'Fix only this error; change the fewest lines.' 3) If the AI loops twice, run: git restore . and re-run the prompt on the next tool in the

ladder (Appendix C). 4) If the build cache seems broken: .\gradlew.bat --stop then rebuild (never delete .gradle unless disk is critical).

16. Dependencies

Depends on step 21.3.2; leads to the next step in the phase order.

17. Estimated download / storage

No download.

STEP 22.1.2 — Install, Final Documents and Final Report

Provenance: [SPEC 18.3]

1. Step ID / 2. Title

22.1.2 — Install, Final Documents and Final Report (type: document)

3. Objective

Install on both devices; update all six documents; write final report in Spec 18.3 format.

4. Why it is necessary

Required by the Spec area named in the tag.

5. Prerequisites

Step 22.1.1 done and verified (tracker ticked). Phase Gate: signed APK installed; final report written with honest verdict.

6. Required software

VS Code or any text editor, Git, one AI tool from the AI field.

7. Exact instructions

1. Verdict COMPLETE or INCOMPLETE with justification.

8. Terminal commands

Where to run: PowerShell opened in C:\dev\MapX (or the folder named above).

```
# No command required. Commit when finished:
git add -A
git commit -m "docs: step done"
```

9. Official download links

None required for this step.

10. Complete AI implementation prompt

Which AI: Primary: Gemini, ChatGPT or Claude free tier (best for architecture questions, code review and troubleshooting; short sessions) Fallback: Local Qwen2.5-Coder-7B-Instruct in LM Studio (offline; best for one file or one pure function at a time, short explanations, unit tests; keep each prompt small) If every free limit is reached: split the prompt into one file per request and run it on local Qwen (Appendix C).

Prompt (copy everything in the box; attach the Context Pack first):

```
ROLE: You are a senior Android engineer helping a non-programmer build MapX, a native Kotlin/Compose offline-
  first geographic briefing app (minSdk 28), Windows 11 + Android Studio.
STEP: 22.1.2 - Install, Final Documents and Final Report
TASK: Draft final report from MAPX_STATUS.
CONTEXT TO ATTACH: Spec 18.3.
FILES TO CREATE OR MODIFY: docs/FINAL_REPORT.md
RULES: Follow Spec 18.1 rules. Native Android only: Kotlin, Jetpack Compose, Material 3, MapLibre Native, Room,
  DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer. Do not add a
  dependency that is not already in the version catalog; if one is needed, stop and say so. Store geographic
  data as WGS84 lat/lon, never screen pixels. Every edit is a command. No hard-coded user-visible strings (use
  string resources, English and Arabic). Touch targets at least 48 dp. Use start/end, never left/right. Write or
  update unit tests. Do not weaken or delete existing tests. Never claim something works unless it was built and
  tested.
OUTPUT: Complete files with paths.
SELF-CHECK BEFORE ANSWERING: list any rule you could not follow and any assumption you made; do not invent
  library versions or API names - say 'verify' instead.
```

Recovery if it fails: Before starting: git add -A; git commit -m "checkpoint"; git tag before-22.1.2. If the result fails verification: git restore . (or git reset --hard before-22.1.2), then retry with a smaller prompt (one file at a time) or the next AI in the ladder. Restore from the backup.ps1 copy only if the repository itself is damaged.

11. Expected result

Feature works and tests pass.

12. Verification procedure

Report written.

13. Acceptance criteria

- [ ] Not tested items listed.

14. Common errors

- AI invents facts -> require it to quote the specification section it relied on.

15. Troubleshooting

1) Compare the document with the listed sources. 2) Ask the AI to list contradictions between the document and the specification. 3) Fix and commit.

16. Dependencies

Depends on step 22.1.1; leads to the next step in the phase order.

17. Estimated download / storage

No download.

Appendix A - Provenance and verified research (4 Nov 2026)

Tag	Marker	Meaning
Requirement (uploaded document)	[SPEC]	Statement taken from the Master Project Specification or the UI/UX Guide.
Current verified information	[VERIFIED date]	Found in official or primary pages on 4 Nov 2026 during research; recheck before pinning.
Technical recommendation	[REC]	The roadmap author's recommendation; changeable with a recorded decision.
Unverified assumption	[ASSUMED]	Not confirmed from supplied material; recorded in MAPX_DECISIONS.md.

Topic	Finding	Roadmap decision
MapLibre Native Android	Maven coordinate org.maplibre.gl:android-sdk. Versions in the 13.x series appear on Maven Central (13.7.1 listed for a related artifact on 16 Oct 2026). The specification cited 11.11.0 and requires 11.7.0 or newer for PMTiles. 16 KB page support was added in 11.5.0 per its release notes.	Pin the newest stable that passes spikes 03.1.1-03.1.4. A newer major may change APIs; unverified until the spike.
Android Studio / AGP	Official release pages show a stable channel with a stable AGP 9.4.0 and a Studio named 'Rabbit 1'; older pages show Otter/Narwhal. Pages disagree because they were cached at different dates.	Use the stable build that Help > Check for Updates offers; let the New Project template choose co-tested AGP, Gradle and Kotlin.
Compose BOM	Official page confirms the BOM approach and that the Compose compiler is tied to the Kotlin version (Kotlin 2.0+). A current BOM number could not be confirmed from search results.	Take the BOM version from the official page in step 02.1.5.
Android 17 / API 37	Stable on 16 Jun 2026. Targeting 37 makes the ACCESS_LOCAL_NETWORK runtime permission mandatory for LAN access and removes large-screen orientation opt-outs. Google Play's own requirement (API 36 from 31 Aug 2026) is irrelevant to a private-use APK.	targetSdk 36 first; decide on 37 in step 21.2.3.
Java-WebSocket	Latest published release 1.6.0 (Dec 2024), MIT licence, uses SLF4J.	Pin 1.6.0 unless the official page shows newer; spike 03.2.1 decides.
Lenovo Tab P12	Wi-Fi only (TB370FU), USB-C 2.0, no DisplayPort listed; Wi-Fi Display listed by one source. GPS: retail sites say GPS/GLONASS, one database says 'Positioning: No'.	Trust only adb results (01.1.9, 03.2.2, 03.2.3).
AI free tiers	Limits change without notice.	No limit values are assumed; record what you observe (01.3.1).

Appendix B - Standing Orders (paste at the start of every AI session)

```
You are the MapX engineering assistant. MapX is a native Android (Kotlin, Jetpack Compose, Material 3, MapLibre
  Native, Room, DataStore, kotlinx.serialization, Coroutines/Flow, ViewModels, manual DI via AppContainer),
  offline-first geographic briefing and presentation app, minSdk 28, tablet landscape first.
Authority order: Master Project Specification, then UI/UX Design Guide, then the Layout Blueprint inside it (as
  amended BA-01..BA-17). Do not silently change requirements.
1 Geographic truth: store WGS84 lat/lon or explicit geographic geometry; never pixels or offsets; handles are
  transient.
2 Every edit is a command; one gesture = one undo step; UI never mutates project state.
3 Layout: four modes, one classifier, one LayoutSpec, regions are siblings, no offset/zIndex fixes, touch
  targets >= 48 dp.
4 Storage split: Room structured data, DataStore settings, app files for media and map sources.
5 Honest support: report unsupported formats; ordinary images never get invented coordinates.
6 Localization: no hard-coded strings; English LTR and Arabic RTL; start/end only.
7 Dependencies: add none without stating what, why, size, licence, alternatives.
8 Tests for all new logic; never delete or weaken a test.
9 Data safety: failures never corrupt projects; destructive actions confirmed.
10 Performance: no main-thread I/O; stream large files.
11 Privacy: no telemetry; no network in core workflows.
12 Evidence: never claim it works unless built and tested; list what was not tested.
13 Smallest safe change; fix root causes.
14 Presenting and the phone remote never change project data.
15 No plaintext passwords or secrets; validate every remote message against the allow-list.
Always answer with complete files and their full paths, then verification steps. If unsure of an API or version,
  say 'verify' instead of guessing.
```

Appendix C - Free-tier AI tool matrix and fallback ladder

Free limits change without notice and are deliberately not quoted. Record what you observe in AI_WORKFLOW.md (step 01.3.1).

Rank	Tool	Best for	Typical limit event	Fallback action
1	AI Studio free tier	Large multi-file generation, long documents	Message/token/rate limit reached	Split the task by file; move to AntiGravity or Gemini/ChatGPT/Claude
2	AntiGravity IDE	Editing several files in place	Quota or plan limit	Use AI Studio in the browser and paste files; or local Qwen
3	Gemini / ChatGPT / Claude free	Architecture, review, troubleshooting	Daily message cap or reduced model	Move the same prompt to the next platform; keep sessions short
4	Local Qwen2.5-Coder-7B (LM Studio)	One file or function; tests; explanations; unlimited	Context overflow or weak answer	Shrink the prompt (one file, under ~6,000 tokens), ask for tests first, then code; use a cloud AI only to review

Context Pack template (what make-context.ps1 produces, in this order): 1) Standing Orders; 2) MAPX_HANDOFF.md; 3) MAPX_STATUS.md (only relevant rows); 4) specification sections named by the step; 5) the source files named by the step; 6) the step prompt. Keep it under ~5,000 tokens for local Qwen.

Appendix D - Engineering document lifecycle

Document	Created in	Updated when
ARCHITECTURE.md	02.1.3	End of every phase; whenever a format, package or rule changes; matrix Evidence cells after 03.1.2, 03.1.3, 09.3.4.
MAPX_DECISIONS.md	02.1.4	Every time a decision, dependency or assumption is made (spikes, version gate, targetSdk, dock/rail slots).
MAPX_HANDOFF.md	02.1.4	End of every working session (what is built, known failures, what must not change, next action).
MAPX_STATUS.md	02.1.4	Only after verification: statuses Implemented / Built / Automated-test verified / Device verified / Not tested / Deferred.
SYMBOLS_LICENSE.md	02.1.4	Every time a symbol is added (11.3.1, 11.3.2).
SECURITY.md	02.1.4	Phase 06, 16, 17, 21.2.2: account design, lock, pairing protocol, permission table.

Appendix E - Dependency register

Only dependencies allowed by Specification section 11 are listed. Versions are pinned in step 02.1.5; sizes marked verify must be read from Maven Central or the Gradle cache.

Dependency	Purpose	Why required	Size	Licence	Alternatives	Compatibility
MapLibre Native Android	Map rendering, PMTiles, snapshots	Spec-mandated engine	tens of MB (verify)	BSD-2-Clause	None allowed by Spec	16 KB support from 11.5.0; check .so alignment
AndroidX Compose, Material 3, Activity, Lifecycle	UI and state	Spec 7.1	tens of MB cache	Apache-2.0	None	Via BOM
Room (+KSP)	Structured storage	Spec 8	small	Apache-2.0	Raw SQLite (more code)	KSP version must match Kotlin
DataStore Preferences	Settings	Spec 8	small	Apache-2.0	SharedPreferences	-
kotlinx.serialization, coroutines	JSON documents, async	Spec 7.1	small	Apache-2.0	Gson/Moshi (not allowed)	Plugin matches Kotlin
Java-WebSocket 1.6.0	Remote transport	Android has no WebSocket server (DN-3)	small (verify)	MIT	TLS sockets with JSON lines (documented fallback)	Spike 03.2.1
CameraX + ZXing-core	QR scan and QR generation	Offline QR without ML Kit	few MB (verify)	Apache-2.0	Manual code only	Camera permission only on scan screen
androidx.biometric	Optional unlock (P2)	FR-SEC-09	small	Apache-2.0	Password only	-
LeakCanary (debug only)	Leak detection	Spec 13.2	debug only	Apache-2.0	Android Studio profiler	Not in release
JUnit, coroutines-test, Compose UI test	Testing	Spec 7.1	small	Apache-2.0/EPL	-	Test only
Cairo font, Natural Earth data	Typography; basemap	Spec 12.2, FR-MAP-02	<1 MB; few MB	OFL 1.1; public domain	-	Keep licence files

Appendix F - Format Support Matrix evidence plan

Format	Priority	Expected status	Test steps	Sample needed
MBTiles raster / vector	P0 / P1	Expected, unconfirmed	03.1.3, 09.2.2	Small raster MBTiles; a vector (PBF) MBTiles
PMTiles v3	P0	Supported (documented)	03.1.2, 09.2.1	Small raster or vector PMTiles
XYZ/TMS ZIP	P1	Supported	09.2.3	MOBAC XYZ ZIP
PNG/JPG/WebP ordinary images	P0	Supported (IMAGE_LOCAL)	09.3.1	Any scanned map image
World-file georeferenced image	P1	Supported	09.3.2	Image + .pgw/.jgw/.wld
KML/KMZ GroundOverlay	P1	Supported	12.1.2	Google Earth KMZ overlay
Manual georeferencing	P1	Supported	13.1.1-13.1.3	Scanned map + known points
Calibrated .map (Ozi)	P2	Partial	20.1.2	Ozi .map + image
GeoTIFF	P2	Partial (simple)	20.1.1	Simple 8-bit GeoTIFF and an unsupported one
GeoPackage raster, SQLiteDB	P2	Partial	20.1.2	Samples of each
GeoJSON, KML, KMZ, GPX, CSV/TSV import/export	P1	Lossy	12.1.1-12.2.2	One file of each from Google Earth/AlpineQuest
Shapefile ZIP; GeoPackage vector import	P2	Lossy	20.1.3	Shapefile ZIP
AlpineQuest .AQM/.PGD, Mapsforge, OZF/QCT, DEM, Google imagery tiles	D	Deferred / not available	09.3.4	No sample: documented alternatives only

Appendix G - Requirements traceability

Requirement area	Roadmap steps
Projects, autosave, last-known-good, migrations (FR-PRJ-*, AC-PRJ-1)	04.2.5, 05.1.1, 05.2.1-05.2.5, 07.2.2, 07.2.3, 10.3.4
Map view, controls, coordinate search (FR-MAP-*)	03.1.1, 08.1.1-08.1.4, 08.2.1-08.2.3
GPS location (FR-GPS-*, AC-GPS-1)	03.2.3, 17.1.1-17.1.3
Coordinate systems and metric units (FR-CRD-*, AC-CRD-1)	04.1.1-04.1.5, 08.1.4
Objects, commands, selection, HandleLayer (FR-OBJ-*)	04.2.1-04.2.4, 04.2.7, 10.1.1-10.1.5, 10.3.3
Drawing tools, 11 tools (FR-DRW-*, AC-DRW-1)	04.2.4, 10.2.1-10.2.6
Layers (FR-LYR-*, AC-LYR-1)	04.2.6, 10.3.2, 20.2.3
Images and animation (FR-IMG-*, FR-ANI-*, AC-IMG-1)	11.1.1-11.1.3, 11.2.1, 11.2.2
Symbols (FR-SYM-*)	11.3.1-11.3.3
Scenes (FR-SCN-*)	04.2.6, 14.1.1, 14.2.5
Presenter Mode and broadcast graphics (FR-PRS-*, AC-PRS-1)	14.1.2, 14.2.1-14.2.6
External display (FR-DSP-*, AC-DSP-1)	03.2.2, 15.1.1-15.1.4
Phone remote (FR-REM-*, AC-REM-1)	03.2.1, 16.1.1-16.1.3, 16.2.1-16.2.4
Settings, language, accessibility (FR-SET-*, FR-ACC-*)	02.3.3, 02.3.4, 19.1.1-19.1.4, 20.2.2
Map sources and formats (FR-SRC-*, Spec 6, AC-SRC-1)	03.1.2, 03.1.3, 09.1.1-09.3.4, 20.1.1, 20.1.2, 20.2.1
Georeferencing (FR-GRF-*, AC-SRC-2)	09.3.2, 13.1.1-13.1.4
Import/export, packages, backup, fidelity (Spec 9, AC-EXP-1..5)	12.1.1-12.3.7, 20.1.3
PNG and PDF export (FR-IMX-*, AC-EXP-4)	18.1.1, 18.1.2
Login, accounts, security (FR-SEC-*, AC-SEC-1)	06.1.1-06.2.5, 21.2.2
Permissions and privacy (Spec 10.3, 10.4)	02.2.3, 05.1.4, 16.2.2, 17.1.1, 21.2.3
Offline operation (AC-OFF-1)	05.1.4, 08.2.2, 21.2.1
Performance budgets (AC-PERF-1)	21.1.1, 21.1.2
Localization English/Arabic RTL (AC-I18N-1)	02.3.3, 03.1.4, 07.2.6, 19.1.2
Layout contract, four modes (AC-UI-1, BA-01..BA-17)	07.1.1-07.1.6, 07.2.1-07.2.6
Honest limitations (AC-HON-1)	09.3.4, 12.2.1, 15.1.4, 21.3.1
Engineering documents (Spec 18.2)	02.1.3, 02.1.4, 03.3.1, 22.1.2
Final acceptance and release (Spec 16, 18.3)	21.3.1, 21.3.2, 22.1.1, 22.1.2
Environment, devices, free-AI workflow (master prompt 4-6, 12)	01.1.1-01.3.4

Appendix H - Official links

Resource	URL	Note
Android Studio download	https://developer.android.com/studio	appeared in research results
Android Studio and AGP releases	https://developer.android.com/studio/releases	appeared in research results
sdkmanager	https://developer.android.com/tools/sdkmanager	stable official address; confirm it opens
adb	https://developer.android.com/tools/adb	stable official address; confirm it opens
Platform-Tools	https://developer.android.com/tools/releases/platform-tools	stable official address; confirm it opens
Compose BOM	https://developer.android.com/jetpack/compose/bom	appeared in research results
Room	https://developer.android.com/jetpack/androidx/releases/room	stable official address; confirm it opens
DataStore	https://developer.android.com/jetpack/androidx/releases/datastore	stable official address; confirm it opens
CameraX	https://developer.android.com/jetpack/androidx/releases/camera	stable official address; confirm it opens
16 KB page sizes	https://developer.android.com/guide/practices/page-sizes	stable official address; confirm it opens
Local network permission	https://developer.android.com/privacy-and-security/local-network-permission	appeared in research results
Presentation API	https://developer.android.com/reference/android/app/Presentation	stable official address; confirm it opens
DisplayManager	https://developer.android.com/reference/android/hardware/display/DisplayManager	stable official address; confirm it opens
Gradle releases	https://gradle.org/releases/	stable official address; confirm it opens
Kotlin releases	https://kotlinlang.org/docs/releases.html	stable official address; confirm it opens
MapLibre Native	https://github.com/maplibre/maplibre-native	appeared in research results
Java-WebSocket	https://github.com/TooTallNate/Java-WebSocket	appeared in research results
PMTiles docs	https://docs.protomaps.com/pmtiles/	stable official address; confirm it opens
Natural Earth	https://www.naturalearthdata.com/	stable official address; confirm it opens
Cairo font (OFL)	https://fonts.google.com/specimen/Cairo	stable official address; confirm it opens
MOBAC	https://mobac.sourceforge.io/	stable official address; confirm it opens
QGIS	https://qgis.org/download/	stable official address; confirm it opens
Git for Windows	https://git-scm.com/download/win	stable official address; confirm it opens
Node.js	https://nodejs.org/	stable official address; confirm it opens
Python for Windows	https://www.python.org/downloads/windows/	stable official address; confirm it opens
LM Studio	https://lmstudio.ai/	stable official address; confirm it opens
Google AI Studio	https://aistudio.google.com/	stable official address; confirm it opens
Flutter docs (reference only)	https://docs.flutter.dev/	stable official address; confirm it opens

No URL was invented; none was fetched from this environment beyond search results.

Appendix I - Assumptions and unresolved limitations

Assumptions

- Project folder C:\dev\MapX and applicationId app.mapx (assumption D-R05).
- CPU is an i7-9700 (identified from a screenshot; verify in 01.1.1).
- 'Cloud Free' in the master prompt means Claude free tier (confirm in 01.3.1).
- targetSdk 36 first; API 37 decision in 21.2.3 (D-R02).
- Phone portrait dock Select, Pan, Draw, Layers, More follows the Guide (register C-01, owner confirmation flagged). Phone landscape rail slots are not defined by the sources (C-08); the assumed five slots are Select, Pan, Draw, Layers, Scenes plus More (D-R09). More popup contents follow C-09.
- Lightweight state-based navigation instead of the Navigation library (D-R03) to avoid a dependency.
- Bundled basemap is built from simplified Natural Earth GeoJSON rather than vector tiles (08.2.1), because no tile-generation tooling is assumed on Windows.
- Specification open decisions DN-1..DN-10 use their documented defaults; DN-9 is closed by adb evidence.

Limitations

- Nothing in this roadmap has been built or run. [VERIFIED] means checked against documentation, not tested in MapX.
- The exact current stable versions of AGP, Gradle, Kotlin, Compose BOM, Room and MapLibre could not all be confirmed from this environment; step 02.1.5 is the mandatory Version Gate.
- MBTiles direct reading, Arabic base-map label quality, TLS WebSocket pairing, the 16 KB alignment of the chosen MapLibre build, and external display capability on the Lenovo Tab P12 are unproven until the Phase 03 spikes run.
- The Lenovo Tab P12 has USB 2.0 and no listed DisplayPort, so a wired external display may be impossible; the in-app Audience Preview is then the only honest option. GPS hardware on the tablet is unconfirmed.
- Official URLs: those marked (seen) appeared in research results; others are stable official documentation addresses that could not be reopened from this environment; confirm each opens.
- Free AI tier limits are unknown and change; every step has a local-Qwen fallback but a 7B model may fail the hardest tasks (UTM/MGRS, TLS, package streaming) and then needs smaller prompts and more review.
- UI/UX open items C-01, C-08, C-09 need owner confirmation; Arabic strings need a native reviewer.

Appendix J - Command cheat sheet

Purpose	Command
List devices	adb devices -l
Install debug build offline	.\gradlew.bat installDebug --offline
Unit tests	.\gradlew.bat testDebugUnitTest
Device tests	.\gradlew.bat connectedDebugAndroidTest
Release build	.\gradlew.bat assembleRelease
Stop Gradle daemons	.\gradlew.bat --stop
Crash log	adb logcat -d | Select-String 'FATAL|AndroidRuntime'
Clear app data	adb shell pm clear app.mapx
Pull a file from app storage	adb exec-out run-as app.mapx cat files/<path> > out.bin
Page size	adb shell getconf PAGE_SIZE
Checkpoint tag	git add -A; git commit -m 'checkpoint'; git tag before-<id>
Undo uncommitted changes	git restore .
Backup copy	.\tools\backup.ps1
Quality gate	.\tools\check.ps1
Build context pack	.\tools\make-context.ps1 -Spec '5.1' -Files '<path>'

Appendix K - Quality audit (master prompt section 17)

#	Check	Where satisfied
1	Every Specification feature represented	Appendix G maps 27 requirement areas to steps.
2	Every UI/UX requirement represented	Phase 07, 08, 10, 14, 19; BA-01..BA-17 in steps 07.x, 09.3.3, 10.3.2, 13.1.2, 14.2.3, 16.x.
3	Four layout modes	07.1.1-07.1.6, 07.2.6.
4	Arabic and English	02.3.3, 03.1.4, 07.2.6, 19.1.2.
5	Stack matches specification	Standing Orders and 02.2.2; no Flutter/web.
6	Dependency order	Phase order and Overview diagram.
7	Actual computer	Phase 01, disk budget, Gradle memory settings.
8	Storage and internet	01.2.x, emulator skipped, offline builds.
9	Free AI sufficient	Appendix C, local Qwen fallback in every step.
10	Verification per step	Fields 12-13.
11	AI prompt per coding step	Field 10, assembled from step data.
12	Genuine links	Appendix H states which were seen vs stable addresses.
13	Difficult features kept	Phase 13, 15, 16, 20.
14	Unsupported formats documented	Appendix F, 09.3.4.
15	Security	Phase 06, 16, 21.2.2.
16	Offline tested	05.1.4, 21.2.1.
17	Portability tested	12.3.7.
18	Physical devices	Phases 02, 03, 10.3.4, 15-17, 21.
19	Tracker matches roadmap	Generated from the same data; validated by script.
20	No task initially complete	Validated: zero ticked boxes.
21	Readable files	PDF rendered and inspected.

This audit describes the plan, not an implementation. No claim of built or tested software is made.